

Tue, Sep 15 at 12:00 AM

look at the convo about spark to see what i learned and now lets move to teach me about databricks. i want to master it and know everything. even the syntax and debugging so that i am able to pass the exams and also build my projects

Worked for 35s

Yes. I'm going to treat the Spark work as the foundation rather than restart from zero.

From our recent Spark thread, you have already worked through the execution hierarchy, partitions, shuffle mechanics, Catalyst/AQE, statistics and size estimates, `df.explain(mode="extended") / "cost"`, Spark UI reasoning, and the physical join strategies—BHJ, SMJ, SHJ, and BNLJ. The main thing that still needed reinforcement was building an intuitive picture of what physically moves during shuffles and how that translates into join execution. I'll therefore keep revisiting those ideas inside real Databricks debugging cases rather than give you another isolated Spark course.

The timing is good because the **current Databricks Data Engineer Associate exam** explicitly tests platform architecture, Delta Lake, Unity Catalog, compute selection, ingestion, PySpark transformations, Lakeflow Jobs, CI/CD/Automation Bundles, Spark UI troubleshooting, skew/shuffling/spilling, Liquid Clustering, predictive optimization, OOM diagnosis, and governance/security.

🌐 Databricks +1

The Databricks mastery DAG

I would structure your learning like this:

🔗 Plain text

YOUR EXISTING SPARK KNOWLEDGE

- |
- ├ Spark execution model
- ├ partitions / shuffle
- ├ Catalyst / physical plans
- ├ joins
- ├ AQE
- ├ Spark UI / `explain()`
- └ DataFrame API

|

▼

PREREQUISITES — ROUND 1 ← WE DO THIS NOW

- |
- ├ compute vs storage
- ├ object storage
- ├ file ≠ table
- ├ columnar storage / Parquet
- ├ database metadata
- ├ ACID transactions
- ├ transaction logs
- └ data lake → lakehouse motivation

|

▼

PREREQUISITES — ROUND 2

- |
- └ ETL vs ELT

This is deliberately broader than merely memorizing the Associate exam. Databricks' present exam guide itself has moved toward genuinely operational topics such as diagnosing skew, shuffle/spill, OOMs, CI/CD, Lakeflow Jobs and governance.



Prerequisite Round 1

Storage, tables, transactions and why Databricks exists

Before touching the Databricks UI, I want one mental model to become automatic:

Spark computes. Storage persists. Delta gives stored files table semantics. Unity Catalog governs the objects. Databricks integrates these pieces into one platform.

That sentence will eventually explain a surprisingly large amount of Databricks.

1. Compute and storage are different things

Imagine this dataset:

</> Plain text

```
customers
orders
products
```

Something must **store** those bytes when nobody is processing them.

Something else must execute:

</> SQL

```
SELECT customer_id, SUM(amount)
FROM orders
GROUP BY customer_id;
```

Those are fundamentally different responsibilities.

Storage

Storage answers:

Where are the bytes after my program stops running?

Examples in cloud systems:

</> Plain text

```
Amazon S3
Azure Data Lake Storage
Google Cloud Storage
```

You can conceptualize storage as:

</> Plain text

```
s3://company-data/orders/...
abfss://data@company.dfs.core.windows.net/orders/...
gs://company-data/orders/...
```

Compute

Compute answers:

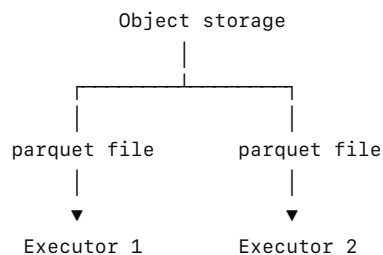
Which CPUs and RAM execute my code?

For Spark:

</> Plain text

```
Driver
|
+-- Executor
+-- Executor
+-- Executor
```

The executors might read files from object storage:

</> Plain text

The compute resources can disappear afterward.

The data remains.

This separation is central to modern lakehouse systems.

Databricks explicitly distinguishes its collaboration/execution environment from persistent cloud storage; workspaces contain artifacts and configurations, while persistent data typically resides in cloud storage. [Databricks Docu...](#) +1

Why is that useful?

Suppose you had a 10 TB dataset.

You might run:

</> Plain text

```
Monday:
20 machines

Tuesday:
```

```
0 machines

Wednesday:
100 machines
```

You do **not** want your 10 TB dataset to vanish because Tuesday has zero compute workers.

Storage and compute therefore have independent lifetimes.

Exercise 1

Your Spark cluster contains:

```
</> Plain text
```

```
driver
worker1
worker2
worker3
```

and reads:

```
</> Plain text
```

```
s3://shop/orders/
```

You terminate the cluster.

What happens to:

1. Executor RAM?
2. Spark shuffle files?
3. s3://shop/orders/ ?

Solution

Executor RAM disappears.

Temporary local Spark/shuffle data normally disappears with the compute environment.

The files in persistent object storage remain.

This distinction will become crucial when we later discuss **Databricks compute versus Delta tables**.

2. A file is not the same thing as a table

This distinction is exceptionally important.

Suppose storage contains:

</> Plain text

```
/orders/  
  part-00000.parquet  
  part-00001.parquet  
  part-00002.parquet
```

Those are files.

You might logically interpret them as:

</> Plain text

```
orders table
```

but the files themselves do not automatically provide full database-table behavior.

Consider:

</> Plain text

```
UPDATE orders  
SET status = 'cancelled'  
WHERE order_id = 123;
```

What does that mean for immutable-ish columnar files?

Perhaps `order_id=123` is inside:

</> Plain text

```
part-00001.parquet
```

You cannot think of this like modifying a cell in Excel.

Often the engine has to create a **new file version** containing the changed rows.

Something must then know:

</> Plain text

```
old file -> no longer part of table  
new file -> now part of table
```

That "something" is metadata/transaction state.

3. What Parquet really gives you

Suppose we have:

Plain text

id	name	age	country
1	Maria	25	GR
2	John	37	US
3	Anna	42	DE

A naive row-oriented mental representation would be:

Plain text

```
1, Maria, 25, GR
2, John, 37, US
3, Anna, 42, DE
```

A columnar representation conceptually groups values by column:

Plain text

```
id:
1
2
3

name:
Maria
John
Anna

age:
25
37
42

country:
GR
US
DE
```

Real Parquet is much more sophisticated—row groups, column chunks, pages, encodings, compression and metadata—but this first-principles picture explains why analytical engines like it.

Imagine:

SQL

```
SELECT AVG(age)
FROM people;
```

You primarily need:

Plain text

```
age
```

not necessarily:

```
</> Plain text
```

```
id
name
country
address
email
...
```

Columnar formats enable engines to avoid reading irrelevant columns.

This is called **column pruning**.

Later you will see plans such as:

```
</> Plain text
```

```
ReadSchema: struct<age:int>
```

even though the physical table may contain 100 columns.

That's not cosmetic.

It means:

Spark only requested the column necessary for the query.

Exercise 2

Table:

```
</> Plain text
```

```
sales(
  sale_id,
  customer_id,
  product_id,
  amount,
  country,
  timestamp,
  salesperson,
  comments
)
```

Query:

```
</> SQL
```

```
SELECT country, SUM(amount)
FROM sales
GROUP BY country;
```

Which columns are fundamentally needed from storage?

Solution

Only:

</> Plain text

```
country
amount
```

The remaining columns can potentially be pruned.

This reduces I/O.

4. Why a directory full of Parquet files is still insufficient

Imagine:

</> Plain text

```
/orders/
  A.parquet
  B.parquet
  C.parquet
```

Reader X begins reading:

</> Plain text

```
A
B
C
```

At exactly the same time, writer Y performs an update and changes the directory to:

</> Plain text

```
A
D
E
```

What should reader X see?

Possibilities include:


```
</> Plain text
```

```
A B C
```

or:

```
</> Plain text
```

```
A D E
```

But this would be dangerous:

```
</> Plain text
```

```
A B E
```

because that could represent no valid state of the table at all.

This is where database transactional semantics enter.

5. ACID

You will encounter **ACID** constantly in Databricks and Delta Lake.

Atomicity

A transaction happens:

```
</> Plain text
```

```
completely
```

or:

```
</> Plain text
```

```
not at all
```

Suppose a pipeline needs to replace 80 files.

You don't want another query to see:

```
</> Plain text
```

```
37 old files  
43 new files
```

because the operation happened to be halfway finished.

Atomicity means the commit becomes visible as one logical change.

Consistency

Transactions should take the database from one valid state to another valid state while respecting enforced invariants.

Don't confuse this with the distributed-systems meaning of "consistency" from CAP theory. Same English word; different concept.

Isolation

Concurrent operations shouldn't produce invalid interleavings.

For example:

⟨/⟩ Plain text

```
Reader R
Writer W
```

Ideally R sees a coherent snapshot:

⟨/⟩ Plain text

```
version 87
```

while W creates:

⟨/⟩ Plain text

```
version 88
```

not some half-87-half-88 dataset.

Durability

After a committed transaction succeeds, that committed state must survive subsequent process/compute failure.

6. Why Delta Lake exists

Traditional data lakes gained cheap scalable storage but historically lacked many of the guarantees expected from databases.

Databricks' core answer is **Delta Lake**.

Delta Lake extends Parquet data files with a **file-based transaction log**, providing ACID transactions and scalable metadata management while remaining compatible with Spark APIs. Delta is now the default table format for Databricks operations unless otherwise specified. [🌐 Databricks Docu...](#)

Conceptually:

</> Plain text

```
my_table/
|
├─ part-00000.parquet
├─ part-00001.parquet
├─ part-00002.parquet
|
└─ _delta_log/
    ...
```

Very simplified:

</> Plain text

```
PARQUET FILES
=
actual rows

_delta_log
=
which files constitute each valid table version
```

Think:

</> Plain text

```
version 0:
  add A
  add B

version 1:
  remove B
  add C

version 2:
  add D
```

Therefore:

</> Plain text

```
snapshot(version 0)
= A + B

snapshot(version 1)
= A + C

snapshot(version 2)
= A + C + D
```

Notice something subtle.

B.parquet might still physically exist for some time.

But the current Delta snapshot says:

</> Plain text

```
B is not part of the current table.
```

So:

Physical existence of a file $\neq$ logical membership in the current table.

This insight is essential for understanding:

</> Plain text

```
DELETE
UPDATE
MERGE
Time Travel
VACUUM
concurrent writes
```

later.

7. SQL operation versus physical files

Suppose:

</> SQL

```
DELETE FROM orders
WHERE order_id = 123;
```

A beginner imagines:

</> Plain text

```
find row 123
erase row
```

For file-oriented analytical storage, a better mental model is often:

</> Plain text

```
old file:

A.parquet
-----
120
121
123
124
125
```

Create replacement:

</> Plain text

```
B.parquet
-----
120
121
124
125
```

Transaction:

</> Plain text

```
REMOVE A.parquet
ADD B.parquet
```

The new table snapshot uses B.

Conceptually:

</> Plain text

```
before
  table -> A

commit

after
  table -> B
```

The operation looks like normal database DML:

</> SQL

```
DELETE
```

but underneath it may involve file-level rewriting and transaction metadata.

Later we'll complicate this because modern Delta has additional mechanisms and optimizations. But this model is the correct foundation.

Exercise 3

Suppose Delta log history conceptually says:

</> Plain text

```
v0:
```

```
ADD A
ADD B

v1:
REMOVE A
ADD C

v2:
ADD D

v3:
REMOVE B
```

Which files represent the current snapshot after v3 ?

Think before reading.

Solution

Start:

⟨/⟩ Plain text

```
v0:
A B
```

v1:

⟨/⟩ Plain text

```
remove A
add C

=> B C
```

v2:

⟨/⟩ Plain text

```
add D

=> B C D
```

v3:

⟨/⟩ Plain text

```
remove B

=> C D
```

Current logical table:

</> Plain text

C + D

8. Data lake versus data warehouse

Now we can understand the word **lakehouse** rather than memorize it.

Traditional simplification:

</> Plain text

DATA LAKE

Cheap/object storage
Files
Flexible schemas/formats
Huge raw datasets
ML/data science friendly

but historically weaker
transaction/table guarantees

versus:

</> Plain text

DATA WAREHOUSE

Structured tables
SQL
transactions
governance
BI
optimized analytical querying

but historically more rigid
and separated from raw/ML data

The lakehouse idea is roughly:

</> Plain text

Data lake characteristics
+
database/warehouse semantics
↓
LAKEHOUSE

Delta Lake is a major part of how Databricks gives object-storage data database-like table semantics. Databricks describes Delta as the optimized storage layer underpinning lakehouse tables. [Databricks Docu...](#)

This is why Databricks is **not simply "Spark with notebooks."**

The platform connects:

⟨⟩ Plain text

```
storage
+
transactional tables
+
Spark/Photon compute
+
governance
+
orchestration
+
SQL/BI
+
ML/AI
+
DevOps
```

9. Table metadata

Suppose you type:

⟨⟩ SQL

```
SELECT *
FROM prod.sales.orders;
```

How does the system know what `prod.sales.orders` means?

It needs metadata.

Conceptually:

⟨⟩ Plain text

```
table name:
orders

schema:
sales

catalog:
prod

columns:
order_id BIGINT
customer_id BIGINT
amount DECIMAL(10,2)

format:
DELTA
```



```
storage location:
s3://.../orders/

owner:
data_engineers

permissions:
...
```

Data itself might contain:

</> Plain text

```
123, 4921, 79.95
124, 8177, 20.30
```

Metadata describes the data.

This distinction will later make Unity Catalog very easy to understand.

In modern Databricks, Unity Catalog organizes governed objects using a three-level namespace:

</> Plain text

```
catalog.schema.object
```

and serves as the centralized governance layer for tables and other data/AI assets. [Databricks Docu...](#) +1

For example:

</> SQL

```
SELECT *
FROM ecommerce.silver.orders;
```

means conceptually:

</> Plain text

```
ecommerce  <- catalog
silver     <- schema
orders     <- table
```

Not:

</> Plain text

```
server.database.table
```

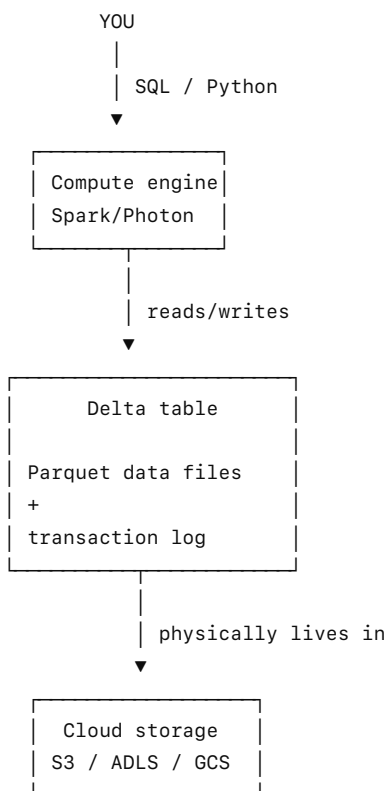
as you might mentally map from some traditional databases.

We'll study the exact semantics later.

10. One extremely important diagram

Keep this model:

Plain text



These things are related, but they are **not synonyms**.

This distinction becomes a frequent source of certification traps.

11. Relating this directly to Spark

This is where your existing knowledge connects.

You already know:

Python

```
df = spark.read.parquet(...)
```

and:

Python

```
df.write.parquet(...)
```

Spark is the **processing engine**.

Parquet is a **storage file format**.

Now add Delta:

```
</> Python
```

```
df.write.format("delta").save(...)
```

Conceptually:

```
</> Plain text
```

```
Spark
  ↓
produces Parquet data files
  +
updates Delta transaction metadata
```

And when using a governed table:

```
</> Python
```

```
df.write.saveAsTable("catalog.schema.orders")
```

another dimension appears:

```
</> Plain text
```

```
Unity Catalog object
  |
  ▼
Delta table
  |
  ▼
cloud storage
```

Do not memorize the syntax yet. We'll dissect each form later—including when these snippets are equivalent and when they absolutely are not.

12. Why deleting the Spark cluster does not mean deleting the table

Now answer this Databricks-style question.

You run:

```
</> Python
```

```
df.write.saveAsTable("prod.sales.orders")
```

Later you terminate the compute resource.

Does:

</> Plain text

```
prod.sales.orders
```

necessarily disappear?

No.

Because the compute environment and persistent data have different lifetimes.

This distinction sounds trivial now, but it explains a large class of Databricks architecture questions.

13. Control plane versus compute plane — first preview

There is one more concept I want in place before Round 2.

Databricks distinguishes roughly:

</> Plain text

```
CONTROL PLANE
```

from:

</> Plain text

```
COMPUTE PLANE
```

The control plane contains Databricks-managed backend/platform services, including the web application and platform coordination.

The compute plane is where your workload actually processes data.

For serverless workloads, Databricks operates serverless compute resources in a Databricks-managed serverless compute plane; classic compute uses cloud infrastructure associated with the customer cloud environment. [🌐 Databricks Docu...](#)

For now, think:

</> Plain text

```
Control plane:  
"What should run?"
```

```
Compute plane:  
"Actually run it."
```

We'll make this much more precise when we enter Databricks proper.

Your first debugging thought experiment

You execute:

</> Python

```
df = spark.read.table("sales.orders")

df.groupBy("country").sum("amount").show()
```

and it is extremely slow.

Which layer could be responsible?

A beginner might answer:

Spark.

That's insufficient.

Possible categories include:

</> Plain text

```
storage
    slow reads?

table layout
    too many tiny files?

Delta
    bad physical organization?

Spark
    enormous shuffle?

data
    severe skew?

compute
    insufficient resources?

query
    inefficient transformation?

statistics
    optimizer made bad choice?

network
    huge exchange?
```

Your Databricks debugging skill will eventually become:

Plain text

```
symptom
↓
identify layer
↓
collect evidence
↓
form hypothesis
↓
check Spark/Databricks metrics
↓
fix
↓
remeasure
```

—not:

Plain text

```
query slow
↓
increase cluster size
```

That distinction separates someone who merely uses Databricks from someone who can operate it professionally.

Round-1 challenge test

Try these before looking at the answers.

Q1

You have:

Plain text

```
part-0000.parquet
part-0001.parquet
part-0002.parquet
```

Does that automatically mean you have a Delta table?

Q2

What is the principal conceptual difference between:

Plain text

Spark

and:

</> Plain text

Delta Lake

Q3

Why can a Delta DELETE operation require creation of new Parquet files?

Q4

A Parquet file physically exists in storage but the current Delta snapshot no longer references it.

Is it logically part of the current table?

Q5

Why would:

</> SQL

```
SELECT AVG(age)
FROM people;
```

potentially read far less data from Parquet than a row-oriented representation?

Q6

What problem occurs if a reader sees some files from before a transaction and some files created during the transaction?

Which ACID idea is most directly involved?

Q7

Suppose:

</> Plain text

```
v0 ADD A
v0 ADD B
v1 REMOVE A
v1 ADD C
v2 ADD D
v3 REMOVE C
v3 ADD E
```

What is the current snapshot?

Q8

What does:

</> Plain text

company.finance.transactions

represent in the Unity Catalog naming model?

Q9

Which one normally executes:

```
</> Python
```

```
df.groupBy("country").sum("amount")
```

?

- A. Delta Lake
- B. Spark
- C. Unity Catalog
- D. Parquet

Q10

Which component primarily provides transactional state?

- A. Spark executor
- B. Parquet alone
- C. Delta transaction log
- D. Unity Catalog permission grants

Q11 — harder

Suppose Spark has physically written three replacement files, but the Delta transaction has **not committed**.

Should a reader of the current table suddenly see those three files just because they physically exist?

Q12 — harder

Why is this architecture useful?

```
</> Plain text
```

```
100 TB persistent dataset
```

```
08:00 → 50 compute workers  
12:00 → 200 compute workers  
18:00 → 0 compute workers
```

Q13 — interview question

Explain in one sentence why:

"Delta Lake is a database"

is an imprecise statement.

Q14 — debugging

A table occupies 5 TB in cloud storage, but your query needs two columns and rows from one date.

Name **two fundamentally different mechanisms** that could reduce what Spark has to read.

Q15 — critical thinking

Why isn't adding ACID semantics to an object store simply a matter of "store everything as Parquet"?

Answers

1. **No.** Plain Parquet files do not imply a Delta transaction log.
2. Spark is primarily a distributed computation engine; Delta Lake supplies transactional table/storage semantics over data files.
3. Because columnar data files are generally not manipulated as arbitrary individual mutable cells; a change may create replacement files and transaction metadata describing which files now belong to the table.
4. **No.** Logical membership is determined by the relevant table snapshot, not merely physical file existence.
5. Parquet is columnar, allowing Spark to perform column pruning and retrieve only necessary columns when possible.
6. The reader could observe an invalid partial state. Atomic transaction visibility/isolation are central to preventing this.
- 7.

</> Plain text

```
v0: A B
v1: B C
v2: B C D
v3: B D E
```

Therefore:

</> Plain text

```
B, D, E
```

8.

</> Plain text

```
company      catalog
finance      schema
transactions object/table
```

9. **B — Spark.**

10. **C — the Delta transaction log.**

11. **No.** Physical creation of files is not equivalent to publishing a committed table version.

12. Persistent storage can survive independently while compute capacity scales according to workload.

13. A strong answer:

Delta Lake is a transactional storage/table layer over files rather than a standalone traditional database server.

14. For example:

</> Plain text

```
column pruning
```

reduces the columns read, while data skipping/partition or clustering-related pruning can reduce the files/regions that need scanning. We'll distinguish these mechanisms rigorously later.

15. Parquet describes how data is stored in files; by itself it does not provide the transaction history, atomic commits, coherent snapshots and table-state management required for database-like concurrent modifications.

What you should be able to say after Round 1

Without notes, you should now be able to explain:

</> Plain text

```
Spark ≠ Databricks
Spark ≠ Delta
Delta ≠ Parquet
table ≠ file
table ≠ storage directory
storage ≠ compute
physical file existence ≠ current table membership
metadata ≠ data
Unity Catalog ≠ Delta Lake
```

If any of those still feel fuzzy, challenge me on them before we advance.

Current Databricks documentation confirms that Delta Lake is the default table format on Databricks, built around Parquet plus a transaction log, while Unity Catalog provides centralized governance and `catalog.schema.object` naming.

 Databricks Docu... +1 The official 2026 Associate exam specifically requires these architecture concepts before moving into ingestion, transformations, Jobs, CI/CD, performance troubleshooting and security.  Databricks

[Databricks: High-level architecture ↗](#)

[Databricks: What is Delta Lake? ↗](#)

[Databricks: Unity Catalog getting started ↗](#)

Do you want me to teach you the next part of prerequisites' DAG?

Tue, Sep 15 at 8:35 PM

yes. in greek all

Worked for 34s

Προαπαιτούμενα Databricks — Γύρος 2/2

Στον πρώτο γύρο χτίσαμε το storage model:

</> Plain text

Spark = υπολογισμός
Parquet = file format
Delta Lake = transactional table layer
Unity Catalog = metadata + governance
Cloud storage = μόνιμη αποθήκευση

Τώρα θέλω να χτίσουμε το **pipeline model**. Δηλαδή, πριν δεις κουμπιά και APIs του Databricks, πρέπει να καταλαβαίνεις τι πρόβλημα προσπαθεί να λύσει κάθε εργαλείο.

Η αλυσίδα αυτού του γύρου είναι:

</> Plain text

πηγή δεδομένων
↓
ETL / ELT
↓
batch — streaming — incremental processing
↓
CDC
↓
schema enforcement / schema evolution
↓
Bronze → Silver → Gold
↓
identities / permissions
↓
dev → test → prod
↓
CI/CD

Αν αυτά γίνουν καθαρά, μετά μπορούμε να μπούμε **κανονικά στο Databricks από το μηδέν μέχρι production/debugging/certification**.

1. ETL και ELT

Ας ξεκινήσουμε από ένα πραγματικό πρόβλημα.

Έχεις μια εταιρεία με:

</> Plain text

PostgreSQL
customers
orders

Shopify API
products

CSV files
marketing campaigns

και θέλεις να τα φέρεις όλα στο analytics system σου.

Υπάρχουν τρία θεμελιώδη βήματα:

</> Plain text

Extract
Transform
Load

Extract

Παίρνω δεδομένα από την πηγή.

</> Plain text

PostgreSQL
|
| SELECT ...
▼
raw orders

Transform

Τα μετασχηματίζω.

Για παράδειγμα:

</> Plain text

" Athens " → "ATHENS"

"19.95" → DECIMAL(10,2)

NULL age → validation / quarantine

customer_id + orders
↓
JOIN

Load

Τα γράφω στο σύστημα προορισμού.

Για παράδειγμα:

</> Plain text

Delta table
company.silver.orders

ETL

Παραδοσιακά:

</> Plain text

```
Extract
  ↓
Transform
  ↓
Load
```

Δηλαδή:

</> Plain text

```
source
  ↓
extract
  ↓
ETL server
  ↓
transform
  ↓
warehouse
```

Πρώτα μετασχηματίζεις και μετά φορτώνεις.

ELT

Στα σύγχρονα cloud data platforms είναι εξαιρετικά συνηθισμένο:

</> Plain text

```
Extract
  ↓
Load
  ↓
Transform
```

Δηλαδή:

</> Plain text

```
PostgreSQL
  ↓
raw data
  ↓
lakehouse
  ↓
Spark / SQL transformations
  ↓
clean tables
```

Γιατί;

Επειδή το Databricks μπορεί να αποθηκεύσει τεράστιες ποσότητες raw data φθηνά και να χρησιμοποιήσει distributed compute όταν χρειαστεί.

Άρα μπορείς πρώτα να κρατήσεις:

</> Plain text

ό,τι ακριβώς ήρθε από την πηγή

και μετά να το μετασχηματίσεις.

Αυτό συνδέεται άμεσα με τη Bronze layer που θα δούμε παρακάτω.

Η ίδια η Databricks χρησιμοποιεί πλέον πολύ ευρέως τον όρο ETL για data engineering pipelines, ακόμη και όταν η φυσική αρχιτεκτονική είναι αρκετά ELT-like. Γι' αυτό στις εξετάσεις μην κολλήσεις υπερβολικά στην ορολογία· κατανόησε κυρίως **πού γίνεται το transformation**. [Databricks Docu...](#) +1

Παράδειγμα

Έρχεται:

</> csv

```
order_id,customer_id,amount
101,7,"19.50"
102,12,"52.90"
```

ETL-like

</> Plain text

```
CSV
↓
parse
↓
validate
↓
convert amount → DECIMAL
↓
load clean table
```

ELT-like

</> Plain text

```
CSV
↓
store raw in Bronze
↓
later parse/validate
```



Silver Delta table

Το δεύτερο έχει σημαντικό πλεονέκτημα.

Αν αύριο ανακαλύψεις ότι:

</> Plain text

"19.50"

δεν ήταν EUR αλλά USD, έχεις ακόμα το original raw dataset.

Δεν χρειάζεται απαραίτητα να ξαναρωτήσεις το source system.

Άσκηση

Έχεις API που κάθε βράδυ σου επιστρέφει JSON.

Θέλεις:

1. να μην χάσεις ποτέ το original response,
2. να αλλάζεις αργότερα τους transformation rules,
3. να μπορείς να ξαναφτιάξεις τα downstream tables.

Ποια λογική ταιριάζει περισσότερο;

</> Plain text

Extract → Transform → Load

ή:

</> Plain text

Extract → Load raw → Transform

Απάντηση

Η δεύτερη.

Κρατάς πρώτα immutable/raw representation και χτίζεις downstream datasets από αυτή.

2. Batch processing

Ας πούμε ότι σήμερα έχεις:

</> Plain text

orders.csv

```
1 GB
1,000,000 rows
```

και γράφεις:

```
</> Python

df = spark.read.csv("/data/orders.csv")

result = (
    df.groupBy("country")
        .count()
)

result.write.saveAsTable("sales_per_country")
```

Το dataset έχει σαφές τέλος.

```
</> Plain text

row 1
row 2
...
row 1,000,000
END
```

Αυτό είναι **bounded dataset**.

Κάνεις:

```
</> Plain text

START
↓
διάβασε όλο το input
↓
υπολόγισε
↓
γράψε output
↓
STOP
```

Αυτό είναι batch processing.

3. Streaming processing

Τώρα σκέψου Kafka:

```
</> Plain text

12:00 order 101
12:00 order 102
```



```
12:01 order 103
12:02 order 104
...
...
...
```

Δεν υπάρχει φυσικό:

</> Plain text

END OF FILE

Το dataset είναι λογικά **unbounded**.

</> Plain text

```
events
events
events
events
...
∞
```

Θέλεις το computation να συνεχίζει καθώς φτάνουν νέα δεδομένα.

Αυτό είναι streaming.

Στο Spark Structured Streaming, εκφράξεις συχνά transformations σχεδόν όπως στα batch DataFrames, αλλά το engine ενημερώνει το αποτέλεσμα καθώς φτάνουν δεδομένα. Η Databricks το περιγράφει ως near-real-time processing με incremental computation και fault-tolerance. [Databricks Docu...](#)

Πολύ σημαντικό: streaming ≠ επεξεργασία ενός row κάθε φορά

Αυτό είναι συχνό λάθος.

Μπορεί να φαντάζεσαι:

</> Plain text

```
event1 → process
event2 → process
event3 → process
```

Το Spark Structured Streaming συχνά χρησιμοποιεί **micro-batches**.

Για παράδειγμα:

</> Plain text

```
events:
1 2 3 4 5
```

↓

```
microbatch 1:
1 2

microbatch 2:
3 4

microbatch 3:
5
```

Κάθε μικρό batch περνά από distributed Spark execution.

Άρα συνδέεται με όσα ήδη μάθαμε:

⌕ Plain text

```
microbatch
  ↓
Spark job
  ↓
stages
  ↓
tasks
  ↓
partitions
  ↓
possible shuffle
```

To streaming **δεν ακυρώνει** το execution model του Spark.

Χτίζει πάνω του.

4. Τι είναι incremental processing;

Εδώ υπάρχει σημαντική διάκριση:

Streaming και incremental processing δεν είναι το ίδιο πράγμα.

Έστω ότι έχεις:

⌕ Plain text

```
Day 1
100 GB

Day 2
+2 GB

Day 3
+3 GB
```

Ένας αφελής pipeline μπορεί κάθε ημέρα να ξανακάνει:

⟨/⟩ Plain text

```
Day 1:  
process 100 GB
```

```
Day 2:  
process 102 GB
```

```
Day 3:  
process 105 GB
```

Αλλά γιατί;

Την Day 3 ίσως χρειάζεσαι μόνο:

⟨/⟩ Plain text

```
new = 3 GB
```

Incremental processing σημαίνει περίπου:

⟨/⟩ Plain text

```
Previous state  
+  
new/changed input  
=  
new state
```

και όχι:

⟨/⟩ Plain text

```
recompute everything from zero
```

Batch μπορεί να είναι incremental

Αυτό είναι σημαντικό.

Κάθε βράδυ στις 02:00 μπορείς να πεις:

⟨/⟩ SQL

```
SELECT *  
FROM transactions  
WHERE transaction_date = CURRENT_DATE - 1;
```

Ο pipeline τρέχει μια φορά τη μέρα.

Άρα είναι:

</> Plain text

```
batch
```

αλλά επεξεργάζεται μόνο:

</> Plain text

```
new rows
```

άρα είναι επίσης:

</> Plain text

```
incremental
```

Streaming συνήθως είναι incremental

Streaming:

</> Plain text

```
checkpoint says:
```

```
processed up to event 100000
```

έρχονται:

</> Plain text

```
100001
```

```
100002
```

```
100003
```

και επεξεργάζεσαι αυτά.

Δεν ξαναδιαβάζεις αναγκαστικά τα προηγούμενα εκατό χιλιάδες.

Άσκηση

Έχεις:

</> Plain text

```
1 TB historical orders
```

Κάθε βράδυ έρχονται:

</> Plain text

```
10 MB new orders
```

Κάνεις job στις 03:00 μόνο για αυτά τα 10 MB.

Τι είναι;

- A. streaming only
- B. batch only
- C. batch + incremental
- D. streaming + full refresh

Απάντηση

C. batch + incremental.

5. Checkpoint: γιατί χρειάζεται;

Φαντάσου streaming pipeline.

Έχει επεξεργαστεί:

</> Plain text

```
events 1 → 900
```

και πέφτει ο server.

Ξανασηκώνεται.

Από πού συνεχίζει;

Χρειάζεται κάπου να υπάρχει state όπως:

</> Plain text

```
I've already processed until offset 900.
```

Αυτό είναι ένας από τους ρόλους του **checkpoint**.

Conceptually:

</> Plain text

```
SOURCE
```

```
1 2 3 ... 899 900 901 902
```

```
      ↑
    checkpoint
    "900"
```

```
restart
```

```
process:  
901  
902  
...
```

Χωρίς checkpoint, το engine δεν ξέρει εύκολα αν πρέπει:

```
</> Plain text
```

```
να ξεκινήσει από 1
```

ή:

```
</> Plain text
```

```
να συνεχίσει από 901
```

Θα μάθουμε checkpoint internals πολύ βαθύτερα όταν φτάσουμε Structured Streaming / Auto Loader.

6. CDC — Change Data Capture

Αυτό είναι εξαιρετικά σημαντικό για Data Engineering και Databricks.

Ας έχουμε operational PostgreSQL:

```
</> Plain text
```

```
customers
```

```
id | name  | city  
---+-----+-----  
1  | Maria | Athens  
2  | John  | Patras
```

Στις 10:00:

```
</> Plain text
```

```
Maria moves Athens → Volos
```

Άρα:

```
</> SQL
```

```
UPDATE customers  
SET city = 'Volos'  
WHERE id = 1;
```

Πώς θα το μάθει το lakehouse;

Λύση 1: Ξαναπάρε όλο τον πίνακα

</> Plain text

```
10:00
SELECT * FROM customers

11:00
SELECT * FROM customers

12:00
SELECT * FROM customers
```

Αν έχει:

</> Plain text

```
2 billion rows
```

αυτό είναι τραγικά ακριβό.

Λύση 2: στείλε μόνο τις αλλαγές

Η πηγή μπορεί να παράγει:

</> Plain text

```
UPDATE id=1 city=Volos
```

Αυτό είναι **Change Data Capture (CDC)**.

Ένα CDC stream μπορεί να μοιάζει εννοιολογικά έτσι:

</> Plain text

op	id	name	city	seq
I	1	Maria	Athens	10
I	2	John	Patras	11
U	1	Maria	Volos	12
D	2	John	null	13

όπου:

</> Plain text

```
I = INSERT
```

```
U = UPDATE
D = DELETE
```

και:

```
</> Plain text

seq
```

μας λέει τη σειρά των αλλαγών.

Γιατί το sequence είναι σημαντικό;

Έστω ότι φτάσουν:

```
</> Plain text

UPDATE city = Volos
sequence = 17

UPDATE city = Larisa
sequence = 18
```

αλλά λόγω distributed network φτάσουν στο lakehouse με σειρά:

```
</> Plain text

18
17
```

Αν κάνεις απλώς:

```
</> Plain text

last event received wins
```

θα καταλήξεις λανθασμένα σε:

```
</> Plain text

Volos
```

ενώ το σωστό latest state είναι:

```
</> Plain text

Larisa
```

Άρα το CDC χρειάζεται έννοια:

</> Plain text

ordering / sequencing

και συνήθως:

</> Plain text

primary/business key

ώστε να ξέρεις ποια οντότητα αλλάζει.

CDC και SCD Type 1

Θέλεις μόνο την τρέχουσα κατάσταση.

</> Plain text

customer_id = 1

Maria | Athens
↓ UPDATE

Maria | Volos

Τελικά κρατάς:

</> Plain text

Maria | Volos

και χάνεις το Athens.

Αυτό είναι η βασική ιδέα του:

Slowly Changing Dimension Type 1.

CDC και SCD Type 2

Μερικές φορές θέλεις ιστορικό.

</> Plain text

customer	city	from	to
Maria	Athens	2024-01-01	2026-08-10
Maria	Volos	2026-08-10	NULL

Τώρα μπορείς να ρωτήσεις:

Plain text

Πού έμενε η Μαρία στις 1/7/2025;

Αυτό είναι η βασική ιδέα του:

SCD Type 2.

Η σημερινή Databricks παρέχει `AUTO CDC` και `AUTO CDC FROM SNAPSHOT` στα Lakeflow pipelines και μπορεί να διαχειριστεί SCD Type 1 και Type 2. Τα παλαιότερα `APPLY CHANGES` APIs εξακολουθούν να υπάρχουν, αλλά η τρέχουσα τεκμηρίωση προτείνει `AUTO CDC` . [Databricks Docu...](#)

Αυτό είναι σημαντικό και για παλαιότερα courses/exam questions: μπορεί να συναντήσεις ακόμα την παλαιά ορολογία.

Άσκηση

Έρχονται:

Plain text

```
seq 20: INSERT id=4 salary=1000
seq 21: UPDATE id=4 salary=1300
seq 22: UPDATE id=4 salary=1500
```

SCD1

Τι μένει;

Plain text

```
id=4 salary=1500
```

SCD2

Τι θέλουμε εννοιολογικά;

Plain text

id	salary	valid_from	valid_to
4	1000	...	
4	1300	...	
4	1500	...	current

Δηλαδή ιστορικό εκδόσεων.

7. Schema

Αυτό πρέπει να το δεις ως **contract**.

Έστω:

</> Plain text

```
orders
```

```
order_id    BIGINT
customer_id BIGINT
amount      DECIMAL
country     STRING
```

Αυτό:

</> Plain text

```
column names
+
column types
```

είναι το schema.

Schema enforcement

Έχεις target:

</> Plain text

```
amount DECIMAL
```

και κάποιος προσπαθεί να γράψει:

</> Plain text

```
amount = {"foo": 12}
```

ή:

</> Plain text

```
unknown_column
```

Θέλεις να πεις:

Δεν δέχομαι αυθαίρετα δεδομένα που δεν συμβαδίζουν με το contract.

Αυτό είναι **schema enforcement**.

Για Delta tables η Databricks κάνει schema validation κατά το write και απορρίπτει ασύμβατα δεδομένα, αν δεν έχεις ενεργοποιήσει κάποια επιτρεπτή διαδικασία schema evolution. [🌐 Databricks Docu...](#)

Γιατί είναι απαραίτητο;

Φαντάσου pipeline:

</> Plain text

```
API
↓
Silver
↓
Gold
↓
Power BI
```

Το Power BI περιμένει:

</> Plain text

```
amount: DECIMAL
```

Ξαφνικά upstream producer στέλνει:

</> Plain text

```
amount: STRING
```

με:

</> Plain text

```
"unknown"
```

Αν απλώς δεχτείς οτιδήποτε:

</> Plain text

```
upstream schema bug
↓
Silver corrupted
↓
Gold wrong
↓
dashboard broken
```

Το schema enforcement μπορεί να σταματήσει το πρόβλημα νωρίς.

Schema evolution

Αλλά τα schemas αλλάζουν νόμιμα.

Σήμερα:

</> Plain text

```
orders
```

```
id
```

```
amount
```

Αύριο το source system προσθέτει:

</> Plain text

```
discount_code
```

Άρα:

</> Plain text

```
id
```

```
amount
```

```
discount_code
```

Δεν θέλεις πάντα:

</> Plain text

```
FAIL FOREVER
```

Θέλεις ίσως το schema να εξελιχθεί με ελεγχόμενο τρόπο.

Αυτό είναι **schema evolution**.

Η Databricks αναφέρει ως συνηθισμένες εξελίξεις νέα columns, rename/drop columns και type widening ή άλλες αλλαγές τύπων. [Databricks Docu...](#)

Enforcement και evolution ΔΕΝ είναι αντίθετα

Αυτό είναι exam-worthy.

Μπορείς να έχεις:

</> Plain text

```
schema enforcement
```

```
+
```

```
controlled schema evolution
```

Δηλαδή:

Από default προστατεύω το schema, αλλά επιτρέπω συγκεκριμένες μεταβολές με συγκεκριμένους κανόνες.

Δεν είναι:

⟨/⟩ Plain text

```
enforcement OR evolution
```

Είναι συχνά:

⟨/⟩ Plain text

```
enforcement
+
explicit evolution policy
```

Παράδειγμα debugging

Χθες ο pipeline δούλεψε.

Σήμερα:

⟨/⟩ Plain text

```
AnalysisException
schema mismatch
```

Δεν πρέπει αμέσως να γράψεις:

⟨/⟩ Plain text

```
enable schema evolution everywhere
```

Πρώτα πρέπει να ρωτήσεις:

⟨/⟩ Plain text

```
Άλλαξε πραγματικά το source schema;
|
├─ ναι → intentional;
|   │
|   ├── yes → evolve carefully
|   └─ no  → source bug
|
└─ όχι → parsing / type inference bug;
```

Αυτό είναι production thinking.

Άσκηση

Target:

⟨/⟩ Plain text

```
customer_id BIGINT
```

```
age INT
country STRING
```

Source ξαφνικά:

```
</> Plain text

customer_id BIGINT
age INT
country STRING
loyalty_points INT
```

Τι συνέβη;

Schema evolution candidate: add column.

Δεν σημαίνει αυτομάτως ότι πρέπει να το αποδεχθείς.

Πρέπει πρώτα να επιβεβαιώσεις ότι η αλλαγή είναι αναμενόμενη.

8. Medallion architecture

Τώρα όλα τα προηγούμενα ενώνονται.

Η Databricks προτείνει ευρέως το pattern:

```
</> Plain text

Bronze
  ↓
Silver
  ↓
Gold
```

Δεν είναι υποχρεωτικό· είναι design pattern. Η τρέχουσα τεκμηρίωση το περιγράφει ως progressively increasing data quality από Bronze προς Silver προς Gold. [Databricks Docu...](#) +1

Bronze

Σκοπός:

Κράτα όσο γίνεται πιστά αυτό που ήρθε από την πηγή.

Παράδειγμα:

```
</> Plain text

Kafka
  ↓

Bronze orders
```

```
{  
  "id": "00192",  
  "amount": "25.98",  
  "customer": "  MARIA ",  
  "event_time": "..."  
}
```

Μπορεί να έχει:

</> Plain text

```
duplicates  
nulls  
bad formatting  
incorrect cases  
unexpected fields
```

Δεν μας πειράζει απαραίτητα.

Το Bronze είναι πολύ κοντά στην **source truth**.

Γιατί να κρατήσουμε "βρώμικα" δεδομένα;

Έστω ότι σήμερα ο transformation rule σου ήταν λάθος:

</> Python

```
amount = amount * 100
```

και δημιούργησες λανθασμένο Silver.

Αν έχεις Bronze:

</> Plain text

```
Bronze  
↓  
correct code  
↓  
rebuild Silver
```

Αν δεν έχεις raw data:

</> Plain text

```
???
```

ίσως πρέπει να ξαναζητήσεις τα δεδομένα από την πηγή.

Και ίσως η πηγή να μην τα έχει πια.

Silver

Σκοπός:

cleaned, validated, usable data.

Εδώ κάνουμε συνήθως:

</> Plain text

```
parsing
casting
deduplication
null handling
quality checks
CDC application
joins
normalization
business-key validation
```

Παράδειγμα:

Bronze:

</> Plain text

```
"id": "00192"
"customer": " MARIA "
"amount": "25.98"
```

Silver:

</> Plain text

```
id      BIGINT  192
customer STRING  "MARIA"
amount  DECIMAL 25.98
```

Η Databricks περιγράφει το Silver ως cleansed/validated/refined layer που μπορεί να χρησιμοποιηθεί από data analysts και data scientists. [Databricks Docu...](#) +1

Gold

Gold σημαίνει:

data shaped for a specific business use.

Για παράδειγμα:

</> Plain text

```
gold.daily_sales
```

date	country	revenue
2026-09-14	GR	190000
2026-09-14	DE	310000

ή:

```
</> Plain text
```

```
customer_lifetime_value
```

ή:

```
</> Plain text
```

```
monthly_revenue_by_product
```

Δεν είναι απλώς:

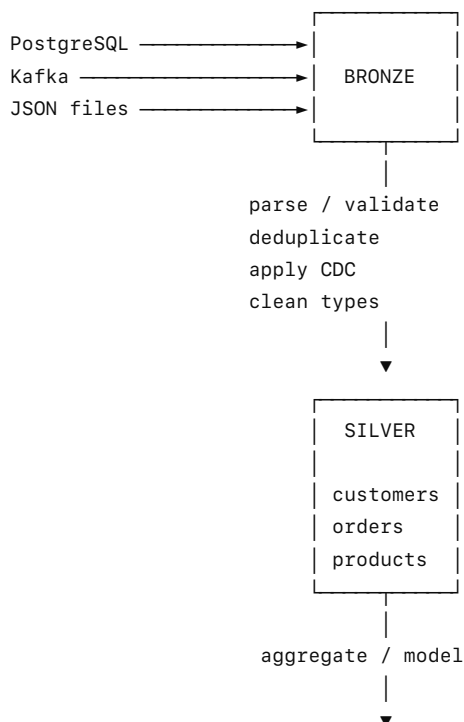
Silver αλλά πιο καθαρό.

To Gold είναι συχνά **consumer/business oriented**.

Ολόκληρο example

Έστω e-shop.

```
</> Plain text
```



GOLD

Μια σημαντική παρανόηση

Μην σκέφτεσαι:

</> Plain text

```
Bronze = CSV
Silver = Parquet
Gold = database
```

Λάθος.

Και τα τρία μπορεί να είναι:

</> Plain text

```
Delta tables
```

Η διαφορά είναι **semantic / quality / purpose**, όχι αναγκαστικά file format.

Η σημερινή Databricks προτείνει μάλιστα Lakeflow pipelines όπου Bronze μπορεί να χρησιμοποιεί streaming tables, Silver streaming tables ή materialized views ανάλογα με το transformation και Gold materialized views για aggregations/metrics.

 Databricks Docu...

Άσκηση

Πού θα έβαζες τα παρακάτω;

</> Plain text

- A. raw JSON exactly as received
- B. orders with validated types and duplicates removed
- C. monthly revenue by country
- D. customer table after applying CDC
- E. malformed records captured for investigation

Απάντηση

</> Plain text

- A → Bronze
- B → Silver
- C → Gold

D → Silver
E → συνήθως Bronze/quarantine layer

Το E εξαρτάται από την αρχιτεκτονική, αλλά **δεν το εξαφανίζεις σιωπηλά**.

9. Identity: ποιος είσαι;

Πηγαίνουμε τώρα στην ασφάλεια.

Πρέπει πρώτα να χωρίσεις δύο έννοιες.

</> Plain text
Authentication

και:

</> Plain text
Authorization

Authentication

Ποιος είσαι;

Για παράδειγμα:

</> Plain text
nikos@company.com

κάνει login.

Το σύστημα αποδεικνύει:

</> Plain text
αυτός είναι πράγματι ο Nikos

Authorization

Μετά ρωτάμε:

Τι επιτρέπεται να κάνει ο Nikos;

Για παράδειγμα:

</> Plain text

```
SELECT customers      YES
DELETE customers      NO
CREATE TABLE         NO
```

Αυτό είναι authorization.

10. Principal

Στο Databricks θα βλέπεις συνεχώς τον όρο:

principal

Principal είναι μια identity στην οποία μπορούμε να δώσουμε permissions.

Σημαντικές κατηγορίες:

```
</> Plain text

user
group
service principal
```

Η Unity Catalog χρησιμοποιεί principals όπως users, groups και service principals για grants πάνω σε securable objects.

 Databricks Docu... [+1](#)

User

Άνθρωπος.

```
</> Plain text

nikos@company.com
```

Group

Σύνολο principals.

```
</> Plain text

data_engineers
```

με:

```
</> Plain text

Nikos
Maria
John
```

αντί να κάνεις:

</> SQL

```
GRANT ... TO Nikos;
GRANT ... TO Maria;
GRANT ... TO John;
```

κάνεις:

</> SQL

```
GRANT ... TO `data_engineers`;
```

και διαχειρίζεσαι membership.

Η Databricks συνιστά access management μέσω groups αντί για individual users όπου είναι δυνατό. [Databricks Docu...](#) +1

Service principal

Αυτό είναι πολύ σημαντικό.

Ένα production job:

</> Plain text

02:00 κάθε βράδυ

δεν πρέπει ιδανικά να "τρέχει σαν τον Νίκο".

Γιατί;

Τι γίνεται αν:

</> Plain text

Nikos leaves company

και το account απενεργοποιηθεί;

Θέλεις machine identity:

</> Plain text

orders-etl-prod

Αυτό είναι service principal.

Conceptually:

⟨/⟩ Plain text

```
human
  ↓
user identity

automation
  ↓
service principal
```

Θα το ξαναδούμε σε Jobs και CI/CD.

11. Securable object

Έχεις:

⟨/⟩ Plain text

```
catalog
schema
table
volume
function
...
```

Αν μπορείς να ελέγξεις permission πάνω του, είναι **securable object**.

Παράδειγμα:

⟨/⟩ Plain text

```
catalog:
company

schema:
company.finance

table:
company.finance.payroll
```

12. Privilege

Privilege σημαίνει:

τι action επιτρέπεται να εκτελέσει principal πάνω σε securable object.

Παράδειγμα:

⟨/⟩ SQL

```
GRANT SELECT
ON TABLE company.finance.transactions
```

```
TO `analysts`;
```

Διάβασέ το κυριολεκτικά:

</> Plain text

```
GRANT
  ↓
δώσε

SELECT
  ↓
το δικαίωμα ανάγνωσης

ON TABLE ...
  ↓
πάνω σε αυτό το securable

TO analysts
  ↓
σε αυτό το principal
```

Αυτό το grammatical parsing είναι χρήσιμο για να μην αποστηθίζεις SQL μηχανικά.

Για SELECT συνήθως χρειάζεσαι και το path

Έχεις:

</> Plain text

```
company.finance.transactions
```

Conceptually περνάς:

</> Plain text

```
company
  ↓
finance
  ↓
transactions
```

Άρα permissions μπορεί να περιλαμβάνουν:

</> SQL

```
GRANT USE CATALOG
ON CATALOG company
TO `analysts`;

GRANT USE SCHEMA
ON SCHEMA company.finance
TO `analysts`;
```



```
GRANT SELECT
ON TABLE company.finance.transactions
TO `analysts`;
```

Θα μελετήσουμε αργότερα privilege inheritance και τις ακριβείς Unity Catalog semantics.

Permission debugging

Έστω:

</> SQL

```
SELECT *
FROM prod.finance.transactions;
```

και παίρνεις:

</> Plain text

```
PERMISSION_DENIED
```

Κακό debugging:

</> Plain text

```
"δώστε μου admin"
```

Σωστό debugging:

</> Plain text

```
Can principal access workspace?
↓
Can it USE CATALOG prod?
↓
Can it USE SCHEMA finance?
↓
Does it have SELECT on table?
↓
Are there row filters / masks / policies?
↓
Which identity is ACTUALLY running?
```

Το τελευταίο είναι κρίσιμο.

Εσύ μπορεί να έχεις SELECT.

Αλλά το Job ίσως τρέχει ως:

```
</> Plain text
```

```
orders-prod-service-principal
```

και εκείνο να μην έχει.

13. Least privilege

Βασική security αρχή:

Δίνω μόνο τα permissions που είναι απαραίτητα.

Αν ένας analyst χρειάζεται:

```
</> Plain text
```

```
SELECT
```

δεν του δίνω:

```
</> Plain text
```

```
ALL PRIVILEGES
```

"για να δουλέψει πιο εύκολα".

Η Databricks προειδοποιεί ειδικά να χρησιμοποιούνται με φειδώ ισχυρά privileges όπως `ALL PRIVILEGES`, `MANAGE` και `ownership`. [🌐 Databricks Docu...](#)

14. Dev, test/staging, prod

Τώρα software engineering.

Φαντάσου ότι έχεις production table:

```
</> Plain text
```

```
prod.gold.monthly_revenue
```

και dashboard του CEO.

Θέλεις να δοκιμάσεις:

```
</> Python
```

```
.filter(col("country") != "GR")
```

για debugging.

Θα το αλλάξεις απευθείας στο production notebook και θα πατήσεις Run All;

Όχι.

Θέλουμε environment isolation.

</> Plain text

```
DEV
|
| code changes
▼
TEST / STAGING
|
| validation
▼
PROD
```

DEV

Εδώ δουλεύει ο developer.

Μπορεί:

</> Plain text

- να σπάσει πράγματα
- να δοκιμάσει
- να κάνει debug
- να χρησιμοποιήσει sample data

Παράδειγμα:

</> Plain text

```
dev_nikos.silver.orders
```

TEST / STAGING

Πιο κοντά στο production.

Εδώ ελέγχεις:

</> Plain text

- permissions
- dependencies
- configuration
- integration
- data quality
- deployment

χωρίς να επηρεάζεις πραγματικούς consumers.

PROD

Εδώ εκτελούνται πραγματικά workloads.

Θέλεις:

</> Plain text

```
controlled deployments
stable configs
monitoring
restricted permissions
reproducibility
```

Το μεγάλο λάθος

</> Plain text

```
Notebook in browser
  ↓
edit production code manually
  ↓
Run All
```

Αυτό δεν είναι σοβαρό production deployment model.

Γιατί μετά ρωτάς:

</> Plain text

```
Ποια version τρέχει;

Ποιος την άλλαξε;

Πότε;

Τι άλλαξε;

Πώς επιστρέφω στην προηγούμενη version;

Το ίδιο code πέρασε tests;
```

και δεν έχεις αξιόπιστη απάντηση.

15. Source control

Για αυτό χρησιμοποιούμε Git.

</> Plain text

```
developer
  ↓
branch
  ↓
commit
  ↓
pull request
  ↓
review
  ↓
main
```

Άρα το source of truth για code γίνεται:

</> Plain text

Git repository

και όχι:

</> Plain text

"εκείνο το notebook που έχει ο Νίκος στο workspace"

16. CI

Continuous Integration

Ας κάνεις:

</> Plain text

git push

Το σύστημα μπορεί αυτόματα να κάνει:

</> Plain text

```
lint
  ↓
unit tests
  ↓
integration tests
  ↓
configuration validation
  ↓
build
```

Αν κάτι αποτύχει:

</> Plain text

✖ μην κάνεις merge/deploy

17. CD

CD μπορεί να σημαίνει Continuous Delivery ή Continuous Deployment ανάλογα με τη διαδικασία.

Για εμάς η βασική ιδέα είναι:

</> Plain text

```
validated artifact/code
  ↓
automated controlled deployment
  ↓
target environment
```

Για παράδειγμα:

</> Plain text

```
main branch
  ↓
tests
  ↓
deploy staging
  ↓
integration tests
  ↓
deploy production
```

18. Infrastructure as Code

Τώρα κάτι πολύ σημαντικό για Databricks.

Έστω ότι χρειάζεσαι Job με:

</> Plain text

```
task A
task B
schedule
compute
parameters
permissions
```

Μπορείς να τα δημιουργήσεις manually από UI.

Αλλά σε production είναι συχνά καλύτερα να τα δηλώσεις ως code/configuration.

Conceptually:

</> YAML

```
job:
  name: orders_pipeline

  schedule:
    ...

  tasks:
    - ingest
    - transform
```

Γιατί;

Επειδή τώρα το configuration μπορεί:

</> Plain text

```
να μπει σε Git
να γίνει review
να έχει version history
να αναπαραχθεί
να γίνει deploy αυτόματα
```

Αυτό είναι μέρος της φιλοσοφίας:

Infrastructure as Code / configuration as code.

19. Το Databricks εργαλείο που θα μάθουμε αργότερα

Το σημερινό όνομα είναι:

Declarative Automation Bundles

Παλαιότερα:

Databricks Asset Bundles.

Αν δεις tutorials που λένε Asset Bundles, μην νομίζεις ότι μιλάνε για εντελώς άλλο concept.

Η Databricks τα προτείνει πλέον ως βασική προσέγγιση για CI/CD και Infrastructure-as-Code-style deployment Databricks projects. Ένα bundle μπορεί να περιγράψει jobs, pipelines, source files, tests και target environments. [Databricks Docu...](#) +1

Το κεντρικό αρχείο που θα μάθουμε είναι συχνά:

</> Plain text

```
databricks.yml
```

και μπορεί να έχει targets όπως:

</> YAML

```
targets:

  dev:
    mode: development

  prod:
    mode: production
```

Η current Databricks υποστηρίζει ρητά development και production deployment modes. [Databricks Docu...](#) +1

Δεν θέλω ακόμα να αποστηθίσεις τη syntax.

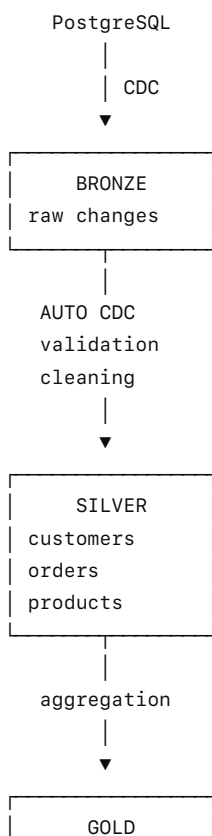
Θα την μάθουμε γραμμή-γραμμή όταν φτάσουμε στα Bundles.

20. Τώρα σύνδεσέ τα όλα

Ας δούμε ολόκληρο production data system.

Έχουμε e-commerce application:

</> Plain text



Τώρα πρόσθεσε execution:

</> Plain text

Lakeflow / Spark



processing

Governance:

</> Plain text

Unity Catalog



├─ who?

├─ what object?

└─ what privilege?

Development:

</> Plain text

Git



DEV



tests



STAGING



PROD

Deployment:

</> Plain text

Declarative Automation Bundle

+

CI/CD system

Storage:

</> Plain text

Delta tables



cloud object storage

Αυτό είναι ήδη ένα αρκετά σωστό νοητικό μοντέλο ολόκληρης της Databricks data-engineering πλατφόρμας.

21. Ένα σημαντικό νέο σημείο για το 2026

Θα συναντήσεις tutorials με διάφορα ονόματα:

</> Plain text

```
Delta Live Tables
DLT
Lakeflow Declarative Pipelines
Spark Declarative Pipelines
Lakeflow pipelines
```

Η ορολογία έχει εξελιχθεί.

Στη σημερινή τεκμηρίωση, τα **Lakeflow pipelines** χτίζονται πάνω στο **Apache Spark Declarative Pipelines (SDP)** και υποστηρίζουν batch και streaming pipelines σε SQL/Python, με automatic dependency orchestration και incremental processing. [Databricks Docu...](#) +2

Θα σου μάθω τη **σημερινή syntax**, αλλά θα σου δείχνω και τα παλιά ονόματα όταν είναι σημαντικά για:

</> Plain text

```
παλαιότερα tutorials
Stack Overflow
παλιές εξετάσεις
existing production systems
```

22. Debugging case 1 — Δεν εμφανίζονται νέα δεδομένα

Έχεις:

</> Plain text

```
source files
↓
bronze
↓
silver
↓
gold
```

Το source έχει σημερινά δεδομένα.

Το Gold όχι.

Μην πεις:

Databricks bug.

Σκέψου pipeline.

</> Plain text

```
Source:
υπάρχουν τα νέα files;
↓
Bronze:
```

```
ingested;
  ↓
checkpoint:
προχώρησε;
  ↓
Silver:
transformation rejected rows;
  ↓
schema:
άλλαξε;
  ↓
CDC:
sequence/key correct;
  ↓
Gold:
refresh/job ran;
```

Έτσι θα κάνουμε debugging σε όλη τη σειρά μαθημάτων:

Βρες πρώτα το πρώτο layer στο οποίο η πραγματικότητα αποκλίνει από αυτό που περιμένες.

23. Debugging case 2 — Διπλές εγγραφές

Έρχεται:

</> Plain text

```
order 123
order 123
```

Πιθανές αιτίες είναι πολύ διαφορετικές:

</> Plain text

```
source produced duplicate

file ingested twice

checkpoint reset

retry semantics

CDC key incorrect

deduplication missing

MERGE condition incorrect
```

Άρα:

</> Plain text

```
"έχω duplicates"
```

δεν είναι root cause.

Είναι symptom.

24. Debugging case 3 — Works in dev, fails in prod

</> Plain text

DEV

✓ success

PROD

✗ PERMISSION_DENIED

Πιθανόν ο κώδικας να είναι τέλειος.

Μπορεί να διαφέρει:

</> Plain text

identity

permissions

catalog

schema

storage credential

configuration

secret

compute

runtime

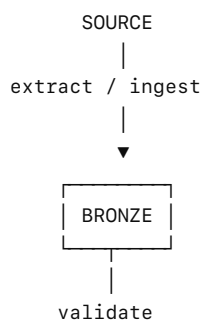
Αυτός είναι ο λόγος που production debugging δεν είναι απλώς:

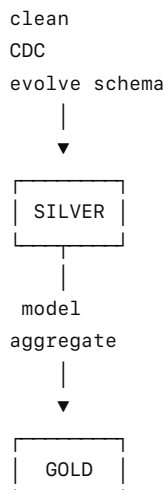
διάβασε το Python traceback.

Πρέπει να καταλάβεις **ολόκληρο το execution environment**.

Το νοητικό μοντέλο που θέλω να έχεις τώρα

</> Plain text





Τελικό τεστ prerequisites

Μην κοιτάξεις αμέσως τις απαντήσεις.

1

Ένα job τρέχει στις 02:00 μία φορά την ημέρα και διαβάζει μόνο τα rows που δημιουργήθηκαν από την προηγούμενη εκτέλεση.

Είναι:

</> Plain text

```
batch;
streaming;
incremental;
```

2

Γιατί:

</> Plain text

```
streaming
```

δεν σημαίνει απαραίτητα:

</> Plain text

```
ένα event → ένα Spark task αμέσως
```

;

3

Έχεις 5 TB table και καθημερινά αλλάζουν 20 MB.

Γιατί είναι πιθανώς κακή ιδέα να φορτώνεις καθημερινά όλο το table;

4

Τι πρόβλημα λύνει το CDC;

5

Έρχονται:

</> Plain text

```
seq=100 UPDATE balance=50
```

```
seq=101 UPDATE balance=80
```

αλλά το event 101 φτάνει πριν από το 100 .

Γιατί δεν πρέπει να χρησιμοποιήσεις απλώς arrival order;

6

Θέλεις να ξέρεις μόνο το current address ενός πελάτη.

SCD1 ή SCD2;

7

Θέλεις να ξέρεις πού έμενε ο πελάτης στις:

</> Plain text

```
2019
```

```
2022
```

```
2025
```

SCD1 ή SCD2;

8

Ποια η διαφορά μεταξύ:

</> Plain text

```
schema enforcement
```

και:

</> Plain text

```
schema evolution
```

;

9

Ένα upstream API προσθέτει καινούργιο field.

Είναι απαραίτητα error;

10

To Bronze περιέχει duplicates.

Είναι αυτό κατ' ανάγκη bug;

11

Γιατί κρατάμε Bronze αντί να γράφουμε κατευθείαν Gold;

12

Πού ανήκει:

</> Plain text

```
customer_id
lifetime_value
```

ανά πελάτη;

Bronze, Silver ή Gold;

13

Πού ανήκει συνήθως:

</> Plain text

```
deduplicated validated customer table
```

;

14

Τι διαφορά έχει:

</> Plain text

```
authentication
```

από:

</> Plain text

```
authorization
```

;

15

Τι είναι principal;

16

Γιατί production job συνήθως είναι καλύτερο να χρησιμοποιεί service principal αντί για προσωπικό account developer;

17

Τι είναι securable object;

18

Εξήγησε:

</> SQL

```
GRANT SELECT
ON TABLE prod.finance.transactions
TO `analysts`;
```

λέξη προς λέξη.

19

Έχεις:

</> Plain text

```
SELECT on table
```

αλλά όχι:

</> Plain text

```
USE CATALOG
```

Μπορεί να υπάρξει permission problem;

20

Γιατί:

</> Plain text

```
give ALL PRIVILEGES to everybody
```

είναι κακή λύση σε permission errors;

21

Γιατί δεν πρέπει να αλλάζεις production notebook χειροκίνητα;

22

Τι είναι CI;

23

Τι είναι CD;

24

Τι κερδίζουμε αν η περιγραφή ενός Job βρίσκεται σε YAML/Git αντί να έχει δημιουργηθεί μόνο με clicks στο UI;

25 — δύσκολη

Έχεις:

</> Plain text

Kafka → Bronze → Silver → Gold

Το Kafka έχει event 500.

Bronze έχει event 500.

Silver δεν το έχει.

Σε ποια περιοχή αρχίζεις το debugging;

26 — δύσκολη

Ένα pipeline τρέχει σωστά όταν το εκτελείς εσύ, αλλά αποτυγχάνει όταν τρέχει scheduled.

Ποιο είναι από τα πρώτα πράγματα που πρέπει να ελέγξεις;

27 — δύσκολη

Ένας πελάτης διαγράφηκε από PostgreSQL.

Στο Silver εξακολουθεί να υπάρχει για πάντα.

Ποια κατηγορία προβλήματος υποψιάζεσαι;

28 — δύσκολη

Το source schema άλλαξε.

Τι είναι χειρότερη πρακτική;

</> Plain text

A. fail και investigate

B. επιβεβαιώνω ότι η αλλαγή είναι intentional και εφαρμόζω evolution

C. ενεργοποιώ άκριτα οποιαδήποτε δυνατή schema evolution παντού

29 — interview

Εξήγησε σε 30 δευτερόλεπτα τι είναι medallion architecture.

30 — interview

Εξήγησε τη διαφορά batch, streaming και incremental processing χωρίς να χρησιμοποιήσεις buzzwords.

Απαντήσεις

1. Batch και incremental.
2. Επειδή engines όπως Spark Structured Streaming μπορούν να συγκεντρώνουν incoming records σε micro-batches και να τα εκτελούν μέσω του συνηθισμένου Spark execution model.
3. Επειδή επανεπεξεργάζεσαι τεράστιο historical dataset παρότι άλλαξε ελάχιστο subset. Incremental processing μπορεί να επεξεργαστεί μόνο τις αλλαγές.
4. Να μεταφέρει inserts/updates/deletes από source system χωρίς να χρειάζεται full snapshot/reload κάθε φορά.
5. Arrival order $\neq$ source modification order. Χρειάζεσαι sequence/version/event ordering.
6. SCD Type 1.
7. SCD Type 2.
8. Enforcement προστατεύει το υπάρχον schema contract. Evolution επιτρέπει ελεγχόμενη μεταβολή αυτού του schema.
9. Όχι. Μπορεί να είναι απολύτως νόμιμη schema evolution.
10. Όχι. Το Bronze μπορεί σκόπιμα να διατηρεί raw source behavior.
11. Για auditability, reprocessing, debugging και δυνατότητα να ξαναδημιουργήσουμε downstream layers όταν αλλάξει η transformation logic.
12. Συνήθως Gold.
13. Silver.
- 14.
15. Identity στην οποία μπορούν να αποδοθούν privileges, π.χ. user, group ή service principal.
16. Επειδή το production workload πρέπει να έχει σταθερή machine identity ανεξάρτητη από lifecycle/permissions ενός συγκεκριμένου υπαλλήλου.
17. Governed object πάνω στο οποίο μπορούν να εκχωρηθούν permissions.
- 18.

</> Plain text

Authentication:

ποιος είσαι;

Authorization:

τι επιτρέπεται να κάνεις;

</> Plain text

GRANT δώσε

SELECT permission ανάγνωσης

ON TABLE πάνω σε αυτό το table

TO analysts προς αυτό το principal/group

19. Ναι. Η πρόσβαση περνά μέσα από catalog/schema hierarchy.

20. Παραβιάζει least privilege, αυξάνει blast radius, κίνδυνο accidental/destructive actions και security exposure.

21. Χάνεις reproducibility, controlled review/testing, reliable version history και repeatable deployment.

22. Automatic integration/validation/testing των αλλαγών κώδικα.

23. Controlled delivery/deployment των validated changes στα target environments.

24.

</> Plain text

```
version control
review
reproducibility
automation
repeatable deployments
environment-specific configuration
```

25. Στο boundary:

</> Plain text

Bronze → Silver

γιατί έχουμε ήδη αποδείξει ότι το event έφτασε στο Bronze.

26. **Run identity.** Μπορεί εσύ να έχεις permissions αλλά η scheduled execution identity όχι.

27. CDC/delete handling. Πιθανόν το delete event να μην συλλέγεται ή να μην εφαρμόζεται σωστά.

28. C.

29. Καλή συνέντευξη:

Η medallion architecture οργανώνει τα lakehouse data σε προοδευτικά ποιοτικότερα layers: Bronze για raw/source-faithful data, Silver για validated και cleaned reusable datasets και Gold για business-oriented aggregates ή models που εξυπηρετούν συγκεκριμένους consumers.

30.

Batch σημαίνει ότι επεξεργάζομαι ένα πεπερασμένο σύνολο δεδομένων σε μια εκτέλεση. Streaming σημαίνει ότι το input συνεχίζει να φτάνει και το computation συνεχίζει να ενημερώνεται. Incremental σημαίνει ότι αντί να ξαναυπολογίζω ολόκληρο το ιστορικό, επεξεργάζομαι μόνο ό,τι είναι νέο ή άλλαξε· επομένως ένα workload μπορεί να είναι batch και incremental ταυτόχρονα.

Εδώ τελειώνουν τα prerequisites

Τώρα έχουμε αρκετή βάση για να αρχίσουμε **το ίδιο το Databricks** χωρίς να αποστηθίζεις ονόματα προϊόντων χωρίς να καταλαβαίνεις τι κάνουν.

Η επόμενη διδασκαλία που προτείνω είναι:

</> Plain text

DATABRICKS — PART 1

- |
- |— τι ακριβώς είναι το Databricks
- |— account vs workspace
- |— control plane vs compute plane
- |— workspace UI
- |— serverless vs classic compute
- |— clusters / compute
- |— SQL warehouses
- |— Databricks Runtime
- |— Photon
- |— notebooks
- |— catalogs / schemas / tables μέσα στο UI
- |— πώς εκτελείται πραγματικά ένα notebook
- |— πρώτη πραγματική άσκηση
- |— debugging όταν compute/notebook/query αποτυγχάνει

Και από εκεί θα προχωρήσουμε συστηματικά σε **Delta Lake → Unity Catalog → ingestion/Auto Loader → Lakeflow → Jobs → Databricks SQL → performance/Spark UI → Bundles/CI-CD → security → production projects → certification drills**. Θα κάνουμε τη syntax αναλυτικά, όχι απλώς conceptual overview.

Οι βασικές επίσημες πηγές που χρησιμοποίησα εδώ είναι η τρέχουσα τεκμηρίωση για Medallion Architecture [Databricks Docu...](#) , **Structured Streaming** [Databricks Docu...](#) , **schema enforcement/evolution** [Databricks Docu...](#) +1 , **Unity Catalog permissions** [Databricks Docu...](#) +1 , **AUTO CDC** [Databricks Docu...](#) και το σημερινό **CI/CD/Bundles model** [Databricks Docu...](#) +1 .

Θέλεις να αρχίσουμε τώρα το αρχικά ζητούμενο θέμα, δηλαδή το **Databricks Part 1**;

Wed, Sep 16 at 8:22 PM

Ναι

Worked for 15s

Databricks — Part 1

Αρχιτεκτονική, Workspace, Compute, Runtime, Photon, Notebooks και πρώτο debugging

Από εδώ και πέρα μιλάμε για **το ίδιο το Databricks**.

Θέλω πρώτα να αποκτήσεις ένα μοντέλο τόσο καθαρό ώστε όταν βλέπεις:

</> Plain text

Workspace

Serverless

Compute

SQL Warehouse

- Databricks Runtime
- Photon
- Notebook
- Unity Catalog

να μη φαίνονται σαν 7 άσχετα προϊόντα.

1. Τι είναι τελικά το Databricks;

Η πιο επικίνδυνη υπεραπλούστευση είναι:

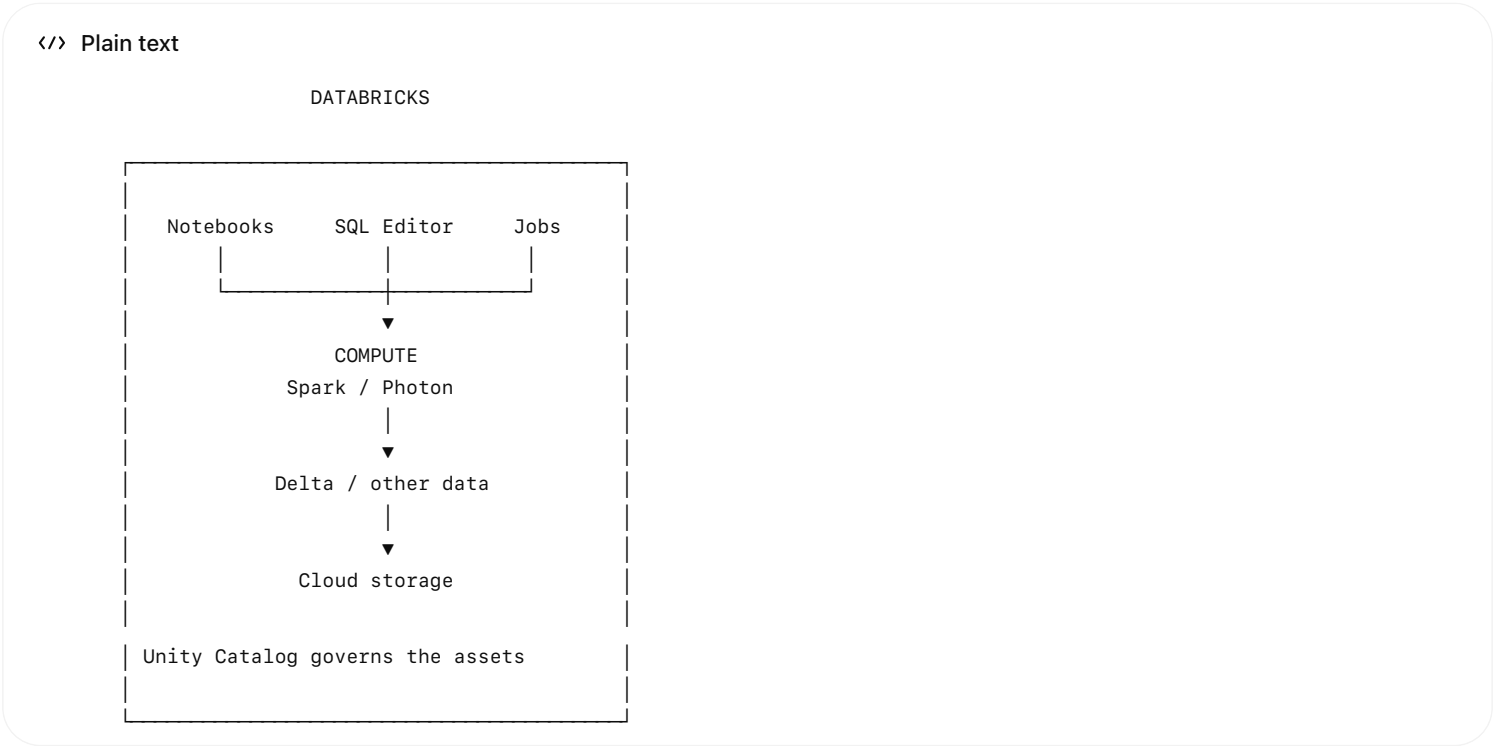
Databricks = managed Spark.

Ιστορικά αυτό ήταν πολύ πιο κοντά στην πραγματικότητα. Σήμερα όχι.

Η Databricks περιγράφει την πλατφόρμα ως ενιαίο περιβάλλον για data engineering, analytics, AI/ML, governance και deployment. Το Apache Spark παραμένει κεντρικό execution technology, αλλά το Databricks περιλαμβάνει πλέον Delta Lake, Unity Catalog, SQL warehouses, Lakeflow, serverless compute, Photon, ML/AI υπηρεσίες, orchestration και DevOps tooling.

 Databricks Docu... +1

Σκέψου:



Άρα:

</> Plain text

Spark

είναι κυρίως execution engine.

Ενώ:

</> Plain text

Databricks

είναι ολόκληρη data/AI platform.

Άσκηση 1

Ποιο είναι σωστό;

</> Plain text

- A. Databricks και Spark είναι συνώνυμα.
- B. Το Spark είναι μία από τις βασικές execution technologies μέσα στο Databricks.
- C. Το Databricks είναι file format.
- D. Το Spark είναι cloud provider.

Λύση

B.

2. Account και Workspace

Αυτή είναι από τις πρώτες διακρίσεις που πρέπει να ξέρεις για exam αλλά και πραγματική δουλειά.

Έστω εταιρεία:

</> Plain text

ACME Corporation

Έχει Databricks.

Στην κορυφή υπάρχει:

</> Plain text

Databricks Account

Το **account** είναι organization-level boundary.

Η σημερινή Databricks documentation αναφέρει ότι στο account level διαχειρίζεσαι μεταξύ άλλων users/groups/service principals, workspaces, Unity Catalog metastores, billing και account-level policies. Ένα account μπορεί να περιέχει πολλά workspaces. [🌐 Databricks Docu...](#)

Conceptually:

</> Plain text

```
DATABRICKS ACCOUNT
|
├── identities
├── billing
├── account policies
├── Unity Catalog configuration
|
├── Workspace A
├── Workspace B
└── Workspace C
```

3. Τι είναι Workspace;

Ένα **workspace** είναι το operational/collaboration environment στο οποίο δουλεύουν οι χρήστες.

Εκεί βρίσκεις πράγματα όπως:

</> Plain text

```
notebooks
files
Git folders
jobs
pipelines
SQL queries
dashboards
compute
Catalog Explorer
```

Η Databricks το περιγράφει ως το collaboration environment μέσα στο οποίο τρέχουν workloads όπως ingestion, interactive analysis, scheduled jobs και ML training. [Databricks Docu...](#)

Έστω:

</> Plain text

```
Company: ACME
```

Μπορεί να έχει:

</> Plain text

```
Account
|
├── analytics-prod
├── engineering-dev
└── research
```

Αυτά είναι τρία workspaces.

Account ≠ Workspace

Σκέψου το περίπου έτσι:

</> Plain text

Microsoft Azure tenant
≈ organization-wide level

Databricks account
≈ organization-wide Databricks level

και:

</> Plain text

workspace
≈ συγκεκριμένο environment
όπου δουλεύουν teams/workloads

Όχι απολύτως ταυτόσημη αντιστοίχιση, αλλά χρήσιμο mental model.

Ένα πιο πρακτικό παράδειγμα

Είσαι data engineer.

Μπαίνεις:

</> Plain text

<https://....databricks....>

και βλέπεις:

</> Plain text

Workspace
Catalog
Jobs & Pipelines
Compute
SQL
...

Αυτό που χρησιμοποιείς καθημερινά είναι κατά κύριο λόγο το **workspace**.

Ο admin μπορεί αντίθετα να χρειάζεται account-level διαχείριση:

</> Plain text

Create workspace

Configure SSO

Add account group

Manage billing

Άσκηση 2

Πού θα περίμενες περισσότερο να γίνει κάθε operation;

</> Plain text

1. Δημιουργία notebook
2. Δημιουργία organization-wide identity
3. Εκτέλεση Spark query
4. Δημιουργία δεύτερου workspace

Λύση

</> Plain text

- 1 → workspace
- 2 → account-level administration
- 3 → workspace workload / compute
- 4 → account level

4. Control plane και Compute plane

Τώρα ερχόμαστε στην πραγματική αρχιτεκτονική.

Η Databricks διαχωρίζει:

</> Plain text

CONTROL PLANE

και:

</> Plain text

COMPUTE PLANE

Η τρέχουσα αρχιτεκτονική τεκμηρίωση το κάνει ρητά. [Databricks Docu...](#) +1

Control plane

Εδώ βρίσκονται Databricks-managed backend services.

Για αρχή σκέψου:

</> Plain text

Control plane

"διαχειρίζεται και συντονίζει"

Περιλαμβάνει πράγματα που υποστηρίζουν:

</> Plain text

Web UI

APIs

workspace metadata

job orchestration metadata

configuration

platform coordination

Η web εφαρμογή του Databricks βρίσκεται στο control plane. [Databricks Docu...](#)

Compute plane

Εδώ:

γίνεται η πραγματική επεξεργασία των δεδομένων.

Δηλαδή:

</> Plain text

Spark transformations

SQL execution

joins

aggregations

ML computation

τρέχουν σε compute resources.

Η διαδρομή μιας εντολής

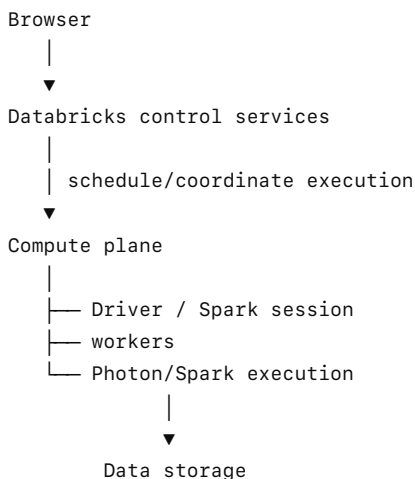
Γράφεις σε notebook:

</> Python

```
df.groupBy("country").sum("amount").show()
```

Conceptually:

Plain text



Πρόσεξε:

To browser σου **δεν κάνει το groupBy**.

To control plane **δεν σημαίνει ότι είναι εκεί που εκτελείται όλο το Spark workload**.

Η computation γίνεται στο compute plane.

5. Δύο διαφορετικά compute planes

Εδώ τα πράγματα έχουν αλλάξει πολύ τα τελευταία χρόνια.

Σήμερα υπάρχουν δύο κύριες περιπτώσεις:

Plain text

Classic compute plane

και:

Plain text

Serverless compute plane

Η Databricks documentation τα διαχωρίζει ρητά. [Databricks Docu...](#)

6. Classic compute

Με classic compute, δημιουργείς/configure compute resources.

Για παράδειγμα:

</> Plain text

```
Driver VM
Worker VM
Worker VM
Worker VM
```

και μπορείς να διαλέξεις πολλά infrastructure characteristics.

Σε AWS, για παράδειγμα, classic compute resources βρίσκονται στο AWS account του πελάτη. Αντίστοιχες cloud-specific αρχιτεκτονικές υπάρχουν σε Azure και GCP. [Databricks Docu...](#) +1

Conceptually:

</> Plain text

```
Your cloud environment
|
├─ Driver VM
├─ Worker VM
├─ Worker VM
└─ Worker VM
```

Έχεις περισσότερο έλεγχο.

Αλλά έχεις και περισσότερο operational responsibility.

7. Serverless compute

Με serverless λες περισσότερο:

Θέλω compute για να τρέξω αυτό το workload.

και λιγότερο:

Φτιάξε μου 1 driver με X RAM και 8 workers αυτού του VM type.

Το infrastructure provisioning/scaling το αναλαμβάνει σε σημαντικό βαθμό η Databricks. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
YOU

run notebook
  |
  ▼
Databricks
  |
  ├─ allocates compute
  ├─ scales it
  ├─ manages runtime
  └─ tears/scales resources as appropriate
```

Η Databricks σήμερα προτείνει serverless compute για τις περισσότερες interactive και automated workloads, όταν είναι διαθέσιμο και υποστηρίζει τις απαιτήσεις σου. [Databricks Docu...](#)

Μην μπερδεύεις serverless με "χωρίς servers"

Φυσικά υπάρχουν servers.

Το:

</> Plain text

```
serverless
```

σημαίνει πρακτικά:

Εσύ δεν διαχειρίζεσαι το underlying server infrastructure με τον ίδιο τρόπο.

Υπάρχουν CPU, RAM, machines.

Απλώς το abstraction boundary μετακινήθηκε.

Classic vs Serverless

To fundamental tradeoff είναι:

</> Plain text

```
SERVERLESS
```

```
λιγότερη υποδομή να διαχειριστείς  
fast provisioning  
automatic scaling  
platform optimizations
```

versus

```
CLASSIC
```

```
περισσότερος έλεγχος  
custom configuration  
ειδικά APIs/runtime/network needs
```

Δεν σημαίνει:

</> Plain text

```
serverless = καλό  
classic = παλιό και άχρηστο
```

Υπάρχουν workloads που ακόμα χρειάζονται classic.

Για παράδειγμα, η σημερινή serverless compute έχει περιορισμούς: δεν υποστηρίζει R, δεν υποστηρίζει Spark RDD APIs, και βασίζεται σε Spark Connect APIs. [Databricks Docu...](#)

Αυτό είναι πολύ σημαντικό για σένα επειδή ξέρεις Spark.

Παλιό code:

</> Python

```
rdd = spark.sparkContext.parallelize([1,2,3])
```

δεν είναι κάτι που πρέπει να θεωρείς ότι δουλεύει αυτόματα σε serverless notebook.

Ασκηση 3

Έχεις interactive Python notebook με:

</> Python

```
spark.read.table(...)
```

DataFrame transformations και SQL.

Δεν έχεις ειδικό infrastructure requirement.

Τι προτείνει γενικά σήμερα η Databricks;

Λύση

Serverless compute, εφόσον είναι διαθέσιμο και το workload υποστηρίζεται. [Databricks Docu...](#)

8. "Cluster" και "Compute"

Θα δεις παλαιότερα tutorials να λένε συνεχώς:

</> Plain text

```
cluster
```

Σήμερα το UI/documentation χρησιμοποιεί συχνά περισσότερο τον ευρύτερο όρο:

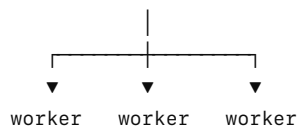
</> Plain text

```
compute
```

Classic Spark compute μπορεί ακόμα ουσιαστικά να έχει την κλασική αρχιτεκτονική:

</> Plain text

```
DRIVER
```



Και εκεί εφαρμόζονται όσα ήδη ξέρεις:

</> Plain text

```

job
↓
stage
↓
task
↓
partition

```

9. All-purpose compute

Classic **all-purpose compute** είναι compute που χρησιμοποιείται συνήθως για interactive development.

Για παράδειγμα:

</> Plain text

```

Notebook A ┌
Notebook B ┤ → All-purpose compute
Notebook C └

```

Είναι καλό για:

</> Plain text

```

development
interactive exploration
debugging

```

Αλλά δεν είναι η πρώτη επιλογή που προτείνει σήμερα η Databricks για automated Jobs. Για jobs προτείνει serverless για τις περισσότερες περιπτώσεις ή classic jobs compute όταν χρειάζεσαι custom cluster characteristics. [Databricks Docu...](#)

10. Jobs compute

Φαντάσου pipeline:

</> Plain text

```

02:00
  ingest
    ↓
  transform

```

↓
aggregate

Δεν χρειάζεσαι interactive cluster ανοιχτό όλη μέρα.

Μπορείς να έχεις compute που χρησιμοποιείται για το Job:

</> Plain text

```
Job starts
  ↓
compute available/provisioned
  ↓
tasks run
  ↓
workload finishes
```

Αργότερα θα εξετάσουμε πολύ βαθύτερα:

</> Plain text

```
serverless jobs
classic jobs compute
shared job compute
task-level compute
```

11. SQL Warehouse

Αυτό είναι διαφορετικό concept και πρέπει να το καταλάβεις καλά.

Ένας **SQL warehouse** είναι compute resource βελτιστοποιημένο για SQL analytics. Χρησιμοποιείται από SQL editor, dashboards, BI tools και άλλα SQL workloads. [Databricks Docu...](#)

Παράδειγμα:

</> SQL

```
SELECT
  country,
  SUM(revenue)
FROM prod.gold.sales
GROUP BY country;
```

Μπορεί να εκτελείται από SQL warehouse.

Conceptually:

</> Plain text

```
Power BI ───┐
            │
SQL Editor ─┴───> SQL Warehouse ──> tables
```


[Dashboard](#)

12. SQL Warehouse ≠ database

Αυτό είναι καλό exam/interview trap.

SQL warehouse εδώ σημαίνει:

Plain text

compute resource

όχι:

Plain text

το μέρος όπου "αποθηκεύονται" τα tables

Τα tables εξακολουθούν να έχουν data/storage layer.

Το warehouse εκτελεί queries.

Τι SQL warehouse types υπάρχουν;

Σήμερα η Databricks έχει:

Plain text

Serverless

Pro

Classic

SQL warehouses. [Databricks Docu...](#)

Και υπάρχει επίσης Lakehouse Real-Time σε Beta για συγκεκριμένα low-latency/high-concurrency use cases. [Databricks Docu...](#)

Για τις περισσότερες SQL analytics workloads, η Databricks προτείνει serverless SQL warehouse όταν είναι διαθέσιμο.

[Databricks Docu...](#) +1

13. Ένα πολύ σημαντικό distinction

Έστω ότι έχεις notebook:

Python

```
df = spark.read.table("prod.sales.orders")
```

Μπορεί να χρησιμοποιεί:

</> Plain text

Serverless notebook compute

ή:

</> Plain text

Classic all-purpose compute

Έχεις SQL dashboard:

</> SQL

```
SELECT COUNT(*) FROM prod.sales.orders;
```

Πιθανό compute:

</> Plain text

SQL Warehouse

Έχεις scheduled Python ETL:

</> Python

...

Πιθανό compute:

</> Plain text

Serverless jobs compute

ή classic jobs compute.

Άρα δεν υπάρχει:

"το Databricks cluster".

Υπάρχουν **διαφορετικά compute types** για διαφορετικά **workload shapes**.

14. Compute-selection mental model

Για interview/exam:

</> Plain text

Interactive Python/DataFrames

↓

Serverless notebook compute
(generally preferred)

Interactive SQL / BI

↓

Serverless SQL Warehouse

Automated Python/SQL job

↓

Serverless Jobs
(generally preferred)

Need custom cluster config / unsupported feature

↓

Classic compute

RDD / R requirement

↓

Classic compute

Αυτό συμφωνεί με τις τρέχουσες recommendations της Databricks. [Databricks Docu...](#) +1

15. Τι είναι Databricks Runtime;

Τώρα ένα concept που συχνά διδάσκεται πολύ επιφανειακά.

Έχεις ένα compute machine.

Μόνο hardware δεν αρκεί.

Χρειάζεσαι software environment:

</> Plain text

Spark
Python
Scala
JVM
Delta libraries
Databricks integrations
optimized libraries
system dependencies

Η Databricks πακετάρει ένα tested software stack ως:

Databricks Runtime — DBR

Think:

</> Plain text

Compute VM

```
|
├─ Databricks Runtime
│   │
│   ├── Apache Spark
│   ├── Delta support
│   ├── Python environment
│   ├── libraries
│   ├── Databricks optimizations
│   └─ integrations
```

Runtime version

Μπορείς να δεις εκδόσεις όπως:

</> Plain text

```
17.3 LTS
18.x
19 Beta
```

Η Databricks τον Ιούνιο 2026 είχε DBR 19 σε Beta πάνω σε Spark 4.2.0, ενώ LTS branches όπως 17.3 και 16.4 συνέχιζαν να λαμβάνουν maintenance updates. [Databricks Docu...](#)

Δεν θέλω να αποστηθίσεις version numbers.

Θέλω να καταλάβεις:

</> Plain text

```
Different DBR
  ↓
different Spark version
different Python/packages
different capabilities
different bug fixes
different behavior
```

Τι σημαίνει LTS;

Long-Term Support.

Για production περιβάλλοντα, LTS runtime μπορεί να είναι προτιμητέο όταν θες:

</> Plain text

```
σταθερότητα
predictable maintenance
λιγότερο frequent migration
```

αντί να κυνηγάς κάθε καινούργια runtime release.

Runtime debugging

Έστω:

</> Plain text

```
DEV
DBR 18.x

PROD
DBR 17.3 LTS
```

To code:

</> Python

```
some_new_api(...)
```

δουλεύει στο dev.

Στο production:

</> Plain text

```
AttributeError
```

Πιθανός root cause;

</> Plain text

```
runtime version mismatch
```

Γι' αυτό production debugging δεν είναι μόνο:

Έχει bug ο Python κώδικας;

Μπορεί να είναι:

</> Plain text

```
code
+
runtime
+
libraries
+
configuration
```

16. Runtime ML

Υπάρχει επίσης **Databricks Runtime for Machine Learning**.

Προσθέτει prebuilt ML/DL infrastructure και συνηθισμένες ML libraries ώστε να μη στήνεις ολόκληρο environmentμόνος σου.

 Databricks Docu...

Conceptually:

 Plain text

Standard DBR

+

ML libraries

+

ML/DL infrastructure

=

DBR ML

Θα το εξετάσουμε πολύ αργότερα στο ML μέρος.

17. Τι είναι Photon;

Εδώ υπάρχει μια πολύ σημαντική διάκριση.

Spark είναι execution framework/engine ecosystem.

Photon είναι Databricks-native vectorized query engine, σχεδιασμένο να επιταχύνει SQL και DataFrame workloads.

 Databricks Docu... +1

Δεν είναι:

 Plain text

άλλη programming language

ούτε:

 Plain text

database

ούτε:

 Plain text

storage format

Χωρίς Photon

Conceptually:

⟨/⟩ Plain text

SQL / DataFrame

↓

Spark plan

↓

Spark/JVM execution

Με Photon

Για operations που υποστηρίζει:

⟨/⟩ Plain text

SQL / DataFrame

↓

query plan

↓

Photon-native execution

Το σημαντικό:

Μπορεί να γράψεις τον ίδιο SQL/DataFrame κώδικα.

Για παράδειγμα:

⟨/⟩ Python

```
result = (  
    spark.table("prod.sales.orders")  
        .groupBy("country")  
        .sum("amount")  
)
```

Δεν γράφεις:

⟨/⟩ Python

```
photon.groupBy(...)
```

Το optimization γίνεται κάτω από το API abstraction.

Γιατί vectorized;

Αφελές mental model:

⟨/⟩ Plain text

```
row 1
process

row 2
process

row 3
process
```

Vectorized processing προσπαθεί να χειρίζεται batches δεδομένων πιο αποτελεσματικά αξιοποιώντας CPU characteristics.

Conceptually:

</> Plain text

```
[value1 value2 value3 value4 ...]
```

↓

```
operate on vector/batch
```

Αυτό μπορεί να μειώσει overhead και να βελτιώσει CPU efficiency.

Δεν χρειάζεται ακόμα να μπορούμε στα C++/SIMD internals.

Θα το κάνουμε στο performance section.

Photon availability

Σήμερα Photon είναι enabled σε serverless compute και SQL warehouses, ενώ στα classic compute resources είναι επίσης διαθέσιμο και συνήθως ενεργοποιημένο by default μέσω της αντίστοιχης configuration. [Databricks Docu...](#)

Άσκηση 4

Έχεις:

</> Python

```
df.groupBy("customer_id").sum("amount")
```

Με Photon πρέπει να ξαναγράψεις τον κώδικα ως Photon API;

Λύση

Όχι.

Η ίδια DataFrame/SQL abstraction μπορεί να εκτελεστεί μέσω Photon για υποστηριζόμενες operations.

18. Τώρα πάμε στο Notebook

Databricks Notebook:

⟨/⟩ Plain text

interactive document

στο οποίο έχεις cells.

Παράδειγμα:

⟨/⟩ Plain text

Cell 1: Python

Cell 2: SQL

Cell 3: Markdown

Cell 4: Python

Τα notebooks υποστηρίζουν Python, SQL, Scala και R ανάλογα με το compute/runtime που χρησιμοποιείς. [Databricks Docu...](#)

19. Το notebook από μόνο του δεν εκτελεί τίποτα

Αυτό είναι θεμελιώδες.

Ένα notebook είναι code/document artifact.

Χρειάζεται **compute**.

⟨/⟩ Plain text

Notebook

|

|

attach/connect

▼

Compute

|

▼

execution

Αν γράψεις:

⟨/⟩ Python

```
print("hello")
```

χρειάζεται ενεργό execution session.

Η σημερινή Databricks μάλιστα μπορεί σε Unity Catalog-enabled workspaces να συνδέσει νέο notebook αυτόματα σε serverless compute όταν εκτελέσεις cell χωρίς να έχεις διαλέξει άλλο compute. [Databricks Docu...](#)

20. Πρώτο πραγματικό Databricks notebook

Έστω Python notebook.

Cell 1

</> Python

```
data = [
    (1, "Athens", 120.0),
    (2, "Volos", 80.0),
    (3, "Athens", 50.0),
    (4, "Patras", 200.0),
    (5, "Athens", 30.0),
]

df = spark.createDataFrame(
    data,
    ["order_id", "city", "amount"]
)

display(df)
```

Το:

</> Python

spark

δεν το δημιούργησες εσύ.

Σε Databricks notebook υπάρχει έτοιμο Spark session/context abstraction για supported compute.

Τι έγινε;

</> Plain text

```
Python list
↓
spark.createDataFrame()
↓
Spark DataFrame

schema:

order_id : long
city      : string
amount    : double
```

21. `display()` έναντι `show()`

Spark-native:

```
</> Python
```

```
df.show()
```

παράγει text-oriented output.

Databricks:

```
</> Python
```

```
display(df)
```

δίνει richer notebook rendering.

Για παράδειγμα μπορείς να δεις table visualization πιο εύκολα.

Αυτό είναι Databricks notebook functionality, όχι core PySpark DataFrame method.

Άρα:

```
</> Python
```

```
df.show()
```

είναι Spark API.

Ενώ:

```
</> Python
```

```
display(df)
```

είναι notebook/platform helper.

Αυτή είναι σημαντική διάκριση όταν μεταφέρεις code εκτός Databricks.

22. DataFrame transformation

Cell 2:

```
</> Python
```

```
from pyspark.sql import functions as F
```

```
city_sales = (  
    df  
    .groupBy("city")
```

```
.agg(  
    F.sum("amount").alias("total_sales"),  
    F.count("*").alias("orders")  
)  
.orderBy(F.desc("total_sales"))  
)  
  
display(city_sales)
```

Output:

</> Plain text

city	total_sales	orders
Athens	200.0	3
Patras	200.0	1
Volos	80.0	1

Και εδώ ξέρεις ήδη τι συμβαίνει.

`groupBy("city")` πιθανότατα απαιτεί:

</> Plain text

```
shuffle
```

ώστε όλα τα ίδια `city` keys να συγκεντρωθούν στο κατάλληλο downstream partition.

Η γνώση Spark αρχίζει εδώ να εφαρμόζεται μέσα στο Databricks.

23. Δες το physical plan

Κάνε:

</> Python

```
city_sales.explain("formatted")
```

ή:

</> Python

```
city_sales.explain("extended")
```

Θα δεις physical plan με operators όπως:

</> Plain text

```
HashAggregate  
Exchange
```

```
HashAggregate
Sort
```

Το:

```
</> Plain text

Exchange
```

είναι συχνά το σημείο όπου βλέπεις redistribution/shuffle boundary.

Άρα το Databricks **δεν κρύβει τη Spark γνώση σου**.

Την αξιοποιείς.

Άσκηση 5

Γιατί το:

```
</> Python

df.filter(F.col("amount") > 100)
```

συνήθως δεν χρειάζεται shuffle, ενώ:

```
</> Python

df.groupBy("city").sum("amount")
```

πιθανότατα χρειάζεται;

Λύση

Το filter μπορεί να αποφασιστεί ανεξάρτητα μέσα σε κάθε υπάρχον partition:

```
</> Plain text

partition 1 → filter locally
partition 2 → filter locally
partition 3 → filter locally
```

Για το `groupBy("city")` , όμως, όλα τα rows με:

```
</> Plain text

city = Athens
```

πρέπει να συνδυαστούν λογικά, ακόμη κι αν βρίσκονταν αρχικά σε διαφορετικά partitions.

Άρα απαιτείται redistribution/shuffle.

24. Mixed-language notebooks και magic commands

Ας πούμε ότι το notebook έχει default language Python.

Μπορείς σε cell να γράψεις:

```
</> SQL

%sql

SELECT current_date();
```

Το %sql λέει:

αυτό το cell να το ερμηνεύσεις ως SQL.

Η Databricks υποστηρίζει magic commands όπως %python , %sql , %scala , %r , %md και %pip . [Databricks Docu...](#)

Python cell

```
</> Python

%python

x = 5
print(x * 2)
```

SQL cell

```
</> SQL

%sql

SELECT
    current_timestamp() AS now;
```

Markdown

```
</> Plain text

%md

# Sales Analysis

Αυτό είναι το daily sales notebook.
```

Install library

</> Python

```
%pip install requests
```

H %pip εγκαθιστά notebook-scoped Python libraries. Αυτές είναι scoped στο notebook/session context και δεν πρέπει να θεωρείς ότι γίνονται permanent installation σε όλο το workspace. [Databricks Docu...](#)

25. SQL και Python στην ίδια ροή

Έστω ότι έχεις table:

</> Plain text

```
main.sales.orders
```

Python:

</> Python

```
orders = spark.table("main.sales.orders")

display(orders)
```

ή SQL:

</> SQL

```
%sql

SELECT *
FROM main.sales.orders;
```

Και τα δύο μπορεί τελικά να οδηγήσουν στο ίδιο logical table.

26. spark.table()

Πολύ χρήσιμο:

</> Python

```
df = spark.table("catalog.schema.table")
```

Παράδειγμα:

</> Python

```
orders = spark.table("prod.silver.orders")
```

Conceptually:

Plain text

```
Unity Catalog name
prod.silver.orders
    ↓
resolve metadata
    ↓
table
    ↓
Spark DataFrame
```

27. spark.sql()

Μπορείς επίσης:

Python

```
df = spark.sql("""
    SELECT
        country,
        SUM(amount) AS revenue
    FROM prod.silver.orders
    GROUP BY country
""")

display(df)
```

Αυτό επιστρέφει DataFrame.

Άρα:

Plain text

```
SQL statement
    ↓
Spark SQL
    ↓
DataFrame
```

28. SQL notebook cell και _sqldf

Έστω:

SQL

```
%sql
```



```
SELECT *  
FROM prod.silver.orders  
WHERE amount > 100;
```

Σε supported notebook environments το αποτέλεσμα SQL cell μπορεί να γίνει διαθέσιμο σε subsequent Python code ως `_sqldf`. Η τρέχουσα notebook documentation το υποστηρίζει. [Databricks Docu...](#)

Παράδειγμα:

```
</> Python  
  
display(_sqldf)
```

Δεν θα βασίσουμε production architecture σε αυτό, αλλά είναι χρήσιμο interactive feature.

29. Widgets — η πρώτη πραγματική παραμετροποίηση notebook

Έστω notebook που θέλεις να τρέχει για διαφορετικές χώρες.

Κακό:

```
</> Python  
  
country = "GR"
```

και κάθε φορά αλλάζεις manually code.

Καλύτερα:

```
</> Python  
  
dbutils.widgets.dropdown(  
    "country",  
    "GR",  
    ["GR", "DE", "FR"],  
    "Country"  
)
```

Παίρνεις τιμή:

```
</> Python  
  
country = dbutils.widgets.get("country")
```

και μετά:

```
</> Python  
  
filtered = (  
    spark.table("prod.silver.orders")  
    .filter(F.col("country") == country)
```

```
)  
  
display(filtered)
```

Τα Databricks widgets υπάρχουν ακριβώς για parameterized notebooks/dashboards και υποστηρίζουν text, dropdown, combobox κ.ά. [Databricks Docu...](#)

SQL parameterization — σημαντικό νέο syntax

Η σύγχρονη syntax για SQL parameter markers είναι:

```
</> SQL  
  
:parameter
```

όχι το παλαιότερο:

```
</> Plain text  
  
${parameter}
```

Το `${param}` είναι deprecated για DBR 15.2+. [Databricks Docu...](#)

Παράδειγμα:

```
</> Python  
  
dbutils.widgets.text("country", "GR")
```

και SQL:

```
</> SQL  
  
%sql  
  
SELECT *  
FROM prod.silver.orders  
WHERE country = :country;
```

Αυτό είναι κάτι που θέλω να μάθεις στη **σημερινή syntax**, όχι από παλιά tutorials.

30. Για identifiers είναι διαφορετικά

Έστω widget:

```
</> Plain text  
  
table = orders
```

Δεν γράφεις απλά:

</> SQL

```
SELECT *  
FROM :table
```

επειδή parameter value και SQL identifier δεν είναι η ίδια semantic κατηγορία.

Χρησιμοποιείς IDENTIFIER() όπου απαιτείται:

</> SQL

```
SELECT *  
FROM IDENTIFIER(:table_name);
```

Η Databricks documentation το επισημαίνει και το IDENTIFIER() βοηθά να χειριστείς parameterized object names σωστά.

⊕ Databricks Docu...

Αργότερα θα μάθουμε σε βάθος:

</> Plain text

```
literal  
identifier  
parameter  
expression
```

γιατί είναι συχνή πηγή SQL bugs.

31. Catalog Explorer

Στην αριστερή πλευρά του workspace θα συναντήσεις το **Catalog** / Catalog Explorer.

Εκεί βλέπεις:

</> Plain text

```
catalog  
↓  
schema  
↓  
tables  
views  
volumes  
functions  
models  
...
```

Παράδειγμα:

</> Plain text

```
prod
├── bronze
│   └── raw_orders
├── silver
│   └── orders
└── gold
    └── daily_sales
```

Η σημερινή Catalog Explorer επιτρέπει discovery των Unity Catalog objects και απαιτεί κατάλληλα privileges όπως `BROWSE` ή/και τα αντίστοιχα data access privileges. [🌐 Databricks Docu...](#)

32. Πρώτη SQL εξερεύνηση Catalog

Θέλω να συνηθίσεις νωρίς τη syntax.

</> SQL

```
SHOW CATALOGS;
```

Μετά:

</> SQL

```
SHOW SCHEMAS IN prod;
```

Μετά:

</> SQL

```
SHOW TABLES IN prod.silver;
```

και:

</> SQL

```
DESCRIBE TABLE prod.silver.orders;
```

ή:

</> SQL

```
DESCRIBE DETAIL prod.silver.orders;
```

Αυτές οι εντολές θα γίνουν βασικά debugging εργαλεία.

33. Δημιουργία schema

</> SQL

```
CREATE SCHEMA IF NOT EXISTS dev_training.sales;
```

Τώρα έχεις:

</> Plain text

```
catalog:
dev_training
```

```
schema:
sales
```

34. Δημιουργία Delta table

</> SQL

```
CREATE TABLE dev_training.sales.orders (
  order_id BIGINT,
  customer_id BIGINT,
  amount DECIMAL(10,2),
  country STRING,
  order_ts TIMESTAMP
)
USING DELTA;
```

Τώρα σύνδεσε όσα μάθαμε:

</> Plain text

```
Unity Catalog
  |
  | name / metadata / permissions
  ▼
dev_training.sales.orders
  |
  | table format
  ▼
Delta
  |
  | files + transaction state
  ▼
storage
```

35. Γράψε δεδομένα

</> SQL

```
INSERT INTO dev_training.sales.orders
VALUES
  (1, 101, 20.50, 'GR', CURRENT_TIMESTAMP()),
  (2, 102, 90.00, 'DE', CURRENT_TIMESTAMP()),
  (3, 103, 15.25, 'GR', CURRENT_TIMESTAMP());
```

και:

</> SQL

```
SELECT *
FROM dev_training.sales.orders;
```

Αυτό είναι πραγματικό table workflow, όχι του DataFrame μόνο στη RAM.

36. Το ίδιο από Python

</> Python

```
orders = spark.table("dev_training.sales.orders")

display(orders)
```

Filter:

</> Python

```
greek_orders = (
    orders
    .filter(F.col("country") == "GR")
)

display(greek_orders)
```

Aggregate:

</> Python

```
(
    orders
    .groupBy("country")
    .agg(F.sum("amount").alias("revenue"))
    .display()
)
```

Αν `.display()` δεν υποστηρίζεται στην εκάστοτε context/version, χρησιμοποίησε ασφαλώς:

</> Python

```
display(
    orders
    .groupBy("country")
    .agg(F.sum("amount").alias("revenue"))
)
```

37. Τι ακριβώς συμβαίνει όταν πατήσεις Run;

Αυτό θέλω να το ξέρεις σε επίπεδο interview.

Έχεις:

</> Python

```
orders = spark.table("prod.silver.orders")

result = (
    orders
    .filter(F.col("amount") > 100)
    .groupBy("country")
    .sum("amount")
)

display(result)
```

Δεν συμβαίνει απλά:

</> Plain text

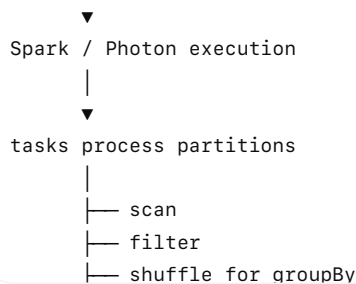
Python runs.

Η πιο σωστή abstraction είναι:

</> Plain text

```

Notebook cell
  |
  ▼
Python code executed in active compute session
  |
  ▼
Spark constructs logical DataFrame transformations
  |
  ▼
Action/display requires result
  |
  ▼
Catalyst analysis/optimization
  |
  ▼
physical execution plan
  |
```



Και έτσι ενώνεται όλο το Spark course με Databricks.

38. Lazy evaluation παραμένει

</> Python

```
orders = spark.table(...)

x = orders.filter(...)

y = x.groupBy(...).sum(...)
```

Αυτά κατά βάση περιγράφουν computation.

Όταν κάνεις:

</> Python

```
display(y)
```

ή:

</> Python

```
y.count()
```

ή:

</> Python

```
y.write...
```

τότε απαιτείται action/evaluation.

Databricks δεν αλλάζει αυτή τη βασική Spark λογική.

39. Πρώτο debugging: notebook δεν τρέχει

Έχεις:

</> Plain text

Notebook cell stuck

Μην αρχίζεις αμέσως να κοιτάς Spark shuffle.

Πρώτα πρέπει να καθορίσεις **σε ποιο layer βρίσκεται το failure**.

Conceptually:

</> Plain text

```
Notebook
↓
Compute attached?
↓
Compute running?
↓
Session healthy?
↓
Code parsed?
↓
Table exists?
↓
Permission?
↓
Query plan?
↓
Runtime execution?
```

Case A — No compute

Symptom:

</> Plain text

notebook not attached

Root cause category:

</> Plain text

execution environment

όχι Spark code.

Case B — Table not found

</> Python

```
spark.table("prod.silver.ordres")
```

Error:

</> Plain text

```
TABLE_OR_VIEW_NOT_FOUND
```

Πρώτο πράγμα:

</> SQL

```
SHOW TABLES IN prod.silver;
```

Μπορεί απλά να έχεις typo:

</> Plain text

```
ordres
```

αντί:

</> Plain text

```
orders
```

ή να είσαι σε λάθος:

</> Plain text

```
catalog/schema
```

40. Fully qualified names

Καλό production habit:

</> Plain text

```
catalog.schema.table
```

αντί για απλώς:

</> Plain text

```
orders
```

Γιατί;

Έστω current context:

</> Plain text

```
dev.sales
```

και γράψεις:

</> SQL

```
SELECT * FROM orders;
```

Μπορεί να διαβάζεις:

</> Plain text

```
dev.sales.orders
```

ενώ νόμιζες:

</> Plain text

```
prod.sales.orders
```

Πολύ επικίνδυνο.

41. Έλεγξε current namespace

</> SQL

```
SELECT current_catalog();
```

και:

</> SQL

```
SELECT current_schema();
```

ή χρησιμοποίησε:

</> SQL

```
USE CATALOG prod;  
USE SCHEMA silver;
```

μετά:

SQL

```
SELECT *  
FROM orders;
```

Αλλά για production transformations εγώ θέλω να συνηθίσεις να αναγνωρίζεις πότε είναι ασφαλέστερο το:

Plain text

```
prod.silver.orders
```

Case C — Permission denied

Έχεις:

SQL

```
SELECT *  
FROM prod.finance.transactions;
```

Error:

Plain text

```
INSUFFICIENT_PRIVILEGES
```

Δεν ψάχνεις:

Plain text

```
Spark partitions
```

Ελέγχεις:

Plain text

```
Which principal am I?  
  
Do I have USE CATALOG?  
  
Do I have USE SCHEMA?  
  
Do I have SELECT?
```

Δηλαδή:

Plain text

```
governance / authorization layer
```

Case D — Python import failure

</> Python

```
import some_library
```

Error:

</> Plain text

```
ModuleNotFoundError
```

Αυτό είναι πιθανότερα:

</> Plain text

```
runtime/environment/dependency
```

και όχι data problem.

Μπορείς σε notebook να χρειαστείς:

</> Python

```
%pip install some-library
```

αλλά σε production πρέπει να διαχειριστούμε dependencies πιο συστηματικά.

Case E — Query τρέχει αλλά είναι αργό

Τώρα αρχίζουμε να εξετάζουμε Spark.

</> Plain text

```
Physical plan  
Spark UI  
tasks  
shuffle  
skew  
spill  
input bytes  
partition counts  
join strategy
```

Άρα:

Slow query ≠ automatically "small cluster".

Αυτή τη συνήθεια θέλω να την εξαφανίσουμε από την αρχή.

42. Ένα πραγματικό debugging tree

Έστω:

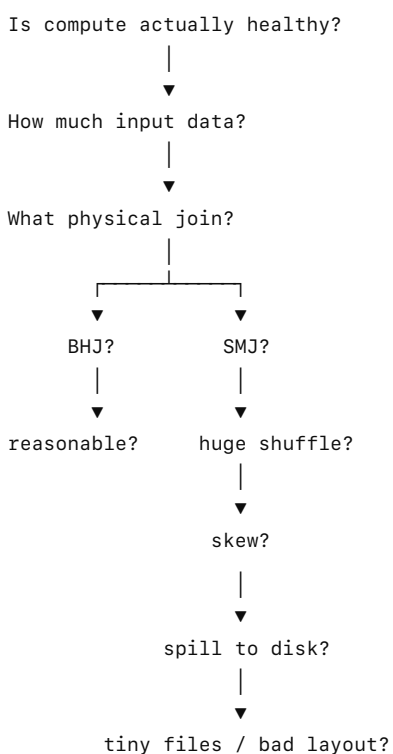
</> Python

```
result = (  
    spark.table("prod.silver.orders")  
    .join(  
        spark.table("prod.silver.customers"),  
        "customer_id"  
    )  
    .groupBy("country")  
    .agg(F.sum("amount"))  
)  
  
display(result)
```

τρέχει 40 λεπτά.

Σκέψου:

</> Plain text



Αυτό θα κάνουμε αργότερα με πραγματικά Spark UI examples.

43. Πρώτο mini-project

Θέλω να το δεις ως άσκηση που θα μπορούσες όντως να υλοποιήσεις στο Databricks Community/Free/normal workspace

ανάλογα με την πρόσβαση που διαθέτεις.

Δημιούργησε table:

</> SQL

```
CREATE TABLE IF NOT EXISTS dev_training.sales.orders (  
  order_id BIGINT,  
  customer_id BIGINT,  
  city STRING,  
  amount DECIMAL(10,2),  
  order_date DATE  
)  
USING DELTA;
```

Βάλε:

</> SQL

```
INSERT INTO dev_training.sales.orders VALUES  
(1, 101, 'Athens', 100.00, DATE'2026-09-10'),  
(2, 102, 'Volos', 80.00, DATE'2026-09-10'),  
(3, 101, 'Athens', 30.00, DATE'2026-09-11'),  
(4, 103, 'Patras', 150.00, DATE'2026-09-11'),  
(5, 104, 'Athens', 200.00, DATE'2026-09-12'),  
(6, 102, 'Volos', 40.00, DATE'2026-09-12');
```

Task A — SQL

Βρες revenue ανά city.

Λύση

</> SQL

```
SELECT  
  city,  
  SUM(amount) AS revenue  
FROM dev_training.sales.orders  
GROUP BY city  
ORDER BY revenue DESC;
```

Task B — PySpark

Το ίδιο:

</> Python

```
from pyspark.sql import functions as F  
  
orders = spark.table("dev_training.sales.orders")
```

```
city_revenue = (  
    orders  
    .groupBy("city")  
    .agg(F.sum("amount").alias("revenue"))  
    .orderBy(F.desc("revenue"))  
)  
  
display(city_revenue)
```

Task C — Physical plan

</> Python

```
city_revenue.explain("formatted")
```

Ψάξε για:

</> Plain text

Exchange

και:

</> Plain text

Aggregate

Γιατί;

Επειδή το:

</> Plain text

GROUP BY city

χρειάζεται να ενώσει logically rows με ίδιο city.

Task D — Parameter

</> Python

```
dbutils.widgets.dropdown(  
    "city",  
    "Athens",  
    ["Athens", "Volos", "Patras"]  
)  
  
city = dbutils.widgets.get("city")
```



```
result = (  
    spark.table("dev_training.sales.orders")  
    .filter(F.col("city") == city)  
)  
  
display(result)
```

Τώρα έχεις ένα μικρό **interactive parameterized Databricks notebook**.

Task E — Κάνε το SQL version

</> SQL

%sql

```
SELECT *  
FROM dev_training.sales.orders  
WHERE city = :city;
```

Αυτό χρησιμοποιεί το σύγχρονο widget parameter marker syntax. [Databricks Docu...](#) +1

44. Τι πρέπει ήδη να αναγνωρίζεις

Αν δω:

</> Python

```
spark.table(...)
```

Θέλω να σκέφτεσαι:

</> Plain text

Spark API

Αν δω:

</> Python

```
display(...)
```

Θέλω:

</> Plain text

Databricks notebook convenience

Αν δω:

</> Plain text

```
prod.silver.orders
```

Θέλω:

</> Plain text

```
Unity Catalog namespace
```

Αν δω:

</> Plain text

```
Serverless
```

Θέλω:

</> Plain text

```
Databricks-managed compute infrastructure
```

Αν δω:

</> Plain text

```
SQL Warehouse
```

Θέλω:

</> Plain text

```
SQL-optimized compute
```

Αν δω:

</> Plain text

```
Photon
```

Θέλω:

</> Plain text

```
vectorized native execution engine
```

Αν δω:

</> Plain text

DBR 17.3

Θέλω:

</> Plain text

software/runtime environment

Και όχι να τα βάζεις όλα στο mental bucket:

</> Plain text

"Databricks stuff"

Τεστ Part 1

1

Τι είναι ανώτερο organizational object;

</> Plain text

account
ή
workspace

Απάντηση

account .

Ένα account μπορεί να περιλαμβάνει πολλά workspaces. [Databricks Docu...](#)

2

Πού γίνεται η πραγματική data processing;

Απάντηση

Στο **compute plane**.

3

Το Databricks UI ανήκει περισσότερο conceptually στο:

</> Plain text

control plane
ή

```
compute plane;
```

Απάντηση

Control plane. [Databricks Docu...](#)

4

Serverless σημαίνει ότι δεν υπάρχουν servers;

Απάντηση

Όχι.

Σημαίνει ότι η Databricks διαχειρίζεται το underlying compute provisioning/infrastructure abstraction για σένα.

5

Γιατί να χρησιμοποιήσεις classic αντί serverless;

Απάντηση

Όταν χρειάζεσαι capabilities/configuration που δεν υποστηρίζει το serverless, όπως σήμερα R, RDD APIs ή ορισμένα custom infrastructure/network/runtime requirements. [Databricks Docu...](#)

6

Ποιο compute θα διάλεγες ως default recommendation για interactive Python/DataFrame notebook;

Απάντηση

Serverless notebook compute, εφόσον υποστηρίζεται.

7

Ποιο για Power BI SQL queries;

Απάντηση

Συνήθως serverless SQL warehouse.

8

Τι είναι SQL warehouse;

Απάντηση

Compute optimized για SQL workloads — όχι το physical storage των tables.

9

Τι είναι Databricks Runtime;

Απάντηση

To software/runtime stack στο οποίο εκτελείται workload, περιλαμβάνοντας Spark και Databricks-integrated/optimized libraries και dependencies.

10

DBR και VM είναι το ίδιο;

Απάντηση

Όχι.

</> Plain text

```
VM / compute resources
=
hardware/infrastructure abstraction
```

```
DBR
=
software runtime environment
```

11

Τι είναι Photon;

Απάντηση

Databricks-native vectorized execution engine που επιταχύνει υποστηριζόμενα SQL/DataFrame workloads. [Databricks Docu...](#)

12

Πρέπει να γράψεις ειδικό Photon code;

Απάντηση

Συνήθως όχι. Χρησιμοποιείς τα υπάρχοντα DataFrame/SQL APIs.

13

Τι κάνει:

</> Python

```
spark.table("prod.silver.orders")
```

;

Απάντηση

Δημιουργεί Spark DataFrame που αναφέρεται στο συγκεκριμένο table object.

14

Τι σημαίνει:

</> Plain text

```
prod.silver.orders
```

;

Απάντηση

</> Plain text

```
prod    = catalog
silver  = schema
orders  = object/table
```

15

Τι κάνει:

</> Plain text

```
%sql
```

;

Απάντηση

Δηλώνει ότι το συγκεκριμένο notebook cell πρέπει να εκτελεστεί ως SQL.

16

Τι κάνει:

</> Plain text

```
%pip
```

;

Απάντηση

Διαχειρίζεται notebook-scoped Python packages. [Databricks Docu...](#)

17

Ποιο είναι το σημερινό syntax widget parameter;

```
</> Plain text  
${country}
```

ή:

```
</> Plain text  
:country
```

;

Απάντηση

```
</> Plain text  
:country
```

Το `${...}` είναι legacy/deprecated στη σύγχρονη runtime syntax. [Databricks Docu...](#)

18

Γιατί:

```
</> SQL  
FROM :table
```

δεν αντιμετωπίζεται όπως:

```
</> SQL  
WHERE country = :country
```

;

Απάντηση

Επειδή το πρώτο απαιτεί **SQL identifier**, ενώ το δεύτερο είναι parameterized value/expression. Για dynamic identifiers χρησιμοποιούμε κατάλληλες constructs όπως `IDENTIFIER()`. [Databricks Docu...](#)

19

Ένα notebook υπάρχει αλλά δεν είναι συνδεδεμένο με compute. Μπορεί να εκτελέσει Spark workload;

Απάντηση

Όχι. Το notebook είναι artifact/interface· απαιτεί execution compute/session.

20

Γιατί:

```
</> Python  
  
df.filter(...)
```

δεν σημαίνει αναγκαστικά ότι τα δεδομένα επεξεργάστηκαν ήδη;

Απάντηση

Λόγω Spark lazy evaluation.

Το transformation συμμετέχει στην κατασκευή computation plan· η actual execution γίνεται όταν ζητηθεί action/result.

Δύσκολες ερωτήσεις

21

Έχεις:

```
</> Plain text  
  
Browser  
↓  
Databricks UI  
↓  
Spark query  
↓  
ADLS
```

Ποια δύο διαφορετικά planes συμμετέχουν;

Απάντηση

Το UI/platform coordination ανήκει στο **control plane**, ενώ η Spark computation γίνεται στο **compute plane**.

22

Το compute σταμάτησε.

Χάθηκε ο Delta table σου;

Απάντηση

Όχι, εφόσον το table βρίσκεται σε persistent storage.

Αυτό επιστρέφει στη βασική διάκριση:

⟨/⟩ Plain text

```
compute lifetime
#
data lifetime
```

23

Query αργεί.

Ο junior engineer λέει:

Χρειαζόμαστε μεγαλύτερο cluster.

Ποιο είναι το πρόβλημα με αυτό το reasoning;

Απάντηση

Δεν έχει εντοπιστεί root cause.

Μπορεί να έχουμε:

⟨/⟩ Plain text

```
skew
wrong join
huge shuffle
spill
poor table layout
small files
unnecessary scan
bad query
insufficient statistics
compute bottleneck
```

Το cluster sizing είναι μόνο μία πιθανότητα.

24

Το notebook δουλεύει σε DEV αλλά όχι PROD και εμφανίζει:

⟨/⟩ Plain text

```
ModuleNotFoundError
```

Ποιες δύο κατηγορίες ελέγχεις νωρίς;

Απάντηση

⟨/⟩ Plain text

```
Databricks Runtime version
```

Python/library environment

25

Το notebook δουλεύει όταν το εκτελείς εσύ αλλά scheduled job παίρνει:

⟨/⟩ Plain text

PERMISSION_DENIED

Πρώτη σοβαρή υπόθεση;

Απάντηση

Διαφορετική **execution identity/principal**.

26

Query:

⟨/⟩ Python

```
(
    spark.table("orders")
      .groupBy("customer_id")
      .sum("amount")
)
```

Εμφανίζει Exchange στο plan.

Τι πιθανότατα σημαίνει;

Απάντηση

Data redistribution/shuffle required για το aggregation.

Αυτό είναι άμεση σύνδεση με όσα έμαθες στο Spark.

27 — interview

"What is the difference between a Databricks workspace and compute?"

Καλή απάντηση:

A workspace is the collaborative and operational environment that contains development, orchestration, governance and analytics interfaces, whereas compute is the execution infrastructure used to actually process workloads submitted from those interfaces.

28 — interview

"Why would you choose a SQL warehouse instead of general-purpose Spark compute?"

Καλή απάντηση:

A SQL warehouse is purpose-built for SQL analytics and BI workloads and provides SQL-oriented performance, concurrency and scaling capabilities, whereas general notebook or classic compute is suited to broader Spark and programmatic workloads.

29 — interview

"What is Photon?"

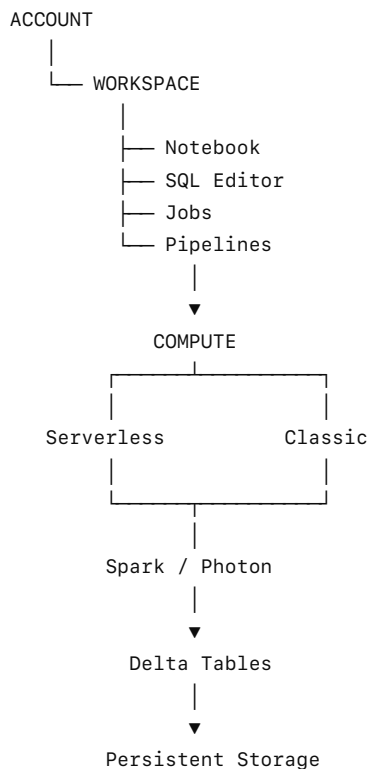
Καλή απάντηση:

Photon is Databricks' native vectorized execution engine that accelerates supported SQL and DataFrame operations underneath the existing APIs, so applications generally do not need a separate Photon-specific programming model.

Πού βρισκόμαστε τώρα

Έχεις πλέον το πρώτο Databricks execution model:

</> Plain text



Και κυρίως έχεις αρχίσει να ξέρεις **πραγματική syntax**:

</> Python

```
spark.table(...)\nspark.sql(...)\ndf.explain(...)\ndisplay(...)\ndbutils.widgets...
```

και:

</> SQL

```
SHOW CATALOGS;\nSHOW SCHEMAS;\nSHOW TABLES;\n\nCREATE SCHEMA;\n\nCREATE TABLE ... USING DELTA;\n\nINSERT INTO ...;\n\nSELECT ...;\n\nUSE CATALOG ...;\nUSE SCHEMA ...;
```

με notebook constructs:

</> Plain text

```
%python\n%sql\n%md\n%pip
```

Οι σημαντικότερες σημερινές επίσημες αναφορές είναι η αρχιτεκτονική Databricks [Databricks Docu...](#), οι current compute-selection recommendations [Databricks Docu...](#), η serverless architecture και οι περιορισμοί της [Databricks Docu...](#) +1, τα SQL warehouses [Databricks Docu...](#), το Photon [Databricks Docu...](#) και η σημερινή notebook syntax [Databricks Docu...](#) +1.

Στο **Databricks Part 2** θα πάμε πλέον πολύ βαθιά στο **Delta Lake**: θα δημιουργήσουμε πραγματικό Delta table και θα ανοίξουμε νοητικά το `_delta_log`, θα δεις `ADD / REMOVE`, `commits`, `snapshots`, `optimistic concurrency`, `UPDATE`, `DELETE`, `MERGE`, `DESCRIBE HISTORY`, `Time Travel`, `VACUUM`, `schema enforcement/evolution` και πώς κάνουμε debugging όταν ένα Delta write αποτυγχάνει.

[Next](#)

Worked for 55s

Databricks — Part 2: Delta Lake σε βάθος

Στο προηγούμενο μάθημα είδαμε το Databricks από πάνω προς τα κάτω:

</> Plain text

```
Notebook / SQL / Job
  ↓
  Compute
  ↓
Spark / Photon
  ↓
Delta tables
  ↓
Cloud storage
```

Τώρα θα ανοίξουμε το κουτί **Delta table**.

Ο στόχος δεν είναι να θυμάσαι απλώς ότι «Delta = Parquet + transaction log». Θέλω, όταν δεις ένα `MERGE` , `UPDATE` , `concurrency error` ή `VACUUM` , να μπορείς νοητικά να προβλέψεις **τι συμβαίνει στα files και στο `_delta_log`** .

Η σημερινή Databricks χρησιμοποιεί Delta Lake ως default table format. Το Delta επεκτείνει τα Parquet data files με versioned transaction log, ώστε να παρέχει ACID semantics, metadata management, batch/streaming interoperability και table versioning.

⊕ Databricks Docu... +1

1. Τι υπάρχει πραγματικά μέσα σε ένα Delta table;

Έστω ότι δημιουργείς:

</> SQL

```
CREATE TABLE dev_training.sales.orders (
  order_id BIGINT,
  customer_id BIGINT,
  amount DECIMAL(10,2),
  status STRING
)
USING DELTA;
```

Θέλω να ξεχάσεις για λίγο το SQL table abstraction.

Στο storage layer, conceptually έχεις κάτι σαν:

</> Plain text

```
orders/
|
|— part-00000-....parquet
|— part-00001-....parquet
|— part-00002-....parquet
|
|— _delta_log/
|   |— 00000000000000000000000000000000.json
|   |— 00000000000000000000000000000001.json
|   |— 00000000000000000000000000000002.json
|   |— ...
```

Τα:

</> Plain text

```
part-....parquet
```

κρατούν **τα πραγματικά columnar data**.

Το:

</> Plain text

```
_delta_log/
```

κρατά τη **versioned κατάσταση του table**.

Κάθε κανονικό Delta commit δημιουργεί μια καινούργια atomic version με monotonically increasing αριθμό:

</> Plain text

```
0
1
2
3
...
```

Τα JSON commit files χρησιμοποιούν 20-digit zero-padded filenames, π.χ. 00000000000000000003.json . Το Delta protocol ορίζει κάθε τέτοιο log entry ως atomic σύνολο actions που μετατρέπει το προηγούμενο snapshot στο επόμενο. [GitHub](#) +1

Ένα concrete παράδειγμα

Έστω αρχικά:

</> SQL

```
INSERT INTO dev_training.sales.orders VALUES
(1, 101, 20.00, 'NEW'),
(2, 102, 30.00, 'NEW');
```

Ας υποθέσουμε ότι δημιουργείται:

</> Plain text

```
part-A.parquet
```

που έχει:

</> Plain text

```
order_id | customer_id | amount | status
```

1	101	20.00	NEW
2	102	30.00	NEW

Το transaction log μπορεί conceptually να πει:

</> Plain text

version 1

ADD part-A.parquet

Στην πραγματικότητα το JSON είναι πιο πλούσιο. Πολύ απλοποιημένα:

</> JSON

```
{
  "add": {
    "path": "part-A.parquet",
    "size": 12345,
    "dataChange": true
  }
}
```

Το πραγματικό `add` action μπορεί επίσης να περιέχει partition values, modification time και file-level statistics όπως record counts και min/max values. [GitHub](#)

Η πρώτη μεγάλη ιδέα

Το Delta table **δεν είναι** η λίστα όλων των `.parquet` files που υπάρχουν **physically** στον φάκελο.

Είναι:

το σύνολο των logical files που το transaction log λέει ότι ανήκουν στο συγκεκριμένο snapshot.

Άρα:

</> Plain text

physical storage

≠

logical current table state

Άσκηση

Το storage έχει:

</> Plain text

A.parquet

B.parquet

C.parquet

αλλά το current snapshot αναφέρει μόνο:

</> Plain text

A
C

Είναι το B μέρος του current table;

Λύση

Όχι.

Μπορεί να είναι παλιό file που χρειάζεται για time travel ή περιμένει να καθαριστεί από VACUUM .

2. Τι ακριβώς είναι snapshot;

Έστω transaction history:

</> Plain text

v0:
ADD A
ADD B

v1:
REMOVE A
ADD C

v2:
ADD D

Ας κάνουμε replay.

Στο v0 :

</> Plain text

A B

Μετά v1 :

</> Plain text

REMOVE A
ADD C

=> B C

Μετά v2 :

</> Plain text

```
ADD D
```

```
=> B C D
```

Άρα το snapshot της version 2 είναι:

</> Plain text

```
{B, C, D}
```

Αυτό σημαίνει:

</> SQL

```
SELECT *  
FROM table VERSION AS OF 2;
```

πρέπει λογικά να διαβάσει τα files που αποτελούν **εκείνο το snapshot**, όχι ό,τι είναι current σήμερα.

Το Delta χρησιμοποιεί MVCC — multi-version concurrency control — και οι readers επιλέγουν ένα συνεπές snapshot μέσω του transaction log. [GitHub](#)

3. add και remove δεν σημαίνουν create/delete filesystem operation

Αυτό είναι πολύ σημαντικό.

Ένα:

</> Plain text

```
ADD A.parquet
```

σημαίνει:

αυτό το logical file συμμετέχει πλέον στο snapshot.

Ένα:

</> Plain text

```
REMOVE A.parquet
```

σημαίνει:

αυτό το file δεν συμμετέχει πλέον στο current snapshot.

Δεν σημαίνει απαραίτητα:

</> Plain text

```
rm A.parquet
```

εκείνη τη στιγμή.

Το Delta κρατά `remove records` ως tombstones για κάποιο διάστημα, ώστε παλιότερα snapshots και concurrent readers να συνεχίζουν να λειτουργούν. Η physical διαγραφή γίνεται αργότερα, τυπικά μέσω `VACUUM` . [GitHub](#) +1

4. Πώς γίνεται ένα UPDATE χωρίς database pages;

Έστω file:

</> Plain text

```
A.parquet
```

order_id	amount	status
1	20	NEW
2	30	NEW
3	40	NEW

Κάνεις:

</> SQL

```
UPDATE dev_training.sales.orders
SET status = 'SHIPPED'
WHERE order_id = 2;
```

Η παλιά, θεμελιώδης Delta λογική είναι ότι δεν αλλάξεις arbitrary bytes μέσα στο υπάρχον Parquet file.

Μπορεί να δημιουργηθεί νέο file:

</> Plain text

```
B.parquet
```

1	20	NEW
2	30	SHIPPED
3	40	NEW

και το commit να γίνει:

</> Plain text

```
REMOVE A
ADD B
```

Άρα:

</> Plain text

before:
snapshot → A

after:
snapshot → B

Αυτό είναι το mental model που πρέπει να μάθεις πρώτο.

Όμως υπάρχει μια **πολύ σημαντική σύγχρονη βελτιστοποίηση** που θα δούμε λίγο παρακάτω: deletion vectors.

5. DELETE

Η SQL syntax είναι απλή:

</> SQL

```
DELETE FROM dev_training.sales.orders  
WHERE status = 'CANCELLED';
```

ή:

</> SQL

```
DELETE FROM dev_training.sales.orders;
```

χωρίς WHERE , που διαγράφει λογικά όλα τα rows.

Το DELETE υποστηρίζεται για Delta tables και μπορεί να χρησιμοποιεί αρκετά σύνθετα predicates και subqueries, με ορισμένους περιορισμούς. [Databricks Docu...](#)

Παράδειγμα

Πριν:

</> Plain text

```
1 NEW  
2 CANCELLED  
3 SHIPPED
```

Μετά:

</> Plain text

```
1 NEW  
3 SHIPPED
```

To table state άλλαξε atomically.

Δεν πρέπει άλλος reader να δει:

⟨/⟩ Plain text

```
μισό παλιό
+
μισό νέο
```

6. Modern Delta: Deletion Vectors

Εδώ η αρχική μας εξήγηση γίνεται πιο ενδιαφέρουσα.

Σκέψου τεράστιο Parquet file:

⟨/⟩ Plain text

```
1 GB
2,000,000 rows
```

Θέλεις να διαγράψεις:

⟨/⟩ Plain text

```
1 row
```

Το naïve file-rewrite model σημαίνει:

⟨/⟩ Plain text

```
read huge file
↓
rewrite almost entire huge file
↓
remove old file
↓
add replacement file
```

Πολύ ακριβό.

Η Databricks έχει **deletion vectors (DV)**.

Αντί να ξαναγράψει άμεσα όλο το file, μπορεί conceptually να κρατήσει:

⟨/⟩ Plain text

```
A.parquet
+
DV:
row 72832 is logically deleted
```

Κατά το read:

</> Plain text

```
read A
↓
apply deletion vector
↓
don't return row 72832
```

Σήμερα deletion vectors μπορούν να επιταχύνουν DELETE , UPDATE και MERGE . Για update, η παλιά row version μπορεί να σημειωθεί ως deleted και η νέα version να γραφτεί σε supplemental data file. [Databricks Docu...](#) +1

Αρα διορθώνουμε το mental model

Το παλιό:

</> Plain text

```
UPDATE one row
↓
always rewrite whole Parquet file
```

δεν είναι πλέον γενικά σωστό.

Το σωστό είναι:

</> Plain text

```
logical table change
  ↓
Delta chooses an implementation
  |
  ├── file rewrite
  |
  └── deletion vector + supplemental data
```

ανάλογα με table features/runtime/engine.

Για Delta, deletion vectors πρέπει να είναι enabled στο table· η Databricks συστήνει τη χρήση τους εκτός αν υπάρχει compatibility constraint με παλαιότερους clients. [Databricks Docu...](#) +1

Παράδειγμα ενεργοποίησης:

</> SQL

```
ALTER TABLE dev_training.sales.orders
SET TBLPROPERTIES (
  'delta.enableDeletionVectors' = true
);
```

Προσοχή: table features μπορούν να αναβαθμίσουν το Delta protocol και να κάνουν το table ασύμβατο με παλιότερους Delta clients. [Databricks Docu...](#)

Άσκηση

Έχεις Parquet file 3 GB και διαγράφεις 2 rows.

Γιατί ένα deletion vector μπορεί να είναι πολύ φθηνότερο από immediate file rewrite;

Λύση

Γιατί το engine μπορεί να καταγράψει ότι συγκεκριμένα rows είναι λογικά deleted χωρίς να ξαναγράψει αμέσως τα υπόλοιπα εκατομμύρια rows του 3 GB file.

7. Πώς ανακατασκευάζει το Delta το table αν υπάρχουν χιλιάδες commits;

Φαντάσου:

</> Plain text

```
v0
v1
v2
...
v1,000,000
```

Αν κάθε reader έπρεπε να ξεκινά:

</> Plain text

```
v0
↓
replay v1
↓
replay v2
↓
...
↓
v1,000,000
```

το planning θα γινόταν ακριβό.

Γι' αυτό υπάρχουν **checkpoints**.

Conceptually:

</> Plain text

```
v0 JSON
v1 JSON
v2 JSON
...
v100 checkpoint
v101 JSON
v102 JSON
v103 JSON
```

</> Plain text

και όχι:

</> Plain text

Υπάρχει επίσης **Checkpoint V2**, σχεδιασμένο για καλύτερη scalability και concurrency σε μεγάλα/frequently updated tables.

 Databricks Docu...

Αυτό είναι production rule.

Μην κάνεις:

</> Plain text

```
edit 00000000000000000000152.json
```

ή:

</> Plain text

```
delete some transaction files
```

για να «διορθώσεις» table.

Το log είναι μέρος του transactional protocol.

Χρησιμοποιείς:

</> SQL

UPDATE

```
DELETE
MERGE
RESTORE
ALTER TABLE
VACUUM
OPTIMIZE
```

και τα supported APIs.

9. DESCRIBE HISTORY : το πρώτο βασικό debugging εργαλείο

Τρέξε:

</> SQL

```
DESCRIBE HISTORY dev_training.sales.orders;
```

Θα πάρεις versions σε reverse chronological order και metadata για τις operations, όπως timestamp, user, operation και operation metrics. [Databricks Docu...](#) +1

Conceptually:

</> Plain text

version	timestamp	operation
8	12:30	MERGE
7	12:10	UPDATE
6	11:00	WRITE
5	10:00	DELETE

Αυτό είναι εξαιρετικά χρήσιμο όταν κάποιος λέει:

Τα δεδομένα χάλασαν περίπου στις 12.

Δεν αρχίζεις μαντεύοντας.

Κάνεις:

</> SQL

```
DESCRIBE HISTORY ...
```

και ψάχνεις:

</> Plain text

```
ποια operation άλλαξε το table;
ποιος;
πότε;
τι metrics είχε;
```


10. Time Travel

Έστω current version:

</> Plain text

v8

Θέλεις να δεις πώς ήταν το table στη v5 .

</> SQL

```
SELECT *  
FROM dev_training.sales.orders  
VERSION AS OF 5;
```

ή χρονικά:

</> SQL

```
SELECT *  
FROM dev_training.sales.orders  
TIMESTAMP AS OF '2026-09-15 10:00:00';
```

Δεν αλλάζεις το current table.

Απλώς διαβάζεις παλιότερο snapshot. Delta table history και time travel υποστηρίζουν version- ή timestamp-based access.

 Databricks Docu...

Παράδειγμα debugging

Κάποιος λέει:

Σήμερα οι customer_id έγιναν null.

Μπορείς να κάνεις:

</> SQL

```
SELECT *  
FROM prod.silver.orders  
VERSION AS OF 124;
```

και:

</> SQL

```
SELECT *  
FROM prod.silver.orders  
VERSION AS OF 125;
```

και να συγκρίνεις.

Έτσι βρίσκεις:

</> Plain text

v124 → σωστό

v125 → χαλασμένο

Μετά:

</> SQL

```
DESCRIBE HISTORY prod.silver.orders;
```

και βλέπεις ποια operation δημιούργησε τη v125.

Αυτό είναι **πραγματικό Delta debugging workflow**.

11. RESTORE

Time travel:

διάβασε παλιό snapshot.

Restore:

κάνε παλιότερη κατάσταση ξανά current.

Παράδειγμα:

</> SQL

```
RESTORE TABLE prod.silver.orders  
TO VERSION AS OF 124;
```

ή:

</> SQL

```
RESTORE TABLE prod.silver.orders  
TO TIMESTAMP AS OF '2026-09-15 10:00:00';
```

To RESTORE είναι supported Delta operation. [Databricks Docu...](#)

Πρόσεξε κάτι πολύ σημαντικό.

Αν current version ήταν:

</> Plain text

```
125
```

και κάνεις:

```
</> Plain text
```

```
RESTORE TO 124
```

δεν σημαίνει:

```
</> Plain text
```

```
delete history 125
```

Γίνεται **νέα transaction**.

Conceptually:

```
</> Plain text
```

```
v124 good
```

```
v125 bad
```

```
v126 RESTORE state equivalent to v124
```

Άρα history παραμένει history.

12. Time travel δεν είναι backup

Πολύ συχνό exam/interview trap.

Η Databricks αναφέρει ρητά ότι table history/time travel **δεν πρέπει να θεωρείται long-term backup strategy**. By default, deleted data files έχουν 7-day retention threshold για VACUUM, ενώ transaction-log history retention είναι default 30 days. Για να μπορείς να time travel σε παλαιότερη version, πρέπει να υπάρχουν και το κατάλληλο log history **και τα data files εκείνης της version**. [Databricks Docu...](#)

Άρα:

```
</> Plain text
```

```
history metadata exists
```

δεν εγγυάται μόνο του:

```
</> Plain text
```

```
all old data files still exist
```

13. VACUUM

Τώρα θα γίνει ξεκάθαρο γιατί χρειαζόμαστε `VACUUM` .

Έστω:

</> Plain text

```
v0 → A B
v1 → B C
v2 → C D
```

To current snapshot είναι:

</> Plain text

```
C D
```

Αλλά physically μπορεί να υπάρχουν ακόμα:

</> Plain text

```
A
B
C
D
```

Γιατί;

Επειδή:

</> Plain text

```
A/B
```

ίσως χρειάζονται για παλιότερα snapshots.

Μετά το retention interval, `VACUUM` μπορεί να αφαιρέσει files που δεν είναι πλέον referenced από versions εντός του retention window. Το default Delta retention threshold για deleted files είναι 7 days. [🌐 Databricks Docu...](#)

Syntax:

</> SQL

```
VACUUM dev_training.sales.orders;
```

ή π.χ.:

</> SQL

```
VACUUM dev_training.sales.orders RETAIN 168 HOURS;
```

168 hours = 7 days.

14. Τι ΔΕΝ κάνει το VACUUM

VACUUM δεν σημαίνει:

</> Plain text

speed up by compacting files

Αυτό είναι διαφορετική operation.

</> Plain text

VACUUM
=
garbage collection / physical cleanup

ενώ:

</> Plain text

OPTIMIZE
=
data-file rewrite/layout optimization

Θα μπορούμε πολύ βαθύτερα στο OPTIMIZE , liquid clustering και predictive optimization στο performance μάθημα.

15. Γιατί δεν πρέπει να κάνεις VACUUM RETAIN 0 HOURS στα τυφλά

Αν διαγράψεις πολύ γρήγορα old/uncommitted files, μπορείς να σπάσεις:

</> Plain text

time travel
long-running readers
long-running writers
recovery assumptions

Η Databricks συστήνει retention τουλάχιστον 7 ημερών και έχει safety check για επικίνδυνα μικρό retention. [Databricks Docu...](#)

Άρα σε exam:

"I want to save storage, so VACUUM RETAIN 0 HOURS every hour."

είναι τεράστιο red flag.

16. MERGE : ίσως η σημαντικότερη Delta εντολή για Data Engineer

Ας έχουμε target:

</> Plain text

```
prod.silver.customers
```

Current:

</> Plain text

id	name	city
1	Maria	Athens
2	John	Patras
3	Anna	Volos

Έρχεται source:

</> Plain text

id	name	city
2	John	Athens
4	George	Larisa

Θέλεις:

</> Plain text

```
id=2
```

να γίνει update.

Και:

</> Plain text

```
id=4
```

να γίνει insert.

Αυτό λέγεται συχνά **upsert**:

</> Plain text

```
UPDATE  
+  
INSERT
```

17. Basic MERGE INTO

</> SQL

```
MERGE INTO prod.silver.customers AS target
USING staging.customer_updates AS source

ON target.id = source.id

WHEN MATCHED THEN
  UPDATE SET
    target.name = source.name,
    target.city = source.city

WHEN NOT MATCHED THEN
  INSERT (id, name, city)
  VALUES (source.id, source.name, source.city);
```

Διάβασέ το όπως compiler.

</> Plain text

```
TARGET
=
ο πραγματικός πίνακας που θέλω να αλλάξω

SOURCE
=
τα νέα δεδομένα

ON
=
πότε source row και target row θεωρούνται η ίδια entity
```

Μετά:

</> Plain text

```
matched?
  YES → UPDATE

matched?
  NO  → INSERT
```

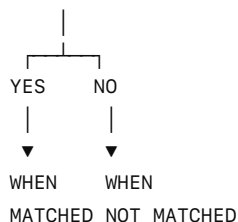
18. MERGE ως Venn diagram

Για source row:

</> Plain text

```
SOURCE row
|
▼
```

Does target contain matching key?



Υπάρχει και τρίτη περίπτωση:

</> Plain text

TARGET row exists
but no matching source row

Αυτό είναι:

</> SQL

WHEN NOT MATCHED BY SOURCE

19. WHEN NOT MATCHED BY SOURCE

Έστω source θεωρείται authoritative snapshot.

Target:

</> Plain text

1
2
3
4

Source:

</> Plain text

1
2
4

To:

</> Plain text

3

υπάρχει target αλλά όχι source.

Μπορείς:

</> SQL

```
MERGE INTO target t
USING source s
ON t.id = s.id

WHEN MATCHED THEN
  UPDATE SET *

WHEN NOT MATCHED THEN
  INSERT *

WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

Η syntax `WHEN NOT MATCHED BY SOURCE` υποστηρίζει `UPDATE` ή `DELETE` . [Databricks Docu...](#) +1

Αλλά πρόσεξε.

Αν το `source` είναι απλώς:

</> Plain text

```
"today's changed records"
```

και όχι full authoritative dataset, τότε:

</> SQL

```
WHEN NOT MATCHED BY SOURCE THEN DELETE
```

μπορεί να διαγράψει σχεδόν όλο το `target`.

Γι' αυτό χρειάζεται να ξέρεις **τι semantics έχει το source**, όχι απλώς SQL syntax.

20. UPDATE SET * και INSERT *

Αν schemas είναι συμβατά:

</> SQL

```
MERGE INTO target t
USING source s
ON t.id = s.id

WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Αυτό είναι βολικό.

Σε production όμως μερικές φορές προτιμάς explicit mapping:

</> SQL

```
UPDATE SET
  t.name = s.name,
  t.city = s.city,
  t.updated_at = current_timestamp()
```

ώστε να ξέρεις ακριβώς τι αλλάζει.

21. Το πιο κοινό MERGE failure: multiple source rows

Target:

</> Plain text

```
id=7
```

Source:

</> Plain text

```
id=7 city=Athens
id=7 city=Volos
```

Και κάνεις:

</> SQL

```
MERGE ...
ON target.id = source.id
WHEN MATCHED THEN UPDATE ...
```

Τι πρέπει να γίνει στο target;

</> Plain text

```
Athens;
Volos;
```

Δεν υπάρχει μονοσήμαντη απάντηση.

Άρα η operation είναι ambiguous και μπορεί να αποτύχει με multiple-source-row matching error. Η Databricks απαιτεί να preprocess το source ώστε να μην υπάρχουν ambiguous multiple matches. Από DBR 16.0+, duplicate-match determination εξετάζει τόσο το ON όσο και relevant WHEN MATCHED conditions· παλαιότερες 15.4 LTS και κάτω χρησιμοποιούσαν μόνο το

ON .  Databricks Docu... +1

22. Πώς διορθώνεις duplicate CDC rows πριν από MERGE ;

Έρχεται:

</> Plain text

```
id | seq | city
---+-----+-----
7  | 10  | Athens
7  | 11  | Volos
```

Θέλεις latest:

</> Plain text

```
seq=11
```

Μπορείς να χρησιμοποιήσεις window:

</> SQL

```
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY id
      ORDER BY seq DESC
    ) AS rn
  FROM staging.customer_changes
)

SELECT *
FROM ranked
WHERE rn = 1;
```

Έτσι:

</> Plain text

```
id=7
```

εμφανίζεται μόνο μία φορά.

Και μετά κάνεις merge το deduplicated source.

Αυτό συνδέει άμεσα:

</> Plain text

```
window functions
+
CDC
+
Delta MERGE
```

με πραγματικό production use case.

23. MERGE σε Python

Η κλασική Delta API μορφή:

</> Python

```
from delta.tables import DeltaTable

target = DeltaTable.forName(
    spark,
    "prod.silver.customers"
)

source = spark.table(
    "staging.customer_updates"
)

(
    target.alias("t")
    .merge(
        source.alias("s"),
        "t.id = s.id"
    )
    .whenMatchedUpdate(
        set={
            "name": "s.name",
            "city": "s.city"
        }
    )
    .whenNotMatchedInsert(
        values={
            "id": "s.id",
```

Η Delta Python API και η SQL `MERGE` υλοποιούν το ίδιο θεμελιώδες source-target matching model. [Databricks Docu...](#)

Σε σύγχρονο PySpark/Databricks υπάρχει επίσης `DataFrame mergeInto()` API, αλλά θέλω πρώτα να είσαι απόλυτα άνετος με SQL `MERGE` και `DeltaTable`, επειδή θα τα συναντάς παντού σε υπάρχον production code. [Databricks Docu...](#)

24. UPDATE ... FROM ... JOIN trap

Αν έρχεσαι από PostgreSQL/SQL Server dialect μπορεί να περιμένεις:

</> SQL

```
UPDATE target
SET ...
FROM source
WHERE ...
```

H Databricks SQL δεν υποστηρίζει το κλασικό `UPDATE ... FROM ... JOIN` pattern.

Για update από άλλο relation χρησιμοποιείς:

</> SQL

```
MERGE INTO ...  
  ```:chatgpt-content-reference{index="21"}
```

Αυτό είναι πολύ καλό exam/interview detail.

---

# 25. Schema Enforcement

Έχεις target:

```
```text  
id BIGINT  
amount DECIMAL(10,2)
```

και DataFrame:

</> Plain text

```
id BIGINT  
amount STRUCT<foo:INT>
```

Default expectation:

</> Plain text

```
write fails
```

αν η incoming schema δεν είναι συμβατή.

Αυτό είναι καλό.

Δεν θέλεις pipeline:

</> Plain text

```
bad producer  
↓  
silent corruption  
↓  
Silver  
↓  
Gold  
↓  
CEO dashboard
```

To Delta χρησιμοποιεί transaction-log metadata για schema validation. [Databricks Docu...](#) +1

26. Schema Evolution: προσθήκη νέου column

Target:

</> Plain text

```
id
name
```

Source:

</> Plain text

```
id
name
email
```

Χωρίς evolution μπορεί να πάρεις mismatch.

Με DataFrame API:

</> Python

```
(
    source_df.write
        .option("mergeSchema", "true")
        .mode("append")
        .saveAsTable("prod.silver.customers")
)
```

Το νέο:

</> Plain text

```
email
```

μπορεί να προστεθεί στο table schema. Η Databricks προτείνει schema evolution **per write** μέσω `mergeSchema` ή explicit SQL syntax αντί για session-wide `spark.databricks.delta.schema.autoMerge.enabled`, επειδή το global setting μπορεί να προκαλέσει ανεπιθύμητες schema changes σε άσχετα writes. [Databricks Docu...](#)

27. Νεότερη SQL syntax

Σε σύγχρονο Databricks:

</> SQL

```
MERGE WITH SCHEMA EVOLUTION INTO target t
USING source s
ON t.id = s.id
```

```
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Η WITH SCHEMA EVOLUTION μορφή επιτρέπει το MERGE να εξελίξει schema στο συγκεκριμένο operation. [Databricks Docu...](#) +1

Ακόμη νεότερα, σε DBR 18.1+, υπάρχει και:

</> SQL

```
INSERT WITH SCHEMA EVOLUTION INTO target
SELECT *
FROM source;
```:chatgpt-content-reference{index="25"}
```

Αρα για 2026 certification/real projects θέλω να ξέρεις **τη νέα syntax**, αλλά και το:

```
```python
.option("mergeSchema", "true")
```

επειδή θα το δεις παντού.

28. Manual schema evolution

Αν η αλλαγή είναι intentional, μπορείς να κάνεις:

</> SQL

```
ALTER TABLE prod.silver.customers
ADD COLUMNS (
    email STRING
);
```

Rename:

</> SQL

```
ALTER TABLE prod.silver.customers
RENAME COLUMN name TO full_name;
```

Drop:

</> SQL

```
ALTER TABLE prod.silver.customers
DROP COLUMN obsolete_field;
```

Metadata-only rename/drop απαιτούν κατάλληλο column mapping support. [Databricks Docu...](#) +1

29. Type widening

Έστω:

```
</> Plain text
```

```
quantity INT
```

και αργότερα χρειάζεσαι:

```
</> Plain text
```

```
quantity BIGINT
```

Αυτό είναι **widening**.

Δεν είναι το ίδιο με αυθαίρετο:

```
</> Plain text
```

```
INT → STRING
```

Το Delta υποστηρίζει type widening ως table feature για supported compatible conversions. [Databricks Docu...](#)

Enable:

```
</> SQL
```

```
ALTER TABLE prod.silver.orders
SET TBLPROPERTIES (
  'delta.enableTypeWidening' = 'true'
);
```

Μετά, supported manual widening:

```
</> SQL
```

```
ALTER TABLE prod.silver.orders
ALTER COLUMN quantity TYPE BIGINT;
```

30. Schema changes και concurrency

Πολύ σημαντικό production caveat:

schema changes είναι metadata changes.

Και metadata changes μπορούν να conflict με concurrent writes.

Η Databricks συνιστά να συντονίζεις schema updates, επειδή table schema changes μπορούν να προκαλέσουν conflict σε concurrent writes και επίσης να σταματήσουν streams που διαβάζουν το table μέχρι να επανεκκινήσουν. [Databricks Docu...](#)

Άρα:

</> Plain text

```
11:59
production streaming pipeline running
```

```
12:00
ALTER TABLE ...
```

```
12:00
writer fails
```

δεν είναι απαραίτητα bug.

Μπορεί να είναι expected concurrency semantics.

31. Τώρα το δύσκολο μέρος: Optimistic Concurrency Control

Έστω δύο jobs:

</> Plain text

```
Job A
Job B
```

Και current table version:

</> Plain text

```
v50
```

Και τα δύο ξεκινούν περίπου μαζί.

</> Plain text

```
A reads snapshot v50
B reads snapshot v50
```

To A:

</> Plain text

```
wants to update customer 10
```

To B:

</> Plain text

```
wants to update customer 20
```

Delta δεν κάνει απαραίτητα pessimistic lock:

</> Plain text

```
LOCK ENTIRE TABLE
everyone wait
```

Αντίθετα χρησιμοποιεί **optimistic concurrency control**.

Conceptually:

</> Plain text

```
1. READ
   read consistent snapshot

2. WRITE
   prepare candidate output

3. VALIDATE
   did another commit happen,
   and does it conflict with me?

4. COMMIT
   if safe → new atomic version

   otherwise → fail/retry
```

Η Databricks περιγράφει ακριβώς αυτό το read → write → validate-and-commit model. [Databricks Docu...](#)

32. Γιατί λέγεται optimistic;

Γιατί το σύστημα υποθέτει:

Οι περισσότερες concurrent transactions πιθανόν να μην conflict-άρουν.

Οπότε δεν λέει εξαρχής:

</> Plain text

```
A lock table
B wait
```

Αφήνει:

</> Plain text

```
A work
B work
```

και στο commit ελέγχει:

</> Plain text

Can both results coexist consistently?

Αν ναι:

</> Plain text

commit

Αν όχι:

</> Plain text

conflict

33. Atomic version creation

Έστω:

</> Plain text

current = v50

Α κάνει commit:

</> Plain text

v51

Β προσπαθεί να commit βασισμένο στο v50.

Τώρα πρέπει να ελέγξει:

</> Plain text

Τι άλλαξε από v50 → v51;

Επηρεάζει rows/files/data
που εγώ διάβασα ή αλλάζω;

Αν δεν conflict-άρει, μπορεί τελικά να παράγει:

</> Plain text

v52

Αν conflict-άρει:

</> Plain text

```
transaction fails
```

ώστε να μη corrupt-άρει το table.

34. Isolation levels

Σήμερα Delta Lake στο Databricks υποστηρίζει:

</> Plain text

```
WriteSerializable  
Serializable
```

To default είναι:

</> Plain text

```
WriteSerializable
```

Τα reads χρησιμοποιούν snapshot isolation. `Serializable` είναι ισχυρότερο και απαιτεί το αποτέλεσμα concurrent reads/writes να μπορεί να εξηγηθεί από πλήρως serial execution order. [Databricks Docu...](#) +1

Αλλαγή:

</> SQL

```
ALTER TABLE my_table  
SET TBLPROPERTIES (  
    'delta.isolationLevel' = 'Serializable'  
);
```

Για τώρα δεν θέλω να αποστηθίσεις λεπτομερώς κάθε conflict matrix.

Θέλω να θυμάσαι:

</> Plain text

```
readers  
→ consistent snapshots  
  
writers  
→ optimistic validation  
  
default writes  
→ WriteSerializable
```

35. Row-level concurrency

Παλιότερα:

</> Plain text

```
A updates row 1 in file X
B updates row 9000 in same file X
```

Θα μπορούσαν να conflict-άρουν επειδή και οι δύο επηρέαζαν:

</> Plain text

```
file X
```

Με row-level concurrency + deletion vectors, Databricks μπορεί να εντοπίζει conflicts σε granular row level.

Άρα:

</> Plain text

```
A changes row 1
B changes row 9000
```

μπορεί να μη conflict-άρουν, παρότι τα δύο rows βρίσκονται στο ίδιο physical file.

Στη σημερινή documentation, row-level concurrency ενεργοποιείται σε supported DBR για unpartitioned tables με deletion vectors, ενώ liquid-clustered tables αξιοποιούν επίσης αυτή την αρχιτεκτονική. [Databricks Docu...](#)

36. Real concurrency errors

Μπορεί να δεις errors όπως:

</> Plain text

```
DELTA_CONCURRENT_WRITE
DELTA_CONCURRENT_APPEND
DELTA_CONCURRENT_DELETE_READ
MetadataChangedException
ProtocolChangedException
ConcurrentTransactionException
```

Δεν σημαίνουν:

Delta table corrupted.

Σημαίνουν συχνά:

To optimistic concurrency system απέτρεψε unsafe commit.

Η Databricks προτείνει retry logic για legitimate transient transaction conflicts αφού ξαναδιαβαστεί fresh state.

[Databricks Docu...](#) +2

37. Debugging concurrency

Έστω:

</> Plain text

```
MERGE failed:
DELTA_CONCURRENT_WRITE
```

Κακό response:

</> Plain text

Run it again until it works.

Καλύτερη σκέψη:

</> Plain text

```
What other writer touched this table?

What rows/files/partitions did both operations read?

Are predicates too broad?

Can I narrow the MERGE condition?

Is a schema ALTER happening concurrently?

Are deletion vectors / row-level concurrency available?

Should the application retry?
```

Ειδικά ένα:

</> SQL

```
MERGE
ON t.id = s.id
```

σε τεράστιο table μπορεί να έχει broad read scope.

Αν έχεις επιπλέον φυσικό/time-bound scope:

</> SQL

```
ON t.id = s.id
AND t.event_date = DATE'2026-09-16'
```

μπορεί να μειώσεις τόσο το scan όσο και το conflict domain, όταν αυτό είναι semantically σωστό. [Databricks Docu...](#)

38. Το OPTIMIZE και γιατί διαφέρει από VACUUM

Έστω writes παράγουν:

</> Plain text

```
10 MB
12 MB
8 MB
15 MB
...
```

χιλιάδες μικρά Parquet files.

Αυτό δημιουργεί overhead:

</> Plain text

```
file listing
metadata
task scheduling
small reads
```

Το:

</> SQL

```
OPTIMIZE dev_training.sales.orders;
```

μπορεί να compact μικρά files σε μεγαλύτερα.

Η current Databricks επίσης προτείνει liquid clustering και predictive optimization για managed tables, ώστε μεγάλο μέρος αυτής της συντήρησης να αυτοματοποιείται. [Databricks Docu...](#) +1

Conceptually:

</> Plain text

```
A B C D E F
many small files

↓ OPTIMIZE

X Y
fewer larger files
```

Το logical data μπορεί να μην αλλάξει.

Αλλά transaction log μπορεί να γράψει:

</> Plain text

```
REMOVE A
```

```
REMOVE B
REMOVE C
...
ADD X
ADD Y
```

με actions που σημαίνουν data rearrangement rather than logical record changes. Το Delta protocol έχει `dataChange` flag ακριβώς για να ξεχωρίζει operations που αναδιατάσσουν υπάρχον data χωρίς logical data change. [GitHub](#)

39. Liquid clustering: μόνο το πρώτο mental model

Παλιά πολλά systems βασίζονταν σε:

⟨/⟩ Plain text

```
partitioning
+
ZORDER
```

Για νέα Delta tables, η Databricks σήμερα προτείνει **liquid clustering** ως modern layout strategy. Τα clustering keys μπορούν να αλλάζουν καθώς αλλάζουν τα query patterns και το clustering εφαρμόζεται incrementally. [Databricks Docu...](#) +1

Syntax:

⟨/⟩ SQL

```
ALTER TABLE prod.silver.orders
CLUSTER BY (customer_id);
```

και:

⟨/⟩ SQL

```
OPTIMIZE prod.silver.orders;
```

Αλλά θα το μελετήσουμε χωριστά μαζί με:

⟨/⟩ Plain text

```
data skipping
file statistics
predictive optimization
small files
OPTIMIZE FULL
clustering key selection
```

γιατί αλλιώς θα το μάθεις επιφανειακά.

40. Πλήρες real-world MERGE example

Ας χτίσουμε κάτι που μοιάζει με production pipeline.

Target:

</> SQL

```
CREATE TABLE IF NOT EXISTS dev_training.silver.customers (  
  customer_id BIGINT,  
  name STRING,  
  city STRING,  
  updated_at TIMESTAMP  
)  
USING DELTA;
```

Initial:

</> SQL

```
INSERT INTO dev_training.silver.customers VALUES  
(1, 'Maria', 'Athens', current_timestamp()),  
(2, 'John', 'Patras', current_timestamp()),  
(3, 'Anna', 'Volos', current_timestamp());
```

Staging table:

</> SQL

```
CREATE OR REPLACE TABLE dev_training.staging.customer_updates AS  
  
SELECT * FROM VALUES  
  (2L, 'John', 'Athens'),  
  (4L, 'George', 'Larisa')  
AS t(customer_id, name, city);
```

Merge:

</> SQL

```
MERGE INTO dev_training.silver.customers AS t  
  
USING dev_training.staging.customer_updates AS s  
  
ON t.customer_id = s.customer_id  
  
WHEN MATCHED THEN UPDATE SET  
  t.name = s.name,  
  t.city = s.city,  
  t.updated_at = current_timestamp()  
  
WHEN NOT MATCHED THEN INSERT (  
  customer_id,  
  name,  
  city,  
  updated_at
```

```
        updated_at
    )
VALUES (
    s.customer_id,
    s.name,
    s.city,
    current_timestamp()
);
```

Result:

</> Plain text

1 Maria Athens
2 John Athens
3 Anna Volos
4 George Larisa

Μετά:

</> SQL

DESCRIBE HISTORY dev_training.silver.customers;

Θα δεις MERGE ως νέα table version.

Αυτό είναι το πρώτο πράγμα που θέλω να δοκιμάσεις πρακτικά.

41. Ένα production bug που θα συναντήσεις

Source:

</> Plain text

customer_id | name | city
-----+-----+-----
2 | John | Athens
2 | John | Volos

Merge:

</> SQL

ON t.customer_id = s.customer_id

Failure.

Debug sequence:

</> Plain text

```
Is target key unique?
↓
Does source contain duplicates?
↓
Which source row is latest?
↓
Is there event sequence / updated_at?
↓
deduplicate
↓
merge
```

PySpark dedup:

</> Python

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

w = (
    Window
    .partitionBy("customer_id")
    .orderBy(F.col("sequence_number").desc())
)

latest = (
    source_df
    .withColumn(
        "rn",
        F.row_number().over(w)
    )
    .filter(F.col("rn") == 1)
    .drop("rn")
)
```

Αυτό είναι πραγματικό data-engineering pattern, όχι toy window-function exercise.

42. Ένα δεύτερο production bug: schema mismatch

Χθες:

</> Plain text

```
customer_id LONG
name STRING
city STRING
```

Σήμερα source:

</> Plain text

```
customer_id LONG
name STRING
city STRING
```

```
segment STRING
```

Write:

```
</> Python
```

```
source_df.write.mode("append").saveAsTable(...)
```

fails.

Μη γράψεις αμέσως:

```
</> Python
```

```
spark.conf.set(
    "spark.databricks.delta.schema.autoMerge.enabled",
    "true"
)
```

για ολόκληρο session.

Πρώτα ρώτα:

```
</> Plain text
```

To segment είναι intentional addition;

Αν ναι:

```
</> Python
```

```
(
    source_df.write
        .option("mergeSchema", "true")
        .mode("append")
        .saveAsTable(
            "prod.silver.customers"
        )
)
```

ή κάνε explicit ALTER TABLE .

Η Databricks σήμερα συστήνει per-write schema-evolution controls αντί global autoMerge configuration για production.

 Databricks Docu...

43. Τρίτο production bug: DELETE έγινε σωστά αλλά storage δεν μειώθηκε

Έχεις 10 TB.

Κάνεις:

```
</> SQL
```

```
DELETE FROM orders
WHERE event_date < DATE'2020-01-01';
```

Query:

```
</> SQL
```

```
SELECT ...
```

δεν επιστρέφει πλέον αυτά τα rows.

Αλλά cloud storage usage παραμένει κοντά στα 10 TB.

Είναι απαραίτητα bug;

Όχι.

Γιατί:

```
</> Plain text
```

```
logical delete
≠
immediate physical file deletion
```

Old files μπορεί να κρατιούνται για snapshots/time travel.

Με deletion vectors, μπορεί ακόμα και το original file να παραμένει current physical input με συγκεκριμένα rows λογικά masked.

Για πραγματικό physical purge μπορεί να χρειαστεί:

```
</> SQL
```

```
REORG TABLE ... APPLY (PURGE)
```

και στη συνέχεια, αφού λήξει το retention που απαιτείται:

```
</> SQL
```

```
VACUUM ...
```

Η Databricks το περιγράφει ρητά για data που έχει soft-deleted μέσω deletion vectors. [Databricks Docu...](#)

44. Τέταρτο production bug: time travel σταμάτησε να δουλεύει

Χθες:

```
</> SQL
```

```
SELECT *  
FROM orders VERSION AS OF 80;
```

δούλεψε.

Σήμερα όχι.

Πρώτο πράγμα που ελέγχεις:

```
</> SQL
```

```
DESCRIBE HISTORY orders;
```

και:

```
</> Plain text
```

```
Έτρεξε VACUUM;  
Ποιο retention;  
Υπάρχουν ακόμα τα data files του v80;
```

Το VACUUM μπορεί να αφαιρέσει physical files που απαιτούνται από παλιότερες versions, οπότε η ύπαρξη history entry δεν εγγυάται ότι εκείνο το version παραμένει queryable. [Databricks Docu...](#) +1

45. Πέμπτο production bug: δύο jobs κάνουν update ταυτόχρονα

Job A:

```
</> SQL
```

```
UPDATE customers  
SET status='A'  
WHERE id=10;
```

Job B:

```
</> SQL
```

```
UPDATE customers  
SET status='B'  
WHERE id=10;
```

Τώρα υπάρχει πραγματικό logical conflict.

Δεν θέλουμε:

```
</> Plain text
```

random corruption

Θέλουμε:

</> Plain text

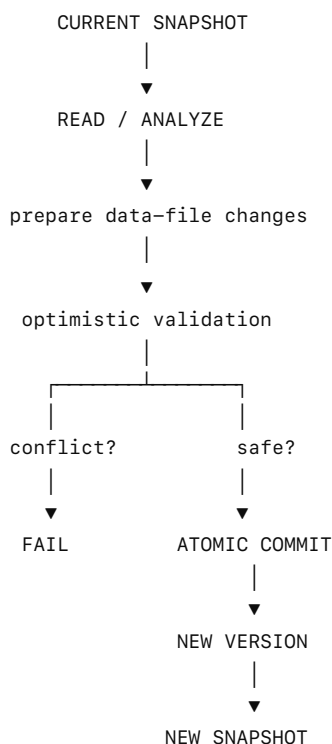
```
one commit
+
other fails or is serialized safely
```

Το optimistic concurrency layer αποτρέπει inconsistent commit. Με row-level concurrency, independent rows μπορούν συχνά να αλλάξουν ταυτόχρονα, αλλά same-row modifications εξακολουθούν να conflict-άρουν. [Databricks Docu...](#)

46. Το mental model που πρέπει να κρατήσεις

Για κάθε Delta write, σκέψου:

</> Plain text



Και από storage πλευρά:

</> Plain text

```
Parquet files
+
optional deletion vectors
+
_delta_log versions
+
checkpoints
```

```
=  
Delta table state
```

Αν αυτό γίνει instinctive, τα περισσότερα Delta concepts παύουν να είναι αποστήθιση.

Μεγάλο τεστ Part 2

1. Ένα Delta table έχει 50 Parquet files physically. Το current snapshot αναφέρει 37. Πόσα είναι logically current;

Απάντηση: 37. Physical existence δεν συνεπάγεται current logical membership.

2. Τι σημαίνει remove action;

Απάντηση: Ότι ένα logical file παύει να ανήκει στο snapshot. Δεν σημαίνει απαραίτητα immediate physical deletion.

3. Γιατί το Delta κρατά old files;

Απάντηση: Για snapshot isolation, concurrent readers, time travel και recovery/history semantics μέχρι να λήξει το retention και να γίνει cleanup.

4. Τι κάνει checkpoint;

Απάντηση: Αποθηκεύει συνοπτική table state ώστε να μη χρειάζεται replay όλου του transaction-log history από version 0.

5. Γιατί δεν πρέπει application code να βασίζεται στο ότι checkpoint γίνεται κάθε ακριβώς N commits;

Απάντηση: Η checkpoint frequency είναι implementation/performance detail και η Databricks μπορεί να την προσαρμόζει.

6. Τι κάνει DESCRIBE HISTORY ;

Απάντηση: Δείχνει τις table versions και provenance/operation metadata για κάθε write.

7. Ποια η διαφορά VERSION AS OF και RESTORE ... VERSION AS OF ;

Απάντηση: Το πρώτο διαβάξει παλιό snapshot. Το δεύτερο δημιουργεί νέα current version που επαναφέρει το table state σε παλιότερη κατάσταση.

8. Μετά από restore σε v7 ενώ current ήταν v10, διαγράφονται οι versions 8–10;

Απάντηση: Όχι. Δημιουργείται νέα version, π.χ. v11.

9. Είναι το Delta history μακροχρόνιο backup system;

Απάντηση: Όχι. Retention και VACUUM περιορίζουν το διαθέσιμο history/time travel.

10. Τι κάνει VACUUM ;

Απάντηση: Φυσικό cleanup παλιών/unreferenced data files αφού ικανοποιηθεί το retention policy.

11. Τι κάνει OPTIMIZE ;

Απάντηση: Αναδιοργανώνει/compacts physical data files για καλύτερο layout και performance. Δεν είναι garbage collection.

12. Γιατί deletion vector βοηθά DELETE ;

Απάντηση: Μπορεί να σημαδέψει συγκεκριμένα rows ως logically removed χωρίς άμεσο rewrite ολόκληρου Parquet file.

13. Τι γίνεται κατά το read table που έχει DV;

Απάντηση: Ο reader συνδυάζει το underlying data file με το deletion-vector state και δεν επιστρέφει logically deleted rows.

14. Τι είναι MERGE ;

Απάντηση: Atomic source-to-target operation που μπορεί να εφαρμόσει updates, inserts και deletes ανάλογα με matching conditions.

15. Τι σημαίνει WHEN MATCHED ;

Απάντηση: Source row βρήκε target row που ικανοποιεί το ON condition.

16. Τι σημαίνει WHEN NOT MATCHED ;

Απάντηση: Source row δεν έχει matching target row· συνήθως χρησιμοποιείται για insert.

17. Τι σημαίνει WHEN NOT MATCHED BY SOURCE ;

Απάντηση: Target row δεν έχει matching source row και μπορεί να update/delete-αριστεί.

18. Γιατί είναι επικίνδυνο το unconditional WHEN NOT MATCHED BY SOURCE THEN DELETE ;

Απάντηση: Αν το source είναι incremental αντί για full authoritative snapshot, μπορεί να διαγράψει legitimate target rows που απλώς δεν περιλαμβάνονταν σε εκείνο το batch.

19. Source έχει δύο rows με ίδιο business key που και τα δύο match-άρουν το ίδιο target update. Τι κάνεις;

Απάντηση: Deduplicate/rank source πρώτα ώστε η επιλογή να είναι deterministic.

20. Ποιο εργαλείο θα χρησιμοποιούσες για latest CDC event ανά key;

Απάντηση: Συνήθως window ROW_NUMBER() OVER (PARTITION BY key ORDER BY sequence DESC) και κρατάς rn=1 .

21. Ποια η διαφορά schema enforcement και evolution;

Απάντηση: Enforcement απορρίπτει incompatible data. Evolution επιτρέπει συγκεκριμένες controlled αλλαγές στο schema.

22. Γιατί autoMerge=true global Spark config δεν είναι η προτιμώμενη production προσέγγιση;

Απάντηση: Μπορεί να επιτρέψει schema changes σε πολλά unrelated writes και κάνει δυσκολότερο να καταλάβεις ποια operation είχε δικαίωμα να εξελίξει schema.

23. Ποια είναι καλύτερη μορφή;

```
</> Python
.option("mergeSchema", "true")
```

ανά συγκεκριμένο write, επειδή το behavior είναι localized και explicit.

24. Τι είναι type widening;

Απάντηση: Μετατροπή προς compatible ευρύτερο datatype, π.χ. INT → BIGINT , αντί arbitrary semantic αλλαγής τύπου.

25. Τι είναι optimistic concurrency control;

Απάντηση: Transactions δουλεύουν πάνω σε snapshot χωρίς να κλειδώνουν pessimistically όλο το table και κατά το commit ελέγχεται αν concurrent changes conflict-άρουν.

26. Γιατί λέγεται optimistic;

Απάντηση: Επειδή υποθέτει αρχικά ότι concurrent transactions πιθανώς μπορούν να συνυπάρξουν και ελέγχει το conflict

αργότερα στο validation/commit.

27. Ποιο είναι το default isolation level για writes στο Databricks Delta;

Απάντηση: WriteSerializable . Reads χρησιμοποιούν snapshot isolation. [Databricks Docu...](#)

28. A και B αλλάζουν διαφορετικά rows του ίδιου file. Είναι υποχρεωτικό να conflict-άρουν;

Απάντηση: Όχι πλέον. Με deletion vectors και row-level concurrency μπορούν να συνυπάρξουν αν δεν αλλάζουν το ίδιο row και ικανοποιούνται τα σχετικά requirements. [Databricks Docu...](#)

29. A και B αλλάζουν το ίδιο row. Τι περιμένεις;

Απάντηση: Πραγματικό write conflict ή serialized resolution· δεν μπορούν απλώς να γίνουν και οι δύο ανεξέλεγκτα.

30. MetadataChangedException κατά τη διάρκεια write. Τι μπορεί να έγινε;

Απάντηση: Concurrent operation άλλαξε table metadata, π.χ. schema μέσω ALTER TABLE ή schema-evolving write.

31. Μετά από DELETE , query δεν επιστρέφει rows αλλά storage δεν μειώθηκε. Είναι contradiction;

Απάντηση: Όχι. Logical table state και physical garbage collection είναι διαφορετικά στάδια.

32. Γιατί μπορεί ένα old .parquet file να υπάρχει αλλά να μην το διαβάσει το current SELECT ;

Απάντηση: Επειδή δεν ανήκει πλέον στο current transaction-log snapshot.

33. Ένα corrupted pipeline εμφανίστηκε στη v101. Τι θα έκανες πρώτα;

Απάντηση: DESCRIBE HISTORY , σύγκριση v100/v101 μέσω time travel, εντοπισμός operation που δημιούργησε v101 και έπειτα απόφαση για fix ή RESTORE .

34. Γιατί MERGE είναι ιδιαίτερα σημαντικό σε CDC;

Απάντηση: Επειδή ένα change feed περιέχει συνήθως inserts, updates και deletes για υπάρχον entities και το MERGE μπορεί atomically να συγχρονίσει αυτά τα changes με target state.

35. Interview: «Parquet already stores data, so why do I need Delta?»

Καλή απάντηση:

Parquet defines a columnar file representation, but it does not by itself define transactional table state across many files. Delta adds a versioned transaction log, atomic commits, snapshot isolation, schema management, DML such as merge/update/delete, concurrency control and time travel over those files.

Τι πρέπει πλέον να μπορείς να σχεδιάσεις στο κεφάλι σου

Όταν βλέπεις:

```
</> SQL
```

```
MERGE INTO prod.silver.customers ...
```

δεν θέλω να σκέφτεσαι μόνο:

κάνει update/insert.

Θέλω να βλέπεις:

</> Plain text

```
source rows
  ↓
matching
  ↓
read target snapshot
  ↓
determine changed rows/files
  ↓
write new data / deletion vectors
  ↓
optimistic conflict validation
  ↓
atomic transaction-log commit
  ↓
new table version
  ↓
new consistent snapshot
```

Αυτό είναι το επίπεδο κατανόησης που χρειάζεσαι για να περάσεις από **Databricks user** σε **Databricks data engineer**.

Στο **Part 3** θα περάσουμε στο **Unity Catalog σε βάθος**: `catalog.schema.object` , managed vs external tables, managed storage, volumes, external locations, storage credentials, ownership, `USE CATALOG` , `USE SCHEMA` , `SELECT` , `MODIFY` , privilege inheritance, service principals, row filters, column masks, lineage και κυρίως πραγματικά `PERMISSION_DENIED` debugging scenarios.

Wed, Sep 16 at 11:17 PM

I need a list with all available computes. Also i didn't understand them and i also didn't understand the different parameters and whether the serverlessness or all-purpose-ness are different axes or just parameters. What combinations of configs can be made given the different axes, in a matrix form?

Dwelve more in computes and be more explanatory because i am a newbie

Worked for 40s

Ναι — εδώ χρειαζόταν να σταματήσουμε. Το προηγούμενο μάθημα παρουσίασε τα compute types σαν μια επίπεδη λίστα, ενώ στην πραγματικότητα το Databricks έχει **πολλούς διαφορετικούς άξονες configuration**.

Η πιο σημαντική απάντηση πρώτα:

Serverless vs Classic και All-purpose vs Jobs ΔΕΝ είναι το ίδιο πράγμα. Είναι διαφορετικές διαστάσεις.

Αλλά δεν μπορούν όλοι οι συνδυασμοί να γίνουν αυθαίρετα. Θα το χτίσουμε από την αρχή.

1. Πρώτα: τι είναι γενικά ένα compute;

Έχεις κώδικα:

</> Python

```
df = spark.table("prod.sales.orders")

df.groupBy("country").sum("amount").display()
```

Ο κώδικας από μόνος του δεν κάνει computation.

Χρειαζόμαστε πραγματικά resources:

</> Plain text

```
CPU
RAM
network
possibly GPU
Spark driver
Spark executors
```

Το σύνολο αυτών των resources που χρησιμοποιεί το Databricks για να εκτελέσει το workload το λέμε γενικά:

compute

2. Το πρόβλημα με την ορολογία

Μπορεί να συναντήσεις όλα αυτά:

</> Plain text

```
Serverless
Classic

All-purpose
Jobs
Pipeline

Standard
Dedicated

Single-node
Multi-node

Fixed-size
Autoscaling

CPU
GPU

Photon
non-Photon

SQL Serverless
SQL Pro
SQL Classic
```

Αυτά **δεν είναι 12 ανταγωνιστικά compute types**.

Πολλά είναι διαφορετικά χαρακτηριστικά του ίδιου compute.

Ένα compute μπορεί, για παράδειγμα, να είναι:

⟨/⟩ Plain text

```
Classic
+
All-purpose
+
Standard access
+
Multi-node
+
Autoscaling
+
CPU
+
Photon
+
DBR 17.3 LTS
```

Αυτό είναι **ένα** compute configuration.

3. Οι βασικοί άξονες

Σκέψου το σαν configuration vector:

⟨/⟩ Plain text

```
Compute =
(
  workload purpose,
  infrastructure model,
  access mode,
  topology,
  scaling,
  hardware,
  runtime,
  execution engine
)
```

Για παράδειγμα:

⟨/⟩ Plain text

```
purpose      = interactive
infrastructure = classic
access       = standard
topology     = multi-node
scaling      = autoscaling
hardware     = CPU
```

```
runtime      = DBR 17.3
engine       = Photon
```

Τώρα θα πάρουμε κάθε άξονα ξεχωριστά.

4. Άξονας 1 — ΤΙ ΘΕΛΩ ΝΑ ΚΑΝΩ;

Αυτό είναι το **workload purpose**.

Οι βασικές κατηγορίες είναι:

</> Plain text

Interactive development
Automated job
Lakeflow pipeline
SQL / BI

Δηλαδή:

Workload	Τι σημαίνει
Interactive	Κάθομαι σε notebook και εκτελώ cells
Job	Κώδικας τρέχει αυτόματα/scheduled
Pipeline	Lakeflow declarative data pipeline
SQL	SQL analytics, dashboards, BI

Αυτός είναι ένας τελείως διαφορετικός άξονας από το serverless/classic.

5. Άξονας 2 — ΠΟΙΟΣ ΔΙΑΧΕΙΡΙΖΕΤΑΙ ΤΑ MACHINES;

Εδώ έχουμε:

</> Plain text

SERVERLESS
vs
CLASSIC

Serverless

Λες στο Databricks:

Τρέξε το workload.

Και το Databricks αποφασίζει/διαχειρίζεται σε μεγάλο βαθμό:

</> Plain text

```
machines
instance types
scaling
runtime server side
infrastructure lifecycle
```

Δεν δημιουργείς εσύ cluster με:

</> Plain text

```
1 driver
8 workers
m5.4xlarge
DBR X
min workers 2
max workers 10
```

Η υποδομή είναι Databricks-managed. [Databricks Docu...](#) +1

Classic

Εσύ/configuration policy καθορίζεις πολύ περισσότερα:

</> Plain text

```
VM type
driver
workers
runtime
autoscaling
access mode
Photon
spot instances
etc.
```

Τα classic all-purpose, jobs και pipeline compute είναι resources που δημιουργείς και παραμετροποιείς εσύ, και εκτελούνται στο cloud account/environment του πελάτη. [Databricks Docu...](#) +1

6. Και πού μπαίνει το All-purpose;

Εδώ ήταν το βασικό σου μπέρδεμα.

All-purpose δεν σημαίνει non-serverless.

Σημαίνει περίπου:

reusable interactive classic Spark compute.

Το χρησιμοποιείς όταν κάθεσαι σε notebook και πειραματίζεσαι.

Για παράδειγμα:

</> Plain text

```
09:00 Nikos opens notebook
      ↓
attaches notebook to cluster

09:10 run cell

09:30 another cell

10:20 debugging

11:00 another notebook attaches
```

To compute υπάρχει ως reusable resource.

Αυτό είναι:

</> Plain text

Classic All-purpose Compute

7. Υπάρχει Serverless All-purpose;

Εδώ η απάντηση είναι λίγο λεπτή.

Conceptually, ναι:

</> Plain text

```
interactive
+
serverless
```

υπάρχει.

Αλλά η Databricks δεν το ονομάζει:

</> Plain text

Serverless All-purpose Cluster

Το ονομάζει:

Serverless compute for notebooks

Άρα έχουμε:

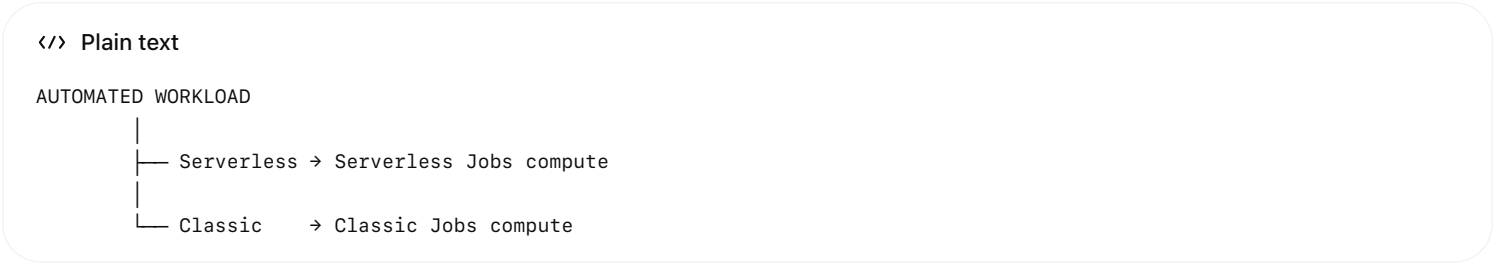
</> Plain text



Αυτό είναι το mental model που σου έλειπε.

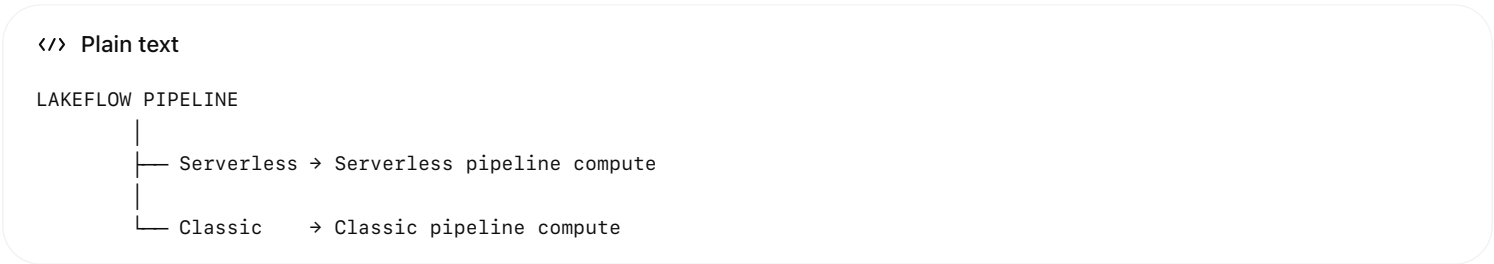
8. Και Jobs;

Το ίδιο.



9. Και Pipelines;

Πάλι:



Η Databricks σήμερα προτείνει serverless για τις περισσότερες νέες notebook, job και pipeline workloads. [Databricks Docu...](#) +1

10. Αυτή είναι η σημαντικότερη matrix

Workload x Infrastructure

Purpose	Serverless	Classic
Interactive notebook	✔ Serverless notebook compute	✔ All-purpose compute
Automated job	✔ Serverless Jobs compute	✔ Jobs compute
Lakeflow pipeline	✔ Serverless pipeline	✔ Classic pipeline
SQL / BI	✔ Serverless SQL warehouse	✔ Pro / Classic SQL warehouse

Αυτός είναι ο πρώτος χάρτης που θέλω να απομνημονεύσεις.

11. Άρα `all-purpose` και `serverless` είναι διαφορετικοί άξονες;

Ναι *conceptually*, αλλά όχι πλήρως ανεξάρτητες επιλογές στο UI.

Δεν έχεις dropdown:

</> Plain text

Purpose:

☒ `all-purpose`

Infrastructure:

☒ `serverless`

και μετά το Databricks δημιουργεί κάτι που λέγεται `"serverless all-purpose"` .

Αν επιλέξεις:

</> Plain text

`interactive + classic`

το προϊόν λέγεται:

</> Plain text

`classic all-purpose compute`

Αν επιλέξεις:

</> Plain text

`interactive + serverless`

το προϊόν λέγεται:

</> Plain text

`serverless notebook compute`

Άρα είναι καλύτερα να το δεις ως **constrained matrix**, όχι ως τελείως ελεύθερο Cartesian product.

12. Η πλήρης λίστα των βασικών `compute types`

Για Data Engineering / Analytics, σήμερα θέλω να ξέρεις αυτές τις κατηγορίες.

Spark/general compute

Plain text

1. Serverless notebook compute
2. Classic all-purpose compute
3. Serverless Jobs compute
4. Classic Jobs compute
5. Serverless Lakeflow pipeline compute
6. Classic Lakeflow pipeline compute

Η επίσημη compute documentation χωρίζει πράγματι serverless compute σε notebooks, jobs και pipelines, ενώ το classic compute περιλαμβάνει all-purpose, jobs και pipeline resources. [Databricks Docu...](#) +1

13. SQL compute είναι ξεχωριστή οικογένεια

Μετά υπάρχουν τα:

Plain text

SQL Warehouses

με:

Plain text

7. Serverless SQL Warehouse
8. Pro SQL Warehouse
9. Classic SQL Warehouse

και πλέον:

Plain text

10. Lakehouse Real-Time SQL Warehouse
[Beta]

για ιδιαίτερα low-latency/high-concurrency read workloads. [Databricks Docu...](#)

14. Άρα συνολικά

</> Plain text

```
DATABRICKS COMPUTE
|
├─ General Spark compute
|   |
|   ├─ Interactive
|   |   |
|   |   ├─ Serverless notebook
|   |   └─ Classic all-purpose
|   |
|   └─ Automated
|       |
|       ├─ Serverless Jobs
|       └─ Classic Jobs
|   └─ Lakeflow pipelines
|       |
|       ├─ Serverless pipeline
|       └─ Classic pipeline
└─ SQL Compute
    |
    ├─ Serverless SQL Warehouse
    ├─ Pro SQL Warehouse
    ├─ Classic SQL Warehouse
    └─ Lakehouse Real-Time
        [Beta]
```

Αυτό είναι πολύ πιο σωστό mental model από μία επίπεδη λίστα 10 compute types.

15. Τι είναι All-purpose συγκεκριμένα;

Ας δούμε πραγματική κατάσταση.

Δημιουργείς:

</> Plain text

```
my-dev-cluster
```

και λες:

</> Plain text

```
Runtime:
17.3 LTS
```

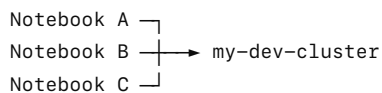
```
Workers:
2-8
```

```
Autoscaling:
yes
```

```
Access:
Standard
```

Μετά:

</> Plain text



Το cluster υπάρχει ανεξάρτητα από συγκεκριμένο job.

Μπορείς να μπεις, να γράψεις:

</> Python

```
df = spark.read.table(...)
```

να σταματήσεις 10 λεπτά, να γράψεις άλλο command, να κάνεις debugging κτλ.

Αυτό είναι η έννοια **all-purpose**:

Ένα reusable classic compute resource που δεν δημιουργήθηκε μόνο για μία συγκεκριμένη automated job execution.

16. Τι είναι Jobs Compute;

Έστω:

</> Plain text

02:00

Job:
Bronze → Silver

Δεν χρειάζεται cluster να κάθεται ανοιχτό στις:

</> Plain text

08:00

12:00

18:00

Θέλεις:

</> Plain text

01:59

nothing

02:00

```
JOB START
↓
compute starts
↓
run pipeline
↓
job ends
↓
compute goes away
```

Αυτό είναι το βασικό lifecycle model του jobs compute.

Στο classic case:

</> Plain text

Classic Jobs Compute

εσύ καθορίζεις το cluster configuration.

Στο serverless:

</> Plain text

Serverless Jobs

η Databricks το διαχειρίζεται. [Databricks Docu...](#) +1

17. Μπορεί Job να τρέξει σε All-purpose;

Ναι, τεχνικά.

Η Databricks επιτρέπει σε αρκετά Job task types να χρησιμοποιήσουν classic all-purpose compute.

Αλλά **δεν το συνιστά για production jobs**. Προτιμά serverless jobs ή classic jobs compute. [Databricks Docu...](#) +1

Γιατί;

Φαντάσου:

</> Plain text

All-purpose:

```
Developer A running query
Developer B debugging
Job C running production ETL
```

Τώρα όλοι ανταγωνίζονται για:

</> Plain text

```
CPU
RAM
executors
shuffle bandwidth
```

και ένας developer μπορεί να καταστρέψει την performance predictability του production job.

18. Άρα All-purpose ≠ "μπορεί να κάνει όλα τα workloads"

Το όνομα είναι λίγο παραπλανητικό.

All-purpose δεν σημαίνει:

Είναι η καλύτερη επιλογή για τα πάντα.

Σημαίνει περισσότερο:

Generic reusable interactive compute.

19. Τώρα μπαίνει ο επόμενος άξονας: Access Mode

Αυτό αφορά κυρίως classic all-purpose/jobs compute.

Έχεις:

</> Plain text

```
STANDARD
vs
DEDICATED
``` :chatgpt-content-reference{index="7"}
```

---

# 20. Standard access mode

Μπορούν πολλοί users να χρησιμοποιούν το ίδιο compute, με workload isolation.

```
```text
Classic cluster
```

```
|
| Nikos workload
| Maria workload
| John workload
|
```

Η Databricks προτείνει Standard για τις περισσότερες classic workloads. [Databricks Docu...](#)

21. Dedicated access mode

To compute ανατίθεται σε:

</> Plain text

```
one user
```

ή συγκεκριμένο:

</> Plain text

```
group
```

και δεν λειτουργεί ως general shared compute.

Χρησιμοποιείται όταν έχεις workloads/features που χρειάζονται πιο direct/dedicated environment, π.χ. ορισμένα ML/GPU/R scenarios. [Databricks Docu...](#) +1

22. Άρα τώρα ένα compute μπορεί να είναι

</> Plain text

```
Classic
+
All-purpose
+
Standard
```

ή:

</> Plain text

```
Classic
+
All-purpose
+
Dedicated
```

ή:

</> Plain text

```
Classic
+
Jobs
+
Standard
```

ή:

</> Plain text

Classic
+
Jobs
+
Dedicated

Αυτό είναι πραγματικός δεύτερος άξονας.

23. Matrix: Classic compute

Lifecycle	Standard	Dedicated
All-purpose	✓	✓
Jobs	✓	✓
Pipeline	έχει δικό του pipeline configuration model	έχει δικό του pipeline configuration model

Standard/Dedicated είναι ρητά access-mode επιλογές για all-purpose και jobs compute. [Databricks Docu...](#)

24. Serverless έχει Standard/Dedicated dropdown;

Όχι με τον ίδιο τρόπο.

Στο serverless δεν δημιουργείς cluster και δεν λες:

</> Plain text

Access mode = Dedicated
Workers = 8
Driver = ...

Αυτά είναι implementation choices που διαχειρίζεται η πλατφόρμα.

Για παράδειγμα, serverless Jobs χρησιμοποιεί Standard access-mode semantics και απαιτεί workload compatible με Standard compute. [Databricks Docu...](#)

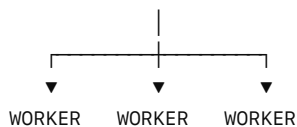
25. Επόμενος άξονας: Single-node vs Multi-node

Αυτό αφορά classic compute configuration.

Multi-node

</> Plain text

DRIVER



Εδώ έχουμε distributed Spark.

Single-node

</> Plain text

```
DRIVER + executor
same machine
```

Δεν υπάρχουν separate worker VMs.

Χρησιμοποιείται για:

</> Plain text

```
small data
non-distributed libraries
certain ML workloads
development
```

και όχι large distributed Spark processing. [Databricks Docu...](#)

26. Επόμενος άξονας: fixed vs autoscaling

Classic cluster:

Fixed

</> Plain text

```
workers = 4
```

Όλη την ώρα:

</> Plain text

```
4 workers
```

Autoscaling

⟨/⟩ Plain text

```
min workers = 2
max workers = 10
```

Μπορεί να γίνει:

⟨/⟩ Plain text

```
low load:
2 workers
```

```
medium:
5 workers
```

```
heavy:
10 workers
```

```
later:
3 workers
```

Η Databricks προσθέτει/αφαιρεί workers μέσα στα όρια που έθεσες. [Databricks Docu...](#)

27. Μην το μπερδεύεις με serverless

Classic autoscaling:

⟨/⟩ Plain text

```
YOU:
min = 2
max = 10
worker type = X
```

```
Databricks:
chooses number between 2 and 10
```

Serverless:

⟨/⟩ Plain text

```
YOU:
run my workload
```

```
Databricks:
handles underlying resource selection/scaling
```

Άρα:

Autoscaling ≠ serverless.

Classic compute μπορεί κάλλιστα να autoscale.

28. Επόμενος άξονας: CPU vs GPU

Classic compute μπορεί να χρησιμοποιεί:

</> Plain text

CPU instances

ή:

</> Plain text

GPU instances

για workloads όπως deep learning. GPU classic compute χρησιμοποιεί ML runtime και Photon δεν υποστηρίζεται στα GPU instance types. [Databricks Docu...](#)

Παράδειγμα:

</> Plain text

```
Classic
+
All-purpose
+
Dedicated
+
GPU
+
DBR ML
```

είναι απολύτως λογικό configuration.

29. Και πλέον υπάρχει Serverless GPU

Από το 2026 υπάρχει και **AI Runtime** σε Public Preview, που φέρνει serverless GPU infrastructure για deep-learning workloads.

[Databricks Docu...](#)

Άρα:

</> Plain text

GPU

και:

</> Plain text

serverless

δεν είναι από μόνα τους αντίθετα concepts.

Πάλι διαφορετικοί άξονες.

30. Επόμενος άξονας: Databricks Runtime

Στο classic compute επιλέγεις κάτι σαν:

⟨/⟩ Plain text

```
DBR 17.3 LTS
DBR ...
DBR ML ...
```

Αυτό είναι:

⟨/⟩ Plain text

```
software stack
```

όχι hardware.

Άρα ένα classic configuration μπορεί να είναι:

⟨/⟩ Plain text

```
AWS VM:
m5.xlarge

software:
DBR 17.3 LTS
```

Η runtime περιλαμβάνει Spark και Databricks software/libraries. [Databricks Docu...](#)

31. Serverless Runtime

Στο serverless δεν επιλέγεις traditional DBR version όπως:

⟨/⟩ Plain text

```
DBR 17.3
```

Η Databricks αναβαθμίζει το server-side runtime και χρησιμοποιεί serverless environment versions για application compatibility.

[Databricks Docu...](#) +1

Άρα:

⟨/⟩ Plain text

```
classic:
  "I choose the DBR"

serverless:
  "Databricks manages the server runtime"
```

32. Επόμενος άξονας: Photon

Classic CPU compute:

</> Plain text

```
Photon ON
ή
Photon OFF
```

Μπορείς να το επιλέξεις.

Serverless compute:

</> Plain text

```
Photon enabled
```

και η Databricks το διαχειρίζεται. SQL warehouses επίσης χρησιμοποιούν Photon. [Databricks Docu...](#)

Άρα Photon είναι πάλι **άλλος άξονας**, όχι compute family.

33. Έτσι μοιάζει η μεγάλη classic matrix

Axis	Επιλογές
Purpose	All-purpose / Jobs
Infrastructure	Classic
Access	Standard / Dedicated
Nodes	Single / Multi
Scale	Fixed / Autoscaling
Hardware	CPU / GPU
Worker type	memory / compute / general purpose etc.
Driver type	same ή διαφορετικό
Runtime	DBR version / DBR ML
Photon	On / Off, όπου υποστηρίζεται
Instances	On-demand / Spot workers
Pool	Pool / direct provisioning
Termination	manual / auto-terminate

Τα περισσότερα από αυτά εμφανίζονται ως configuration settings για classic all-purpose/jobs compute. [Databricks Docu...](#)

34. Παράδειγμα πραγματικού configuration

Θέλω interactive ETL development.

Μπορεί να κάνω:

</> Plain text

Purpose:

All-purpose

Infrastructure:

Classic

Access mode:

Standard

Runtime:

DBR 17.3 LTS

Photon:

ON

```
Topology:
Multi-node

Worker:
general purpose

Scaling:
2 → 8 workers

Driver:
```

Αυτό δεν είναι 10 computes.

Είναι:

ένα classic all-purpose compute με 9 configuration choices.

35. Ένα δεύτερο configuration

Θέλω production scheduled ETL:

</> Plain text

```
Purpose:
Jobs

Infrastructure:
Classic

Access:
Standard

Runtime:
DBR 17.3 LTS

Photon:
ON

Topology:
Multi-node

Autoscaling:
4 → 20 workers
```

Άρα:

</> Plain text

```
Classic Jobs Compute
```

με συγκεκριμένες παραμέτρους.

36. Serverless equivalent

Θέλω το ίδιο ETL αλλά δεν χρειάζομαι custom infrastructure.

</> Plain text

Purpose:
Jobs

Infrastructure:
Serverless

Τώρα ΔΕΝ επιλέγω:

</> Plain text

driver VM
worker VM
2–20 workers
DBR 17.3
spot
pool

Η Databricks διαχειρίζεται αυτά. Autoscaling και Photon ενεργοποιούνται αυτόματα στα serverless Jobs. [Databricks Docu...](#)

37. Αλλά serverless έχει μια άλλη παράμετρο

Για Jobs/Pipelines υπάρχει:

</> Plain text

Performance Optimized
vs
Standard performance mode

Performance Optimized

</> Plain text

faster startup
more latency sensitive

Standard

</> Plain text

slower startup
lower cost
batch-friendly

Standard mode είναι διαθέσιμο για serverless Jobs και Lakeflow pipelines, αλλά **όχι για notebooks**. Τα notebooks

χρησιμοποιούν performance-oriented interactive mode.  Databricks Docu... +1

38. Άρα serverless matrix

Serverless workload	Performance optimized	Standard
Notebook		
Job		
Pipeline		

Πρόσεξε:

Standard performance mode **δεν είναι το ίδιο** με Standard access mode .

Δύο εντελώς διαφορετικά πράγματα.

Αυτό είναι κλασική πηγή σύγχυσης.

39. Standard access vs Standard performance

Standard access mode

Security/isolation concept:

</> Plain text

Who may use this classic compute?
How are users isolated?

Standard performance mode

Serverless cost/latency concept:

</> Plain text

Do I prioritize lower cost
or faster startup?

Καμία σχέση μεταξύ τους πέρα από το ίδιο αγγλικό όνομα.

40. Και SQL Warehouse πώς μπαίνει;

SQL Warehouses είναι λίγο διαφορετικό branch.

</> Plain text



Όλα τα σημερινά warehouse types υποστηρίζουν Photon.

Αλλά:

</> Plain text

Serverless

έχει επίσης Predictive IO + Intelligent Workload Management.

</> Plain text

Pro

έχει Photon + Predictive IO.

</> Plain text

Classic

έχει Photon αλλά όχι τα άλλα δύο. [Databricks Docu...](#)

41. SQL warehouse matrix

Feature	Serverless	Pro	Classic
Photon	✓	✓	✓
Predictive IO	✓	✓	✗
Intelligent Workload Management	✓	✗	✗
Databricks-managed serverless infra	✓	✗	✗
Custom networking flexibility	περιορισμένη	διαφορετικά	✓
Recommended default	✓	ειδικές περιπτώσεις	fallback/basic

42. Μπορώ notebook να το συνδέσω σε SQL Warehouse;

Ναι, αλλά υπάρχει περιορισμός.

Notebook σε SQL warehouse μπορεί να τρέχει:

</> Plain text

SQL

Markdown

όχι Python/R cells.  Databricks Docu...

Άρα:

</> SQL

```
SELECT COUNT(*)  
FROM prod.sales.orders;
```



αλλά:

</> Python

```
from pyspark.sql import functions as F
```



σε notebook attached αποκλειστικά σε SQL warehouse.

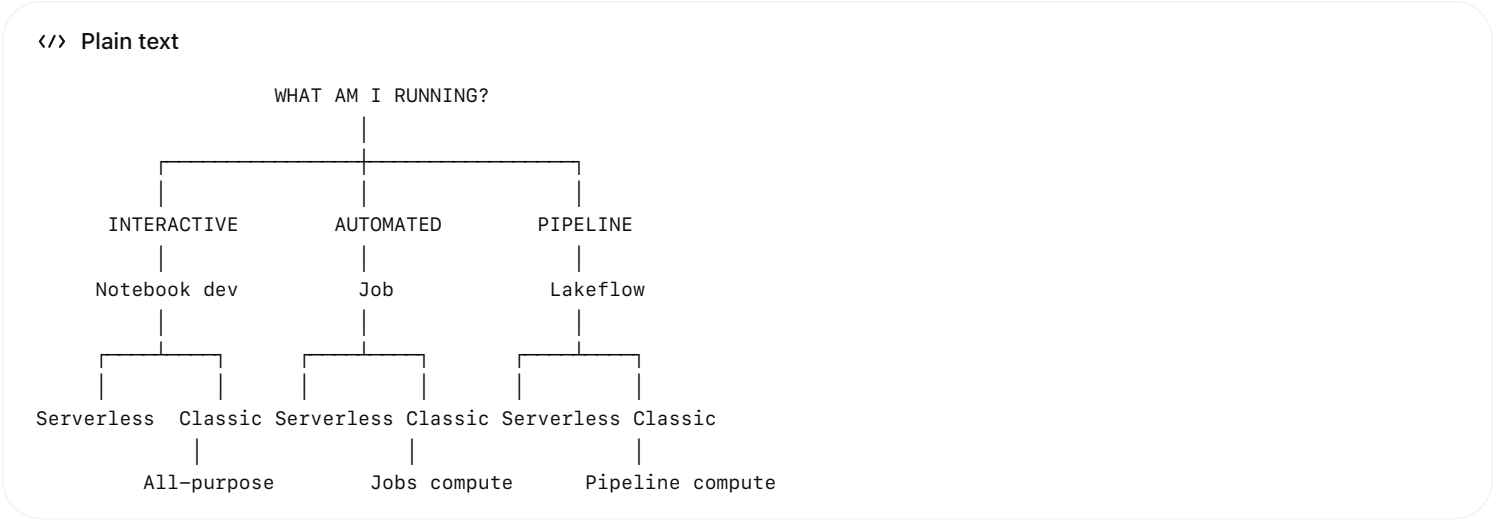
43. Πλήρης matrix χρήσης

Θέλω να κάνω...	Καλύτερο default	Άλλες δυνατότητες
Interactive Python/PySpark notebook	Serverless notebook	Classic all-purpose
Interactive SQL notebook	Serverless compute ή serverless SQL warehouse	Pro/classic warehouse, all-purpose
Scheduled Python notebook	Serverless Jobs	Classic Jobs
Python script job	Serverless Jobs	Classic Jobs
Python wheel	Serverless Jobs	Classic Jobs
JAR	Classic Jobs σε ορισμένα cases / serverless support ανά task	Classic
spark-submit	Classic Jobs	—
SQL Job task	Serverless SQL Warehouse	Pro SQL Warehouse
dbt	Serverless SQL Warehouse	Pro
Lakeflow pipeline	Serverless pipeline	Classic pipeline
BI/dashboard	Serverless SQL Warehouse	Pro / Classic warehouse

Η σημερινή task-support matrix της Databricks κάνει ακριβώς αυτές τις διακρίσεις. [Databricks Docu...](#)

44. Τώρα το σημαντικό conceptual tree

Θέλω να το αποθηκεύσεις νοητικά έτσι:



Και ξεχωριστά:

Plain text

SQL / BI

SQL Warehouse

Serverless

Pro

Classic

45. Και META επιλέγεις parameters

Μόνο αφού αποφασίσεις:

Plain text

Classic all-purpose

ή:

Plain text

Classic Jobs

μπαίνεις σε λεπτομέρειες:

Plain text

Standard / Dedicated

Single / Multi

DBR version

CPU / GPU

Worker type

Driver type

Fixed / Autoscaling

Photon

Spot

Pools

Auto termination

Αυτό είναι το σημείο που σου έλειπε.

46. Για serverless, μεγάλο μέρος της matrix εξαφανίζεται

Αυτό είναι ακριβώς το point του serverless.

Classic

Εσύ:

⌕ Plain text

```
Which VMs?
How many?
Which driver?
Which workers?
Which runtime?
Autoscale from where to where?
Photon?
Spot?
Pool?
```

Serverless

Εσύ:

⌕ Plain text

```
Here is my workload.
Run it.
```

και έχεις πολύ λιγότερες infrastructure choices. Η Databricks αναφέρει μάλιστα ότι serverless δεν υποστηρίζει compute policies, instance pools, compute-scoped init scripts, compute-scoped libraries ή τις περισσότερες Spark cluster configurations, επειδή αυτά τα infrastructure-level concerns είναι platform-managed. [Databricks Docu...](#)

47. Πρακτικό παράδειγμα

Έστω ότι αύριο στη δουλειά σου λένε:

Nikos, κάνε exploration στα 500 GB customer transactions.

Εσύ σκέφτεσαι:

Step 1 — interactive ή automated;

Εγώ θα κάθομαι και θα δοκιμάζω commands.

⌕ Plain text

```
interactive
```

Step 2 — χρειαζομαι custom infra;

Όχι.

</> Plain text

```
serverless
```

Άρα:

</> Plain text

```
Serverless notebook compute
```

Τέλος.

Δεν χρειάζεται να αποφασίσεις worker VMs.

48. Δεύτερο παράδειγμα

Κάθε βράδυ στις 2 π.μ. τρέξε ETL.

</> Plain text

```
automated
+
no special infrastructure requirement
```

Άρα:

</> Plain text

```
Serverless Jobs
```

49. Τρίτο παράδειγμα

Κάθε βράδυ ETL, αλλά χρησιμοποιούμε custom Spark JAR και low-level Spark functionality που δεν υποστηρίζει serverless.

Άρα:

</> Plain text

```
automated
+
classic required
```

→

</> Plain text

```
Classic Jobs Compute
```


και τότε διαλέγεις:

</> Plain text

DBR
workers
autoscaling
access mode
Photon
...

50. Τέταρτο παράδειγμα

100 analysts χρησιμοποιούν Power BI για SQL queries.

Δεν ξεκινάς με:

</> Plain text

all-purpose cluster

Σκέφτεσαι:

</> Plain text

SQL/BI workload

→

</> Plain text

SQL Warehouse

→ κατά κανόνα:

</> Plain text

Serverless SQL Warehouse

Η Databricks το προτείνει σήμερα για BI, ETL SQL και exploratory analytics. [Databricks Docu...](#)

51. Πέμπτο παράδειγμα

Θέλω R notebook.

Serverless notebook σήμερα δεν υποστηρίζει R ή Scala, ούτε RDD APIs. [Databricks Docu...](#)

Άρα:

</> Plain text

Interactive

+

requires unsupported serverless feature

→

</> Plain text

Classic all-purpose

πιθανόν:

</> Plain text

Dedicated

ανάλογα με το workload.

52. Ένα μικρό αλλά σημαντικό terminology trap

Μπορεί να ακούσεις κάποιον να λέει:

"standard compute"

Αυτό δεν σημαίνει:

</> Plain text

normal/basic compute

Στη σημερινή Databricks terminology:

</> Plain text

Standard compute

=

classic compute configured with

Standard access mode

``` :chatgpt-content-reference{index="29"}

Αντίστοιχα:

```text

Dedicated compute

=

classic compute configured with

Dedicated access mode

``` :chatgpt-content-reference{index="30"}

```

53. Η matrix που θέλω πραγματικά να κρατήσεις

Layer 1 – workload

Axis	Options
53. Η matrix που θέλω πραγματικά να κρατήσεις	53. Η matrix που θέλω πραγματικά να κρατήσεις
```



Γιατί αν διάλεξες serverless, πολλά από αυτά πλέον τα διαχειρίζεται Databricks.

Ούτε:

</> Plain text

GPU  
+  
Photon

σε classic GPU compute.



Photon δεν υποστηρίζεται εκεί. [Databricks Docu...](#)

Άρα έχουμε **constraints** μεταξύ axes.

## 55. "Όλες οι πιθανές combinations" σε μία χρήσιμη matrix

| Compute family      | Serverless / Classic | Interactive / Automated | Access configurable? | Nodes configurable? | DBR configurable?      | Autoscaling configurable? |
|---------------------|----------------------|-------------------------|----------------------|---------------------|------------------------|---------------------------|
| Notebook Serverless | Serverless           | Interactive             | ✗                    | ✗                   | ✗ traditional DBR      | ✗ infra managed           |
| All-purpose         | Classic              | Interactive             | ✓                    | ✓                   | ✓                      | ✓                         |
| Serverless Jobs     | Serverless           | Automated               | ✗                    | ✗                   | ✗ traditional DBR      | ✗ infra managed           |
| Classic Jobs        | Classic              | Automated               | ✓                    | ✓                   | ✓                      | ✓                         |
| Serverless Pipeline | Serverless           | Automated pipeline      | managed              | ✗                   | managed                | managed                   |
| Classic Pipeline    | Classic              | Automated pipeline      | pipeline-specific    | ✓                   | ✓                      | ✓                         |
| Serverless SQL WH   | Serverless           | SQL                     | SQL security model   | managed             | managed                | managed                   |
| Pro SQL WH          | non-serverless       | SQL                     | warehouse model      | abstracted          | Databricks SQL runtime | warehouse autoscale       |
| Classic SQL WH      | non-serverless       | SQL                     | warehouse model      | abstracted          | Databricks SQL runtime | warehouse autoscale       |

Αυτό είναι πολύ πιο κοντά στην πραγματικότητα από το:

</> Plain text

"serverless, all-purpose, jobs"

σαν να ήταν τρεις ίδιου επιπέδου επιλογές. [Databricks Docu...](#) +2

## 56. Υπάρχουν και άλλα serverless προϊόντα;

Ναι, αλλά δεν θέλω να τα βάλεις τώρα στον ίδιο κάδο.

Η Databricks έχει και άλλα features που χρησιμοποιούν serverless infrastructure, π.χ. model serving, data-quality monitoring και predictive optimization, αλλά διαχειρίζονται από ξεχωριστά product surfaces. [Databricks Docu...](#)

Επίσης υπάρχει πλέον:

</> Plain text

AI Runtime

για serverless GPU/deep-learning workloads σε Preview. [Databricks Docu...](#)

Για την πορεία σου ως Data Engineer, τα 9–10 compute types που ανέφερα παραπάνω είναι αυτά που πρέπει πρώτα να masterάρεις.

## 57. Μικρό τεστ — κρίσιμο

1. Είναι serverless αντίθετο του all-purpose ;

Όχι.

Τα concepts βρίσκονται σε διαφορετικά επίπεδα.

Το αντίθετο infrastructure model του serverless είναι περίπου:

</> Plain text

classic

ενώ το all-purpose περιγράφει:

</> Plain text

interactive/reusable classic workload lifecycle

2. Ποιο είναι το classic equivalent του serverless notebook compute;

</> Plain text

Classic all-purpose compute

3. Ποιο είναι το classic equivalent του serverless Jobs compute;

</> Plain text

Classic Jobs compute

4. Μπορεί classic compute να autoscale;

Ναι.

Άρα:

</> Plain text

autoscaling ≠ serverless

## 5. Μπορεί all-purpose να είναι Standard ή Dedicated;

Ναι.

</> Plain text

```
Classic all-purpose
 |
 ├── Standard
 └── Dedicated
```

## 6. Μπορώ να κάνω:

</> Plain text

```
Classic
+ Jobs
+ Standard
+ Multi-node
+ Autoscaling
+ Photon
```

Ναι.

Αυτό είναι απολύτως λογικό configuration.

## 7. Μπορώ να κάνω:

</> Plain text

```
Serverless Jobs
+ worker type m5.4xlarge
+ 2-20 workers
```

Όχι.

Αυτή είναι infrastructure configuration που serverless διαχειρίζεται για εσένα.

## 8. Standard Access Mode και Standard Performance Mode είναι το ίδιο;

Καμία σχέση.

</> Plain text

```
Standard Access
→ security/isolation
```

```
Standard Performance
→ serverless cost/startup tradeoff
```

## 9. Για interactive PySpark χωρίς ειδικές απαιτήσεις;

```
</> Plain text

Serverless notebook compute
```

είναι η σημερινή γενική recommendation. [Databricks Docu...](#)

## 10. Για scheduled PySpark production ETL;

```
</> Plain text

Serverless Jobs
```

αν το workload υποστηρίζεται.

Αν χρειάζεσαι custom cluster settings:

```
</> Plain text

Classic Jobs
```:chatgpt-content-reference{index="36"}

---

### 11. Για Power BI;

```text
SQL Warehouse
```

συνήθως:

```
</> Plain text

Serverless SQL Warehouse
```

## 12. Για development με R;

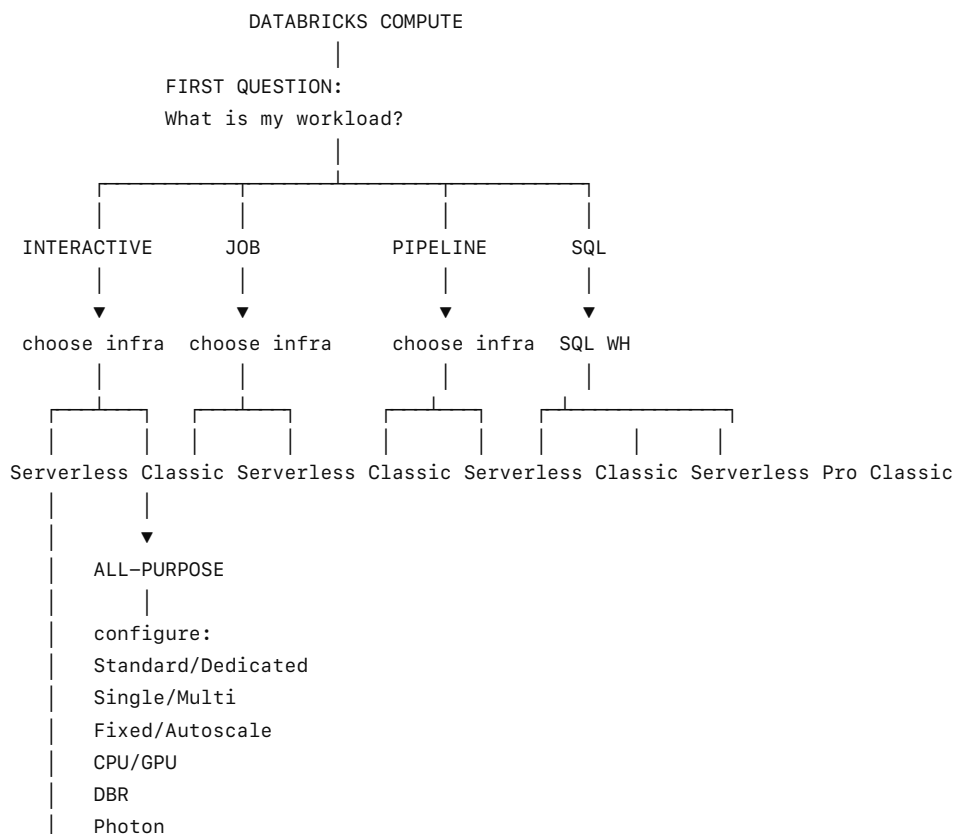
```
</> Plain text

Classic all-purpose
```

γιατί serverless notebooks σήμερα δεν υποστηρίζουν R. [Databricks Docu...](#)

## Το ένα διάγραμμα που πρέπει να θυμάσαι

</> Plain text



Αν αυτό το διάγραμμα γίνει καθαρό, μετά η σελίδα **Compute → Create compute** του Databricks θα πάψει να μοιάζει με δεκάδες άσχετα checkboxes.

Στο επόμενο μάθημα για compute θα άξιζε να μην προχωρήσουμε ακόμα στο **Unity Catalog**. Να πάρουμε συγκεκριμένα ένα **Classic All-Purpose cluster** και να το «συναρμολογήσουμε» από το μηδέν: driver, worker, executor, cores, RAM, node types, min/max workers, autoscaling, Standard/Dedicated, DBR, Photon, Spot, pools και auto-termination — και μετά να δούμε πώς κάθε επιλογή επηρεάζει πραγματικά ένα Spark job και τι φαίνεται στο Spark UI.

What's dbr and what options i have for runtime? Also what is ohoton, other options and explain in details. Also what happens with sql warehouses access management? Explain the pipeline access management. Isn't all-purpose restricted to interactive mode? Is it only a strong suggestion? Can you configure the autoscaling axis with serverless or this option makes sense only in classic? Explain in depth what is the "runtime" . The term in databrick's context. Explain also pool, termination, driver type, worker type.

Answer all the above in the same extend. Do not shorten your answer for the last questions. Answer all in depth and extensively, because i am a newbie

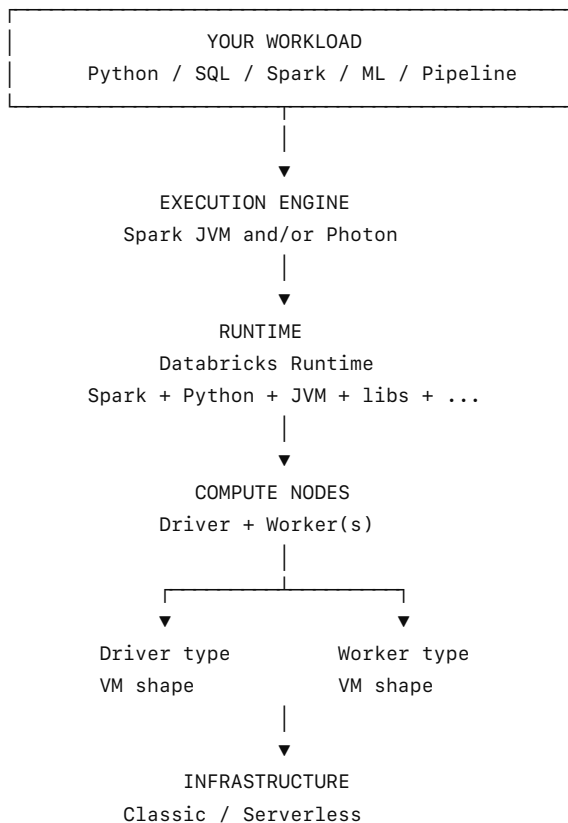
Worked for 1m 24s



Ναι. Εδώ χρειάζεται να καθαρίσουμε αρκετές έννοιες, γιατί το Databricks UI βάζει δίπλα-δίπλα επιλογές που στην πραγματικότητα ανήκουν σε τελείως διαφορετικά layers.

Το βασικό μοντέλο που θέλω να έχεις πριν μπούμε στις λεπτομέρειες είναι αυτό:

</> Plain text



Άρα DBR, Photon, driver type, worker type, pool, autoscaling, all-purpose και serverless δεν είναι εναλλακτικές επιλογές στο ίδιο επίπεδο.

## 1. Τι σημαίνει γενικά «runtime»;

Πρώτα χωρίς Databricks.

Έχεις αυτό:

</> Python

```
from pyspark.sql import functions as F

df = spark.read.table("sales.orders")
df.groupBy("country").sum("amount").show()
```

Ο source code μόνος του είναι απλώς κείμενο.

Για να εκτελεστεί χρειάζεσαι ένα περιβάλλον που περιέχει πράγματα όπως:

**</> Plain text**

```
Python interpreter
JVM
Apache Spark
PySpark
Java/Scala libraries
native libraries
system libraries
configuration
dependency versions
```

Αυτό το **περιβάλλον μέσα στο οποίο εκτελείται το πρόγραμμά σου** ονομάζεται runtime environment.

Ένα απλό Python παράδειγμα:

**</> Plain text**

```
my_program.py
|
▼
Python 3.12 runtime
|
├── Python interpreter
├── standard library
└── installed packages
```

Αν το ίδιο πρόγραμμα πάει σε άλλο environment:

**</> Plain text**

```
Python 3.8
numpy 1.20
```

αντί:

**</> Plain text**

```
Python 3.12
numpy 2.x
```

μπορεί να συμπεριφερθεί διαφορετικά ή να αποτύχει.

## 2. Databricks Runtime — DBR

Στο Databricks το βασικό packaged runtime ονομάζεται:

**Databricks Runtime (DBR)**

Η Databricks το ορίζει ως το σύνολο των core components που τρέχουν πάνω στο compute. Όλες οι DBR εκδόσεις περιλαμβάνουν Apache Spark. [Databricks Docu...](#)

Σκέψου ένα classic worker VM.

Χωρίς software:

</> Plain text

```
VM
|
├── CPU
├── RAM
├── disk
└── network
```

Αυτό από μόνο του δεν ξέρει να κάνει:

</> Python

```
spark.read.table(...)
```

Πάνω του φορτώνεται το runtime:

</> Plain text

```
VM
|
└── Databricks Runtime
 ├── Apache Spark
 ├── JVM
 ├── Python
 ├── PySpark
 ├── Scala
 ├── Delta Lake integrations
 ├── Databricks optimizations
 ├── built-in libraries
 └── Databricks services/integrations
```

και μετά:

</> Plain text

```
your code
 ↓
DBR
 ↓
VM hardware
```

### 3. Άρα runtime ≠ machine

Αυτό είναι πολύ σημαντικό.

Αν επιλέξεις:

```
</> Plain text
```

```
Worker type:
m5.4xlarge
```

επιλέγεις περίπου:

Τι είδους machine θέλω;

Αν επιλέξεις:

```
</> Plain text
```

```
Databricks Runtime:
18 LTS
```

επιλέγεις:

Τι software environment θέλω να εγκατασταθεί πάνω σε αυτά τα machines;

Άρα:

```
</> Plain text
```

```
NODE TYPE
=
hardware/infrastructure configuration
```

ενώ:

```
</> Plain text
```

```
RUNTIME
=
software environment
```

### 4. Πολύ καλή αναλογία

Φαντάσου laptop.

To laptop:

```
</> Plain text
```

```
Ryzen 9
```

32 GB RAM  
1 TB SSD

είναι αντίστοιχο με το:

</> Plain text

worker node type

Το:

</> Plain text

Ubuntu 26  
Python  
Java  
Spark  
libraries

είναι περισσότερο αντίστοιχο με:

</> Plain text

runtime

Δεν είναι ακριβώς λειτουργικό σύστημα, αλλά αυτή η αναλογία είναι καλή για αρχή.

## 5. Γιατί υπάρχουν DBR versions;

Επειδή όλα αυτά εξελίσσονται:

</> Plain text

Spark  
Python  
Scala  
Java  
Delta support  
Databricks APIs  
libraries  
security fixes  
performance optimizations

Άρα:

</> Plain text

DBR 16

και:

</> Plain text

DBR 19

δεν είναι το ίδιο software stack.

Μπορεί να έχουν διαφορετικές:

</> Plain text

Spark versions  
APIs  
bug fixes  
library versions  
optimizer behavior  
Python versions  
features

Για παράδειγμα, αυτή τη στιγμή η Databricks αναφέρει:

DBR	Spark
19	Spark 4.2.0
18 LTS	Spark 4.1.0
17.3 LTS	Spark 4.0.0
16.4 LTS	Spark 3.5.2
15.4 LTS	Spark 3.5.0
14.3 LTS	Spark 3.5.0

Όλες αυτές παραμένουν υποστηριζόμενες στις τρέχουσες release notes· η 19 και όλες οι παραπάνω LTS έχουν επίσης ML variants. [🌐 Databricks Docu...](#)

## 6. Τι σημαίνει DBR 18 LTS;

LTS:

Long-Term Support

Έστω production ETL:

</> Plain text

PostgreSQL  
↓  
Databricks  
↓  
bronze

```
↓
silver
↓
gold
↓
financial reporting
```

Δεν θέλεις κάθε λίγες εβδομάδες:

</> Plain text

```
new runtime
↓
new Spark
↓
new library
↓
something subtly changes
```

Για production μπορείς να πεις:

</> Plain text

Θα χρησιμοποιήσω DBR 18 LTS

και να έχεις μεγαλύτερο support/stability horizon.

Σήμερα η DBR 18 LTS υποστηρίζεται μέχρι τον Ιούνιο του 2029. [Databricks Docu...](#)

Η ίδια η Databricks προτείνει γενικά:

</> Plain text

```
All-purpose development
→ newest/current runtime
```

```
Operational Jobs
→ consider LTS
```

```
Machine Learning
→ consider DBR ML
```:chatgpt-content-reference{index="3"}
```

7. Standard DBR vs DBR ML

Υπάρχουν ουσιαστικά δύο βασικές variants που πρέπει τώρα να γνωρίζεις.

Normal Databricks Runtime

Για:

```
```text
Spark
```

```
SQL
ETL
data engineering
```

Παράδειγμα:

</> Plain text

```
Databricks Runtime 18 LTS
```

## Databricks Runtime for Machine Learning

Παράδειγμα:

</> Plain text

```
Databricks Runtime 18 LTS ML
```

Είναι βασισμένο στο κανονικό DBR αλλά περιλαμβάνει επιπλέον preconfigured ML/DL stack.

Για παράδειγμα το σημερινό DBR 18 LTS ML περιλαμβάνει libraries/tooling όπως PyTorch, transformers, XGBoost, MLflow και GPU-related components. [Databricks Docu...](#) +1

Conceptually:

</> Plain text

```
DBR
+
ML/DL libraries
+
ML integrations
+
GPU software where appropriate
=
DBR ML
```

Δεν χρησιμοποιείς DBR ML απλώς επειδή «είναι καλύτερο».

Για ordinary ETL:

</> Plain text

```
normal DBR
```

είναι συνήθως πιο κατάλληλο.

## 8. Τι runtime επιλέγω;

Για έναν αρχάριο, ο πρώτος decision table είναι:



Use case	Συνήθης επιλογή
Μαθαίνω / interactive development	Current DBR
Production Spark Job	LTS DBR
ML / PyTorch / XGBoost	DBR ML
GPU ML	DBR ML + GPU node
Legacy application	DBR version συμβατό με application
Serverless	Δεν επιλέγω traditional DBR

Το τελευταίο είναι εξαιρετικά σημαντικό.

## 9. Τι γίνεται με DBR στο serverless;

Στο classic:

</> Plain text

YOU

|

├─ DBR 18 LTS

├─ worker type

├─ driver type

└─ ...

Στο serverless:

</> Plain text

YOU

|

└─ Run workload

DATABRICKS

|

├─ runtime

├─ infrastructure

├─ scaling

└─ upgrades

Για serverless Jobs η Databricks αναφέρει ρητά ότι αναβαθμίζει αυτόματα το runtime και διαχειρίζεται runtime version, instance types, memory και processing engines. [Databricks Docu...](#)

Άρα το:

«ποιο DBR να επιλέξω για serverless job;»

είναι συνήθως λάθος ερώτηση.

Δεν ριπάρεις:

</> Plain text

DBR 18 LTS

όπως σε classic cluster.

## 10. Αλλά serverless δεν σημαίνει ότι δεν υπάρχει runtime

Αυτό είναι λεπτό.

Υπάρχει runtime.

Απλώς:

</> Plain text

runtime exists

δεν σημαίνει:

</> Plain text

user chooses exact traditional DBR

Στα serverless notebooks μάλιστα υπάρχει πλέον **Environment** configuration, όπου επιλέγεις base environment/environment version και dependencies. Αυτό όμως είναι διαφορετικό abstraction από το classic DBR Version selector. [Databricks Docu...](#)

Έχεις δηλαδή:

</> Plain text

```
SERVERLESS
|
├─ Databricks-managed underlying runtime
|
└─ user-configurable application environment
 ├── libraries
 ├── dependencies
 └─ environment version
```

## 11. Και στα Lakeflow Pipelines;

Τα pipelines είναι άλλη ειδική περίπτωση.

Το pipeline runtime είναι **managed by the pipeline platform**. Δεν διαλέγεις απλώς:

Plain text

DBR 18.0

όπως σε all-purpose cluster.

Τα Lakeflow pipelines χρησιμοποιούν runtime channels:

Plain text

current  
preview

και Databricks αναβαθμίζει το pipeline runtime. Σήμερα το `current` channel βασίζεται σε DBR 17.3 και το `preview` σε DBR 18, με rolling rollout ανά region. [Databricks Docu...](#)

Άρα:

Plain text

classic all-purpose/job  
→ choose DBR

serverless  
→ Databricks manages DBR/runtime

Lakeflow pipeline  
→ pipeline runtime channel

SQL warehouse  
→ Databricks SQL-managed runtime

Αυτό είναι πολύ σημαντικό mental map.

## 12. Τι είναι Photon;

Τώρα πάμε ένα layer διαφορετικό από DBR.

Το Photon είναι:

Databricks-native vectorized execution engine.

Δεν είναι runtime version.

Δεν είναι VM.

Δεν είναι Spark replacement ως programming model.

Δεν είναι database.

Δεν είναι file format. [Databricks Docu...](#)

## 13. Πού ακριβώς μπαίνει;

Έστω:

</> Python

```
(
 spark.table("orders")
 .filter("amount > 100")
 .groupBy("country")
 .sum("amount")
)
```

Αρχικά το Spark κάνει:

</> Plain text

```
DataFrame API
 ↓
logical plan
 ↓
Catalyst optimizer
 ↓
physical plan
```

Μέχρι εδώ το Photon δεν αντικαθιστά το Catalyst.

Μετά πρέπει να εκτελεστεί το physical plan.

## Χωρίς Photon

Conceptually:

</> Plain text

```
Catalyst physical plan
 ↓
Spark JVM execution
 ↓
CPU
```

## Με Photon

Για supported operators:

</> Plain text

```
Catalyst physical plan
 ↓
Photon native engine
 ↓
C++
```

↓  
vectorized batches  
↓  
CPU/SIMD

Η Databricks εξηγεί ότι το Catalyst εξακολουθεί να σχεδιάζει το query, αλλά το Photon αντικαθιστά το JVM-based execution layer για supported operations χρησιμοποιώντας native C++ και columnar batches. [Databricks Docu...](#) +1

## 14. Τι σημαίνει vectorized;

Φαντάσου ότι πρέπει να υπολογίσουμε:

Plain text

```
amount * 1.24
```

για:

Plain text

```
100
200
300
400
...
```

Ένα απλοποιημένο row-at-a-time model:

Plain text

```
load row 1
calculate

load row 2
calculate

load row 3
calculate
```

Vectorized:

Plain text

```
[100, 200, 300, 400, ...]
|
▼
process batch
```

Οι σύγχρονες CPUs μπορούν να αξιοποιήσουν SIMD instructions:

Single Instruction, Multiple Data

οπότε μπορούν να κάνουν operations σε πολλές τιμές αποτελεσματικότερα.

Photon επεξεργάζεται batches χιλιάδων rows και σχεδιάστηκε για αυτή την columnar/vectorized execution λογική.

⊕ Databricks Docu...

## 15. Πρέπει να αλλάξω Python/SQL;

Όχι.

Γράφεις:

⌕ Python

```
df.groupBy("country").sum("amount")
```

και όχι:

⌕ Python

```
photon.groupBy(...)
```

Το ίδιο DataFrame/SQL code μπορεί να εκτελεστεί:

⌕ Plain text

```
Spark API
 ↓
Catalyst
 ↓
Photon
```

χωρίς αλλαγή programming model.

## 16. Και αν Photon δεν υποστηρίζει κάποιο operator;

Αυτό είναι πολύ ωραίο architectural detail.

Έστω physical plan:

⌕ Plain text

```
Scan
 ↓
Filter
 ↓
SomeUnsupportedOperation
 ↓
Aggregate
```

Το Photon δεν λείπει:

Ολόκληρο query failed.

Μπορεί να γίνει fallback στην Spark/JVM execution για μη υποστηριζόμενη operation. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
Photon
 ↓
Photon
 ↓
unsupported
 ↓
Spark/JVM
```

Άρα Photon και regular Spark execution μπορούν να συμμετέχουν στο ίδιο workload.

## 17. Ποια είναι η «άλλη επιλογή» από Photon;

Για ordinary Spark SQL/DataFrame workloads:

</> Plain text

```
Photon ON
```

versus:

</> Plain text

```
Photon OFF
→ Spark/JVM execution
```

Δεν υπάρχει κάποια τρίτη γενική engine επιλογή:

</> Plain text

```
Photon
X-engine
Y-engine
```

To main comparison είναι:

</> Plain text

```
Spark traditional JVM execution
 vs
Photon native execution
```

## 18. Photon configuration ανά compute type

Σήμερα:

Compute	Photon
Serverless compute	Enabled
SQL Warehouse	Enabled
Serverless Lakeflow pipeline	Enabled
Classic all-purpose	On by default, configurable
Classic jobs	On by default, configurable
Classic pipeline	On by default/configurable in current UI/API semantics

Η current Photon documentation αναφέρει Photon enabled σε serverless compute, SQL warehouses και serverless pipelines, ενώ για classic all-purpose/jobs/pipelines μπορεί να ενεργοποιείται ή απενεργοποιείται με το Photon setting. [Databricks Docu...](#)

## 19. Photon και GPU είναι διαφορετικά πράγματα

Μη σκεφτείς:

</> Plain text

Photon = GPU acceleration

Λάθος.

Photon:

</> Plain text

CPU-native vectorized SQL/DataFrame engine

GPU:

</> Plain text

hardware accelerator

για workloads όπως:

</> Plain text

deep learning  
matrix operations  
model training



Μάλιστα classic GPU compute δεν υποστηρίζει Photon. [Databricks Docu...](#)

Άρα διαφορετικοί άξονες:

</> Plain text

Execution engine:  
Photon / JVM

Hardware:  
CPU / GPU

## 20. Runtime vs Photon — κρίσιμη διάκριση

Έστω:

</> Plain text

DBR 18 LTS  
+  
Photon ON

Αυτά μπορούν να συνυπάρχουν.

Το DBR είναι:

</> Plain text

software environment

Το Photon είναι:

</> Plain text

query execution engine μέσα στο execution stack

Πολύ απλοποιημένα:

</> Plain text

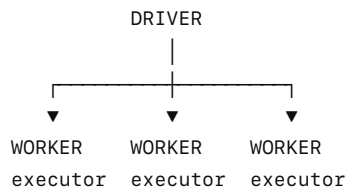
```
graph TD
 A[Your PySpark] --> B[DBR environment]
 B --> C[Catalyst planner]
 B --> D[execution]
 D --> E[Photon]
 D --> F[JVM Spark]
```

## 21. Driver node: τι ακριβώς είναι;

Τώρα πάμε στο hardware layer.

Ένα classic multi-node Spark compute:

Plain text



Ο **driver** είναι ο coordinator.

Η Databricks περιγράφει τον driver ως το node που κρατά το state των attached notebooks, το `SparkContext`, ερμηνεύει commands και τρέχει το Spark master/coordinator για τους executors. [Databricks Docu...](#)

## 22. Τι κάνει πρακτικά ο driver;

Όταν γράφεις:

Python

```
df = spark.read.table("orders")

result = (
 df.groupBy("country")
 .sum("amount")
)

display(result)
```

conceptually ο driver κάνει:

Plain text

```
receive Python/notebook command
 ↓
build Spark logical plan
 ↓
analyze/optimize plan
 ↓
construct stages/tasks
 ↓
schedule tasks on workers
 ↓
monitor execution
 ↓
receive result metadata / result
 ↓
send output to notebook
```

Οι workers κάνουν το bulk distributed processing.

## 23. Τι είναι Driver type ;

Δεν είναι διαφορετικός τύπος software.

Είναι:

ποιο cloud VM instance type θα χρησιμοποιήσω για τον driver;

Παράδειγμα AWS:

</> Plain text

```
m5.xlarge
m5.2xlarge
r5.4xlarge
...
```

Ανάλογα το cloud θα δεις διαφορετικά names.

## 24. Γιατί χρειάζεται RAM ο driver αφού οι workers κάνουν την επεξεργασία;

Επειδή ο driver έχει σημαντικό state.

Ειδικά μπορεί να έχει πρόβλημα αν κάνεις:

</> Python

```
df.collect()
```

Τι σημαίνει collect() ;

</> Plain text

```
distributed rows on workers
 ↓
 ↓
 ↓
 DRIVER
```

Όλα επιστρέφουν στον driver.

Αν:

</> Plain text

```
data = 50 GB
driver RAM = 16 GB
```

έχεις πολύ πιθανό:

</> Plain text

```
driver OOM
```

Η Databricks μάλιστα προτείνει μεγαλύτερο driver type όταν περιμένεις πολλά collected data στον driver. [Databricks Docu...](#)

Άλλα driver-memory stressors είναι μεγάλα plans, πάρα πολλά task metadata, πολλές attached notebook sessions και local Python objects.

## 25. Worker node

Ο worker είναι εκεί όπου γίνεται η distributed εργασία.

</> Plain text

```
WORKER 1
partition 1
partition 2
```

```
WORKER 2
partition 3
partition 4
```

```
WORKER 3
partition 5
partition 6
```

Σε Databricks multi-node classic compute κάθε worker τρέχει Spark executor και η Databricks documentation χρησιμοποιεί συχνά worker/executor σχεδόν εναλλακτικά σε αυτή την αρχιτεκτονική. [Databricks Docu...](#)

## 26. Worker type

Worker type σημαίνει:

Ποιο cloud VM type θέλω για **κάθε worker**;

Έστω:

</> Plain text

```
worker type:
8 CPU cores
32 GB RAM
```

```
workers:
4
```

Έχεις περίπου:

</> Plain text

```
4 workers
x
8 cores
=
32 worker CPU cores
```

και:

</> Plain text

```
4
x
32 GB
=
128 GB aggregate worker RAM
```

όχι όμως σαν ένα ενιαίο 128 GB machine — είναι distributed.

## 27. Worker type και workload characteristics

Διαφορετικά node families δίνουν διαφορετικές αναλογίες:

</> Plain text

```
CPU
RAM
local disk
network throughput
GPU
```

Ένα CPU-heavy workload μπορεί να θέλει compute-optimized machines.

Ένα:

</> Plain text

```
large join
cache
wide aggregation
```

μπορεί να πιέζει περισσότερο RAM.

Ένα huge shuffle μπορεί να πιέσει:

</> Plain text

```
network
local disk
memory
```

Δεν υπάρχει:

ο καλύτερος worker type.

Είναι workload-dependent.

## 28. Driver type και Worker type μπορούν να διαφέρουν

Για παράδειγμα:

</> Plain text

```
Driver:
16 cores
64 GB RAM
```

```
Workers:
8 cores
32 GB RAM
```

ή αντίστροφα ανά ανάγκη.

Default, όμως, το driver type είναι το ίδιο με worker type. [Databricks Docu...](#)

Άρα:

</> Plain text

```
driver type
```

και:

</> Plain text

```
worker type
```

είναι δύο διαφορετικές parameters του classic compute.

## 29. Scaling up vs scaling out

Αυτό θα το συναντάς παντού.

### Scale up

Μεγαλύτερο machine:

</> Plain text

```
worker:
8 CPU → 32 CPU
32 GB → 128 GB
```

## Scale out

Περισσότερα machines:

</> Plain text

```
4 workers → 16 workers
```

Αυτά είναι διαφορετικές στρατηγικές.

Spark είναι σχεδιασμένο κυρίως για distributed scale-out, αλλά node sizing εξακολουθεί να έχει τεράστια σημασία.

## 30. Τι είναι autoscaling στο classic compute;

Έστω:

</> Plain text

```
Worker type:
8 CPU
32 GB
```

και ορίζεις:

</> Plain text

```
Min workers = 2
Max workers = 10
```

Τότε Databricks μπορεί να αλλάζει:

</> Plain text

```
2
↓
4
↓
8
↓
10
↓
6
↓
2
```

ανάλογα με workload.

Το **type** κάθε worker παραμένει αυτό που επέλεξες.

Αλλάζει ο αριθμός τους.

</> Plain text

```
horizontal scaling
=
more/fewer workers
```

Η classic compute configuration επιτρέπει explicit Min και Max workers. [Databricks Docu...](#)

## 31. Τι γίνεται με autoscaling στο serverless;

Εδώ η απάντηση είναι:

**Autoscaling υπάρχει, αλλά συνήθως δεν είναι parameter που εσύ διαχειρίζεσαι.**

Για serverless notebooks/jobs:

</> Plain text

```
YOU
 ↓
submit workload
 ↓
DATABRICKS
 ├── select resources
 ├── scale resources
 ├── adjust memory
 └── optimize infrastructure
```

Στα serverless Jobs, autoscaling και Photon ενεργοποιούνται αυτόματα. [Databricks Docu...](#)

Άρα δεν έχεις classic-style:

</> Plain text

```
min_workers = 4
max_workers = 20
worker_type = r5.4xlarge
```

## 32. Άρα autoscaling είναι axis μόνο για classic;

Χρειάζεται ακριβής διατύπωση:

### Conceptually

Autoscaling υπάρχει και στα δύο.

</> Plain text

```
Classic → autoscaling
Serverless → autoscaling
```



## Ως manual configuration axis

Κυρίως στο classic:

</> Plain text

```
Classic:
fixed 8
```

OR

```
autoscale 2-10
```

Στο general serverless:

</> Plain text

```
automatic scaling is inherent
```

άρα δεν έχεις checkbox:

</> Plain text

```
Enable autoscaling?
```

Είναι ήδη μέρος του service.

---

## 33. Ειδική περίπτωση: serverless Lakeflow Pipeline

Τα pipelines είναι ακόμη πιο ενδιαφέροντα.

Serverless pipelines έχουν:

</> Plain text

```
horizontal autoscaling
+
vertical autoscaling
```

Horizontal:

</> Plain text

```
more/fewer workers/executors
```

Vertical:

</> Plain text

```
larger/smaller machine types
```

Η Databricks μπορεί, για παράδειγμα, να δει OOM και στην επόμενη update να χρησιμοποιήσει μεγαλύτερο worker/driver instance. [Databricks Docu...](#) +1

Δεν επιλέγεις εσύ:

</> Plain text

```
worker type = X
```

σε serverless pipeline.

Η Databricks το κάνει.

Μπορείς όμως να βάλεις **Max workers upper bound** για κόστος/capacity, ενώ δεν κάνεις το full classic worker configuration.

[Databricks Docu...](#)

## 34. Ειδική περίπτωση: SQL Warehouse autoscaling

Και εδώ η λέξη autoscaling χρησιμοποιείται λίγο διαφορετικά.

Έστω SQL warehouse:

</> Plain text

Size:

Medium

Μία cluster unit έχει συγκεκριμένη capacity.

Αν έρχονται πολλοί users:

</> Plain text

10 queries

50 queries

100 queries

το SQL warehouse μπορεί να αυξήσει τον αριθμό των **warehouse clusters** για concurrency:

</> Plain text

1 cluster

↓

2

↓

3

...

Serverless SQL warehouses χρησιμοποιούν Intelligent Workload Management για automatic scaling. [Databricks Docu...](#)

Μπορείς επίσης να έχεις warehouse scaling configuration με minimum/maximum cluster counts. [Databricks Docu...](#)

Αυτό όμως δεν είναι το ίδιο concept με:

</> Plain text

```
Spark cluster:
2-10 workers
```

Άρα έχουμε:

</> Plain text

```
Classic Spark autoscaling
→ workers
```

```
SQL warehouse concurrency scaling
→ warehouse clusters
```

```
Serverless compute scaling
→ managed infrastructure
```

```
Serverless pipeline
→ horizontal + vertical
```

## 35. All-purpose: είναι πραγματικά μόνο interactive;

Όχι. Δεν είναι hard technical restriction.

Αυτό είναι σημαντική διόρθωση.

Το all-purpose compute είναι σχεδιασμένο και προτεινόμενο κυρίως για:

</> Plain text

```
interactive development
ad-hoc analysis
debugging
exploration
```

αλλά μπορείς να χρησιμοποιήσεις existing all-purpose classic compute για ορισμένα Job tasks.

Η Databricks το υποστηρίζει αλλά λέει ρητά ότι **δεν το συνιστά για production jobs**. [Databricks Docu...](#) +1

## 36. Γιατί τότε το λέγαμε interactive;

Γιατί αυτό είναι το intended lifecycle.

All-purpose:

</> Plain text

```
start cluster
```

```
↓
Nikos notebook
↓
run
↓
think for 10 minutes
↓
run again
↓
Maria attaches
↓
Nikos debugs
↓
more notebooks...
```

Το cluster ζει ανεξάρτητα από μία συγκεκριμένη task.

Jobs compute:

```
</> Plain text

Job starts
↓
compute starts
↓
job executes
↓
job completes
↓
compute terminates
```

Αυτό κάνει το Jobs compute πολύ πιο κατάλληλο operational model.

## 37. Γιατί δεν προτείνεται All-purpose για production Jobs;

Υπάρχουν αρκετοί λόγοι.

Το all-purpose μπορεί να χρησιμοποιείται ταυτόχρονα από:

```
</> Plain text

Developer A
Developer B
Notebook C
Production Job
```

και τότε:

```
</> Plain text

CPU
RAM
executors
shuffle bandwidth
```

είναι shared.

Άρα ένα random notebook:

```
</> Python
```

```
df.crossJoin(other_huge_df).count()
```

μπορεί να επηρεάσει production ETL.

Επιπλέον all-purpose compute έχει διαφορετικό billing model, μπορεί να παραμένει idle και οι job-specific tuning recommendations δεν ταιριάζουν πάντα σε interactive shared cluster. [Databricks Docu...](#)

## 38. Υπάρχουν περιπτώσεις που All-purpose για Job έχει νόημα;

Ναι.

Η Databricks αναφέρει ως limited exceptions:

```
</> Plain text
```

```
iterative development/testing ενός Job
```

γιατί αποφεύγεις repeated startup,

και:

```
</> Plain text
```

```
very short frequent jobs
```

όπου ένα ήδη-running all-purpose compute μπορεί να μειώσει startup latency. [Databricks Docu...](#)

Άρα:

all-purpose = interactive by design, όχι interactive-only technical restriction.

## 39. Τι είναι Pool;

Αυτό είναι από τα concepts που μπερδεύουν σχεδόν όλους στην αρχή.

Το πλήρες όνομα είναι:

**Instance Pool**

Δεν είναι:

Plain text

```
thread pool
connection pool
data pool
Spark executor pool
```

Είναι:

μια δεξαμενή από ήδη provisioned cloud VM instances που μπορούν να χρησιμοποιηθούν γρήγορα από classic compute.

## 40. Γιατί δημιουργήθηκαν pools;

Χωρίς pool:

Plain text

```
User:
start cluster

↓

Databricks asks AWS:
give me 8 VMs

↓

AWS provisions VMs

↓

VM boot

↓

DBR install/start

↓

cluster ready
```

Αυτό μπορεί να πάρει χρόνο.

Με pool:

Plain text

```
INSTANCE POOL
```

```
| idle VM |
| idle VM |
| idle VM |
| idle VM |
```



Τα machines είναι ήδη provisioned/warm.

Άρα cluster startup μπορεί να γίνει πολύ ταχύτερα.

## 41. Pool lifecycle

Έστω pool:

</> Plain text

```
minimum idle = 3
maximum capacity = 20
```

Αρχικά:

</> Plain text

```
3 idle VMs
```

Έρχεται Cluster A:

</> Plain text

```
needs 2 VMs
```

Pool:

</> Plain text

```
gives 2 existing idle VMs
```

και προσπαθεί ξανά να διατηρήσει minimum idle capacity.

Έρχεται Cluster B:

</> Plain text

```
needs 5 more
```

παίρνει επίσης instances από το pool ή το pool ζητά νέα cloud machines.

Η Databricks ορίζει ακριβώς minimum idle , maximum capacity και idle instance auto-termination για pools.

## 42. Preloaded DBR σε Pool

Pool μπορεί να έχει:

</> Plain text

```
Preloaded DBR = 18 LTS
```

οπότε τα idle machines μπορούν ήδη να έχουν το runtime cached/preloaded.

Τότε:

</> Plain text

```
cluster wants DBR 18
↓
VM already ready
↓
very fast startup
```

Η Databricks το υποστηρίζει ακριβώς για να επιταχύνει cluster launch. [Databricks Docu...](#)

## 43. Pool και autoscaling μαζί

Ναι.

Έστω:

</> Plain text

```
cluster:
min = 2
max = 10

pool:
max capacity = 20
```

Cluster αυτή τη στιγμή:

</> Plain text

```
2 workers
```

Spark workload μεγαλώνει.

Autoscaler:

</> Plain text

```
I need 4 more workers
```



Αν pool έχει idle:

</> Plain text

```
give 4 immediately
```

Αλλιώς:

</> Plain text

```
pool → cloud → provision more
```

Άρα:

</> Plain text

```
POOL
=
where machines come from
```

Ενώ:

</> Plain text

```
AUTOSCALING
=
how many machines compute currently wants
```

Δύο εντελώς διαφορετικά concepts.

## 44. Pool και Serverless

Στο serverless δεν διαλέγεις instance pool.

Γιατί;

Επειδή όλο το πρόβλημα:

</> Plain text

```
prewarm instances
manage capacity
choose machines
keep idle infrastructure
```

είναι πλέον πρόβλημα της Databricks.

Η σημερινή documentation μάλιστα προτείνει serverless αντί pools όταν το workload το υποστηρίζει. [Databricks Docu...](#)

## 45. Τι είναι Termination;

Πολύ σημαντικό να διαχωρίσουμε:

</> Plain text

```
terminate compute
```

από:

</> Plain text

```
delete compute configuration
```

Αν τερματίσεις classic all-purpose compute:

</> Plain text

```
running VMs
↓
terminated/released
```

αλλά η Databricks configuration:

</> Plain text

```
name
runtime
worker type
autoscaling
settings
```

παραμένει.

Μπορείς αργότερα να κάνεις:

</> Plain text

```
Start
```

και να δημιουργηθούν νέα VMs.

## 46. Τι χάνεται κατά termination;

Χάνεται το ephemeral compute state.

Για παράδειγμα:

</> Plain text

```
executor memory
cached DataFrames
shuffle files
local temporary files
Python process state
in-memory variables/session state
```

Δεν χάνεται:

</> Plain text

```
Delta table in ADLS/S3/GCS
Unity Catalog metadata
notebook source code
job definition
```

Επιστρέφουμε δηλαδή στη βασική αρχή:

</> Plain text

```
compute lifetime
#
persistent data lifetime
```

## 47. Auto Termination

Για all-purpose compute μπορείτε να πεις:

</> Plain text

```
Auto terminate after:
30 minutes inactivity
```

Αν δεν τρέχει command για το configured διάστημα:

</> Plain text

```
active
↓
idle
↓
30 minutes
↓
terminate
```

Η classic compute configuration υποστηρίζει ακριβώς inactivity-based automatic termination. [Databricks Docu...](#)

## 48. Jobs compute termination είναι διαφορετικό

Classic Jobs compute:

</> Plain text

```
job starts
 ↓
compute starts
 ↓
job completes
 ↓
compute terminates
```

Δεν περιμένει:

</> Plain text

30 minutes idle

Η lifecycle του συνδέεται με το Job run. [Databricks Docu...](#)

Άρα:

</> Plain text

```
All-purpose auto termination
=
idle-time based

Jobs compute termination
=
job-lifecycle based
```

## 49. SQL Warehouse Auto Stop

SQL warehouses έχουν αντίστοιχη αλλά δική τους έννοια:

</> Plain text

Auto Stop

Σήμερα defaults είναι συνήθως:

</> Plain text

```
Serverless SQL Warehouse:
10 min

Pro / Classic:
45 min
```

με διαφορετικά minimums. [Databricks Docu...](#)

Άρα ξανά:

</> Plain text

```
classic compute auto termination
```

και:

</> Plain text

```
SQL warehouse auto stop
```

είναι παρόμοιο lifecycle concept αλλά διαφορετικό product configuration.

---

## 50. SQL Warehouse access management

Τώρα πάμε σε security, και εδώ πρέπει να χωρίσεις δύο διαφορετικές πόρτες.

Έστω:

</> SQL

```
SELECT *
FROM prod.finance.payroll;
```

μέσω SQL Warehouse.

Για να πετύχει, υπάρχουν δύο ξεχωριστά questions:

</> Plain text

```
Gate 1:
May Nikos use this compute?
```

```
Gate 2:
May Nikos read prod.finance.payroll?
```

---

## 51. Gate 1 — Warehouse ACL

To SQL warehouse είναι workspace object.

Έχει permissions όπως:

Permission	Τι σου επιτρέπει
CAN VIEW	βλέπεις warehouse/details
CAN MONITOR	βλέπεις/παρακολουθείς και μπορείς να τρέξεις queries
CAN USE	χρησιμοποιείς το warehouse για queries
CAN MANAGE	αλλάζεις configuration/permissions
IS OWNER	ownership/admin-like control

Η current ACL matrix της Databricks ορίζει ακριβώς αυτά τα levels. [Databricks Docu...](#) +1

## 52. CAN USE δεν σημαίνει ότι μπορείς να διαβάσεις όλα τα data

Αυτό είναι εξαιρετικά σημαντικό.

Έστω:

</> Plain text

```
Nikos
CAN USE warehouse
```

αλλά Unity Catalog:

</> Plain text

```
prod.finance.payroll
```

SELECT:



Query:

</> SQL

```
SELECT *
FROM prod.finance.payroll;
```

→ **permission denied.**

Το CAN USE σου λέει:

Επιτρέπεται να χρησιμοποιήσεις αυτή τη μηχανή query execution.

Δεν σου λέει:

Έχεις πρόσβαση σε κάθε table που μπορεί να δει η μηχανή.

## 53. Gate 2 — Unity Catalog

Για:

</> Plain text

```
prod.finance.payroll
```

χρειάζεσαι κατάλληλα UC privileges, conceptual example:

</> Plain text

```
USE CATALOG on prod
USE SCHEMA on finance
SELECT on payroll
```

Άρα:

</> Plain text

```
Warehouse ACL
+
Unity Catalog permissions
=
query possible
```

Αυτό το pattern θα το βλέπεις συνέχεια στο Databricks:

**Permission to use compute ≠ permission to use data.**

## 54. Παράδειγμα SQL Warehouse

Έχουμε:

</> Plain text

```
analysts group
```

```
Warehouse:
analytics-wh
CAN USE 
```

Unity Catalog:

</> Plain text

```
prod.sales.*
SELECT 
```

```
prod.hr.*
SELECT ❌
```

Τότε:

```
</> SQL

SELECT *
FROM prod.sales.orders;
```



αλλά:

```
</> SQL

SELECT *
FROM prod.hr.salary;
```



παρότι χρησιμοποιεί ακριβώς το ίδιο SQL warehouse.

## 55. Τι είναι CAN MONITOR;

Ένας power user / performance engineer ίσως χρειάζεται:

```
</> Plain text

query history
query profiles
warehouse monitoring
```

χωρίς να πρέπει να αλλάζει:

```
</> Plain text

warehouse size
scaling
permissions
```

Εκεί ταιριάζει CAN MONITOR .

Η current ACL table επιτρέπει start/run queries και monitoring/query-profile capabilities. [🌐 Databricks Docu...](#)

## 56. SQL Warehouse access mode Standard/Dedicated;

Όχι.



SQL warehouses **δεν χρησιμοποιούν το same Standard/Dedicated access-mode selector** που έχεις στα classic general-purpose Spark compute.

Έχουν το δικό τους:

Plain text

```
warehouse ACL
+
Unity Catalog user isolation/governance
+
warehouse security architecture
```

Αρα δεν λες:

Plain text

```
Serverless SQL Warehouse
+
Dedicated access mode
```

Αυτό δεν είναι valid axis.

## 57. Pipeline access management

Εδώ έχουμε **τρεις διαφορετικές security layers**, και αυτό θέλω να το καταλάβεις πολύ καλά.

Plain text

```
Layer 1:
Can you access/control the PIPELINE OBJECT?

Layer 2:
Under WHOSE IDENTITY does the pipeline execute?

Layer 3:
What DATA can that identity access?
```

## 58. Layer 1 — Pipeline ACL

Το Lakeflow pipeline είναι workspace object.

Έχει:

Permission	Capabilities
CAN VIEW	βλέπει pipeline/details
CAN RUN	start/stop pipeline update
CAN MANAGE	edit settings + permissions + run
IS OWNER	owner

Η current Databricks ACL matrix το ορίζει έτσι. [Databricks Docu...](#)

## 59. Layer 2 — Run-as identity

Έστω:

</> Plain text

```
Nikos:
CAN RUN pipeline
```

Πατάς:

</> Plain text

```
Start
```

Ποια permissions χρησιμοποιεί το pipeline όταν διαβάσει:

</> Plain text

```
prod.raw.orders
```

;

Όχι απαραίτητα τα δικά σου.

Το pipeline έχει:

</> Plain text

```
Run as:
etl-service-principal
```

και οι updates εκτελούνται με αυτή την identity.

By default, run-as είναι ο creator, αλλά μπορεί να αλλάξει σε άλλο user ή service principal. Databricks συνιστά service principal για production. [Databricks Docu...](#)

## 60. Γιατί αυτό είναι σημαντικό;

Έστω:

</> Plain text

```
Nikos
CAN RUN pipeline
```

αλλά:

</> Plain text

```
Nikos:
SELECT prod.raw.orders ❌
```

Service principal:

</> Plain text

```
pipeline-prod-sp:
SELECT prod.raw.orders ✅
CREATE TABLE prod.silver ✅
```

Ο Nikos μπορεί να ενεργοποιήσει το pipeline επειδή:

</> Plain text

```
CAN RUN
```

και το pipeline μπορεί να εκτελεστεί επειδή:

</> Plain text

```
run-as service principal
```

έχει τα data privileges.

Είναι το ίδιο σημαντικό pattern που είδαμε στα Jobs.

## 61. Layer 3 — Unity Catalog privileges

Ένα production pipeline service principal μπορεί να χρειάζεται, π.χ.:

</> Plain text

```
USE CATALOG
USE SCHEMA
SELECT on sources
```

```
CREATE TABLE
or
CREATE MATERIALIZED VIEW
on target schema
```

Η Databricks προτείνει να δίνεις μόνο τα απαραίτητα UC privileges στο pipeline service principal. [Databricks Docu...](#)

## 62. Pipeline permission ≠ output data permission

Έστω Maria έχει:

</> Plain text

```
CAN VIEW pipeline
```

Αυτό δεν σημαίνει:

</> Plain text

```
SELECT prod.gold.revenue
```

Η output table είναι Unity Catalog object με **δικά της permissions**.

Για να διαβάσει:

</> SQL

```
SELECT *
FROM prod.gold.revenue;
```

χρειάζεται τα αντίστοιχα UC privileges.

Αν θέλει επίσης να ανοίξει το backing pipeline από streaming table/materialized view, non-admin users μπορεί να χρειάζονται και **REFRESH** privilege στο αντίστοιχο object. [Databricks Docu...](#)

## 63. Τι γίνεται με compute permissions ενός pipeline;

Serverless pipeline:

</> Plain text

```
Pipeline
↓
Databricks-managed compute
```

Δεν δημιουργείς cluster και δεν δίνεις:

</> Plain text

CAN ATTACH TO

σε αυτό.

Η Databricks λέει ρητά ότι serverless pipelines δεν έχουν cluster sizing/configuration/security management από τον χρήστη.

[Databricks Docu...](#)

Classic pipeline:

</> Plain text

Pipeline

↓

pipeline-managed classic compute

Έχεις περισσότερες infrastructure επιλογές, αλλά πάλι το compute lifecycle διαχειρίζεται από το pipeline και δεν το αντιμετωπίζεις σαν ένα arbitrary all-purpose cluster. Δεν μπορείς π.χ. να ριπάρεις ελεύθερα Spark version· ο pipeline runtime το διαχειρίζεται. [Databricks Docu...](#)

## 64. Serverless compute έχει access management;

Ναι — απλώς διαφορετικό από Standard/Dedicated.

Σήμερα ένα workspace έχει built-in serverless compute objects:

</> Plain text

Default Interactive Compute

Default Automated Compute

Το πρώτο ελέγχει serverless notebooks / Databricks Connect.

Το δεύτερο ελέγχει serverless Jobs / Lakeflow pipelines.

Έχουν:

</> Plain text

CAN USE

CAN MANAGE

και admins μπορούν να περιορίσουν ποιοι users/groups έχουν πρόσβαση σε serverless. [Databricks Docu...](#) +1

Άρα:

</> Plain text

Standard/Dedicated

δεν είναι universal access axis.

Είναι κυρίως classic compute access-mode concept.

## 65. Τώρα ας τα βάλουμε όλα σε μία πραγματική matrix

Axis	Classic All-purpose	Classic Jobs	Serverless Notebook	Serverless Jobs
Purpose	interactive-oriented	automated	interactive	automated
DBR	επιλέγεις	επιλέγεις	managed	managed
Photon	configurable	configurable	enabled	enabled
Driver type	επιλέγεις	επιλέγεις	managed	managed
Worker type	επιλέγεις	επιλέγεις	managed	managed
Fixed workers	✓	✓	✗	✗
Min/max worker autoscale	✓	✓	✗	✗
Automatic scaling	optional	optional	✓ managed	✓ managed
Pool	✓	✓	✗	✗
Standard/Dedicated	✓	✓	όχι ως selector	όχι ως selector
Auto termination	configurable	lifecycle-based	managed	lifecycle-based
GPU	possible classic configuration	possible	AI Runtime separately	AI Runtime separately
Job use	supported but discouraged prod	intended use	N/A	intended use

## 66. Και Lakeflow Pipeline

Axis	Classic Pipeline	Serverless Pipeline
Compute ownership	customer/classic	Databricks managed
Runtime	pipeline-managed channel	pipeline-managed
Instance types	configurable	managed
Worker autoscaling	enhanced, configurable	enhanced automatic
Vertical scaling	limited/manual node choice	automatic
Photon	configurable	enabled
Unity Catalog	supported	required
Legacy Hive	possible	no
Pool/custom cluster settings	some classic controls	no
Pipeline ACL	CAN VIEW/RUN/MANAGE/OWNER	same
Run-as	user/service principal	same

## 67. Kql SQL Warehouse

Parameter	Serverless SQL WH	Pro	Classic
DBR selector	✗	✗	✗
Photon	✓	✓	✓
Warehouse size	✓	✓	✓
Concurrent cluster scaling	✓	✓	✓
Intelligent workload management	✓	✗	✗
Predictive IO	✓	✓	✗
Standard/Dedicated	✗	✗	✗
Warehouse ACL	✓	✓	✓
Unity Catalog data ACL	✓	✓	✓
Auto Stop	✓	✓	✓

## 68. Ένα complete classic configuration

Τώρα μπορείς να καταλάβεις ένα configuration σαν αυτό:

</> Plain text

Name:  
etl-development

Purpose:  
All-purpose

Infrastructure:  
Classic

Access mode:  
Standard

Runtime:  
DBR 18 LTS

Photon:  
Enabled

Driver:  
r6i.2xlarge

Worker:  
r6i.xlarge

Autoscaling:  
min 2

Αυτό περιγράφει **ένα compute**.

Αν το διαβάσουμε:

</> Plain text

All-purpose  
→ lifecycle/purpose

Classic  
→ infrastructure management

Standard  
→ access/security mode

DBR 18 LTS  
→ software environment

Photon  
→ execution engine

r6i.2xlarge  
→ driver hardware



```
r6i.xlarge
→ worker hardware

2-8
→ horizontal scaling policy

pool
```

Τώρα κάθε parameter έχει διαφορετική θέση στο architecture.

## 69. Τι συμβαίνει όταν πατάς Start σε αυτό;

Conceptually:

⌕ Plain text

```
1. Need compute
 ↓
2. Get driver VM
 possibly from pool
 ↓
3. Get min worker VMs
 ↓
4. Start/install DBR 18 LTS
 ↓
5. Start Spark driver
 ↓
6. Start Spark executors
 ↓
7. Photon available/enabled
 ↓
8. Spark session ready
 ↓
9. Notebook attaches
```

Έπειτα:

⌕ Python

```
df.groupBy("country").sum("amount")
```

μπορεί να γίνει:

⌕ Plain text

```
Notebook
 ↓
Driver
 ↓
Catalyst
 ↓
physical plan
 ↓
Photon
```

```
↓
tasks
↓
workers
↓
partitions
```

Τώρα έχεις ολόκληρη την αλυσίδα.

## 70. Ένα πολύ συνηθισμένο interview/debugging scenario

Έχεις:

</> Plain text

```
Driver:
8 GB
```

```
Workers:
10 × 64 GB
```

Άρα συνολικό worker RAM:

</> Plain text

```
640 GB
```

Κάνεις:

</> Python

```
huge_df.collect()
```

και driver crashes με OOM.

Junior reasoning:

Μα έχω 640 GB RAM!

Λάθος.

Η `collect()` μεταφέρει data:

</> Plain text

```
workers
↓
driver
```

και ο driver έχει μόνο:

</> Plain text

8 GB

Αυτό δείχνει γιατί **driver type** και **worker type** είναι διαφορετικοί axes.

## 71. Άλλο scenario

Spark UI δείχνει:

</> Plain text

```
all workers:
CPU saturated
little/no spill
memory okay
huge CPU processing time
```

Πιθανή σκέψη:

</> Plain text

worker CPU bottleneck

Μπορείς να εξετάσεις:

</> Plain text

```
more workers
or
larger/more CPU-oriented workers
or
Photon
or
query optimization
```

Δεν λες:

πάρε μεγαλύτερο driver.

Ο driver δεν είναι το bottleneck.

## 72. Άλλο scenario

Spark UI:

</> Plain text

```
tasks spilling massive amounts to disk
executor OOMs
```

Εδώ μπορεί να χρειάζεσαι:

</> Plain text

```
more worker memory
better partitioning
less skew
different join
more workers
larger workers
```

πάλι όχι απαραίτητα μεγαλύτερο driver.

## 73. Άλλο scenario: cluster αργεί να ξεκινήσει

Query performance όταν ξεκινήσει:

</> Plain text

```
excellent
```

αλλά startup:

</> Plain text

```
4 minutes
```

Το να αλλάξεις:

</> Plain text

```
worker RAM
```

δεν λύνει απαραίτητα το πρόβλημα.

Μπορεί:

</> Plain text

```
pool
```

να βοηθήσει, επειδή μειώνει VM provisioning/runtime startup latency.

Ή ακόμη καλύτερα, αν workload compatible:

</> Plain text

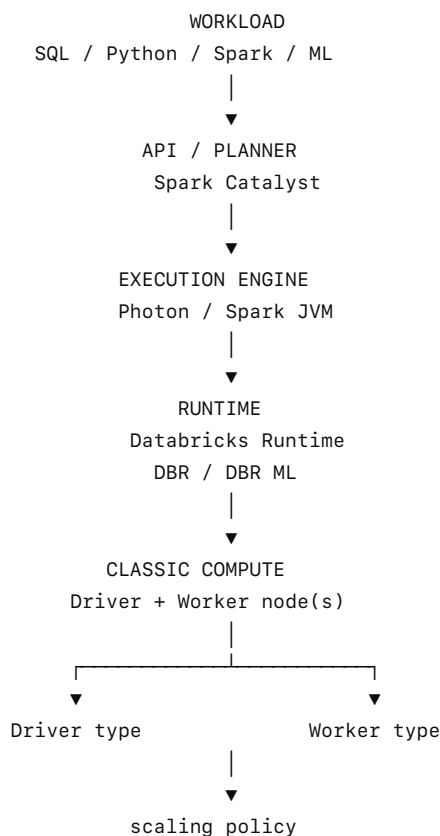
```
serverless
```

ώστε να μην διαχειρίζεσαι το provisioning problem καθόλου.

## 74. Τι θέλω να έχεις ξεκαθαρίσει πλέον

Το πλήρες stack είναι:

</> Plain text



Serverless ουσιαστικά συμπύκνωση μεγάλο μέρος του κάτω μισού:

</> Plain text

```
WORKLOAD
↓
Databricks serverless
↓
managed runtime
managed Photon
managed machine types
managed scaling
managed provisioning
managed lifecycle
```

Και αυτή ακριβώς είναι η αξία του.

## Τελικό τεστ

1. **DBR 18 LTS και r5.4xlarge είναι δύο versions του ίδιου πράγματος;**  
Όχι. DBR = software stack. r5.4xlarge = cloud VM/hardware instance type.
2. **DBR και Photon είναι ανταγωνιστικές επιλογές;**  
Όχι. Ένα compute μπορεί να χρησιμοποιεί DBR 18 LTS και Photon.
3. **Τι είναι το Photon;**  
Native C++ vectorized execution engine για supported Spark SQL/DataFrame operations, κάτω από το ίδιο Spark API.
4. **Ποια είναι η εναλλακτική όταν Photon είναι off/unsupported;**  
Traditional Spark/JVM execution.
5. **Ποιος κάνει Catalyst optimization όταν χρησιμοποιείται Photon;**  
Το Spark Catalyst εξακολουθεί να κάνει planning/optimization. Photon αναλαμβάνει supported execution.
6. **Τι είναι worker type;**  
Το cloud VM shape κάθε worker/executor node.
7. **Τι είναι driver type;**  
Το cloud VM shape του Spark driver/coordinator.
8. **Έχω 20 workers με 64 GB RAM αλλά driver 8 GB. Μπορώ να collect() 100 GB;**  
Όχι. Το result συγκεντρώνεται στον driver και πιθανότατα θα προκαλέσει OOM.
9. **Autoscaling σημαίνει serverless;**  
Όχι. Classic cluster μπορεί να autoscale.
0. **Μπορώ να επιλέξω min=2/max=20 workers σε ordinary serverless Job;**  
Όχι με το classic cluster configuration model. Η Databricks διαχειρίζεται αυτόματα scaling/resources.
1. **Υπάρχει scaling στο serverless;**  
Ναι. Είναι intrinsic/platform-managed και όχι συνήθως user-selected fixed/autoscaling axis.
2. **Τι είναι pool;**  
Reservoir/pre-provisioned set cloud instances που χρησιμοποιούν classic clusters για γρηγορότερο provisioning.
3. **Pool και autoscaling είναι το ίδιο;**  
Όχι. Autoscaling αποφασίζει πόσα nodes θέλει το cluster. Pool μπορεί να τα παρέχει γρηγορότερα.
4. **Τι γίνεται όταν all-purpose compute auto-terminates;**  
Τα VMs/ephemeral state σταματούν, αλλά η compute configuration και persistent tables/notebooks παραμένουν.
5. **All-purpose είναι τεχνικά restricted σε notebooks;**  
Όχι. Μπορεί να χρησιμοποιηθεί από Jobs σε supported cases, αλλά Databricks δεν το προτείνει για production Jobs.
6. **Γιατί όχι;**  
Billing, resource contention, idle lifecycle και λιγότερο predictable production behavior.
7. **CAN USE SQL Warehouse = SELECT σε όλα τα tables;**  
Όχι. Είναι μόνο permission πάνω στο compute. Unity Catalog data privileges είναι ξεχωριστά.
8. **Maria έχει CAN RUN pipeline αλλά όχι SELECT στη source table. Μπορεί να τρέξει pipeline;**  
Ναι, εφόσον το pipeline's run-as identity έχει τα απαραίτητα source/target permissions.
9. **CAN VIEW pipeline = SELECT στα output tables;**  
Όχι. Pipeline object ACL και Unity Catalog data ACL είναι διαφορετικά.
0. **Τι identity χρησιμοποιεί production pipeline;**  
Το configured run-as principal. Η Databricks συνιστά συνήθως service principal αντί προσωπικού user.
1. **Serverless SQL Warehouse έχει Standard/Dedicated access mode;**  
Όχι. Τα SQL warehouses έχουν δικό τους warehouse ACL/security model.
2. **Serverless pipeline έχει worker type dropdown;**  
Όχι. Databricks επιλέγει τα underlying resources.
3. **Serverless pipeline μπορεί παρ' όλα αυτά να autoscale;**  
Ναι — και horizontal και vertical scaling είναι διαθέσιμα στο managed model. [⊕ Databricks Docu...](#)

#### 4. Ποια είναι η ουσιαστική διαφορά classic και serverless;

Classic:

</> Plain text

You manage/configure the infrastructure envelope.

Serverless:

</> Plain text

You specify the workload;  
Databricks manages the infrastructure envelope.

#### 5. Πού βρίσκεται το runtime μέσα σε όλο το stack;

</> Plain text

```
hardware
 ↑
runtime
 ↑
execution engine / Spark
 ↑
your application
```

ή από την πλευρά του application:

</> Plain text

```
your code
 ↓
Spark/Photon
 ↓
Databricks Runtime
 ↓
driver/workers
 ↓
VM/CPU/RAM
```

Αυτό είναι το μοντέλο που θέλω να έχεις πλέον για το Databricks Compute. Η επόμενη φυσική ενότητα πριν επιστρέψουμε στο Delta/Unity Catalog είναι να πάρουμε ένα συγκεκριμένο **Classic cluster configuration** και να υπολογίσουμε πραγματικά **cores, RAM, executors, task slots, partitions, driver memory, worker memory, autoscaling και shuffle**, ώστε όταν βλέπεις π.χ. `4 workers × 8 cores` να μπορείς να προβλέψεις τι σημαίνει αυτό για ένα πραγματικό Spark job.

Sunday 8:52 AM

Continue with teaching databricks in depth. What's the plan? Remember i want to learn both theory/best practices AND syntax/practical use of platform.

Ναι. Από εδώ και πέρα θέλω να αλλάξουμε λίγο τη δομή του μαθήματος: όχι άλλο «ένα concept τη φορά» αποκομμένο από το πραγματικό Databricks. Θα χτίσουμε **ολόκληρη την πλατφόρμα σαν να προετοιμάζεσαι ταυτόχρονα για δουλειά, projects και certification.**

Και σε κάθε ενότητα θα κάνουμε υποχρεωτικά και τα δύο:

**Theory / architecture / best practices + πραγματικό UI, SQL, PySpark, CLI/YAML syntax, debugging και exercises.**

Αυτό ταιριάζει και με τη σημερινή Data Engineer Associate εξέταση, η οποία πλέον εξετάζει όχι μόνο transformations αλλά architecture, compute, Delta/Unity Catalog, ingestion, orchestration, Spark troubleshooting και deployment concepts.



## Ο συνολικός χάρτης

Έχουμε ήδη κάνει:

</> Plain text

FOUNDATIONS

|

|— Spark execution foundations

|— partitions / shuffle

|— joins / physical plans

|— batch / streaming / incremental

|— CDC

|— schema evolution

|— medallion architecture

|

|— Databricks fundamentals

|— account / workspace

|— control vs compute plane

|— serverless vs classic

|— all-purpose / jobs / pipelines

|— SQL warehouses

|— DBR / runtime

|— Photon

|— driver / workers

|— pools

|— autoscaling

|— compute access

Τώρα θα κινηθούμε σε **10 μεγάλες φάσεις.**

## Φάση 1 — Compute mastery

Εδώ σταματήσαμε, αλλά δεν έχουμε τελειώσει.

Θέλω να μπορείς να κοιτάξεις ένα πραγματικό cluster configuration και να καταλαβαίνεις **κάθε field**, όχι απλώς τα ονόματα.

Θα μάθουμε:

</> Plain text



#### Classic compute internals

- |
- |— driver
- |— executors
- |— workers
- |— cores
- |— memory
- |— executor memory
- |— task slots
- |— partitions
- |— worker types
- |— driver types
- |— autoscaling
- |— pools
- |— Spot / on-demand
- |— policies
- |— access modes
- |— DBR
- |— Photon
- |— termination

και μετά:

#### </> Plain text

##### Serverless compute

- |
- |— τι κρύβει από εσένα
- |— τι μπορείς ακόμη να configure
- |— environment/dependencies
- |— limitations
- |— scaling
- |— serverless jobs
- |— serverless pipelines
- |— debugging

Η σημερινή Databricks χωρίζει επίσημα το compute σε serverless, classic και SQL warehouses· classic compute περιλαμβάνει all-purpose, jobs και Lakeflow pipeline resources. [Databricks Docu...](#) +1

## Το πρακτικό κομμάτι

Θα πάρουμε configuration σαν:

#### </> Plain text

Driver: 16 cores / 64 GB  
Workers: 8 cores / 32 GB  
Workers: min 2, max 10  
DBR: 18 LTS  
Photon: enabled  
Access: Standard

και θα υπολογίσουμε πραγματικά:

#### </> Plain text

```
πόσα cores έχω
πόσα tasks μπορούν να τρέξουν παράλληλα
πού βρίσκεται executor memory
τι γίνεται αν έχω 2 vs 10 workers
τι συμβαίνει σε shuffle
πού γίνεται spill
πότε OOMάρει driver
πότε OOMάρει executor
```

Θα το συνδέσουμε με Spark UI.

Αυτό είναι το **αμέσως επόμενο μάθημα**.

## Φάση 2 — Delta Lake από χρήση, όχι μόνο internals

Έχουμε ήδη καταλάβει:

⟨/⟩ Plain text

```
Parquet
+
_delta_log
+
snapshots
+
MVCC
+
ADD / REMOVE
```

Τώρα θα δουλέψουμε πραγματικά.

Θα μάθεις SQL:

⟨/⟩ SQL

```
CREATE TABLE
INSERT
UPDATE
DELETE
MERGE
TRUNCATE
ALTER TABLE
DESCRIBE DETAIL
DESCRIBE HISTORY

RESTORE
VACUUM
OPTIMIZE
```

και PySpark:

⟨/⟩ Python

```
spark.read.table(...)
```

```
df.write
df.writeTo(...)

saveAsTable(...)
insertInto(...)

DeltaTable.forName(...)
.merge(...)
.update(...)
.delete(...)
```

Θα δούμε με πραγματικά examples:

</> Plain text

```
append
overwrite
dynamic overwrite
idempotent MERGE
CDC
deduplication
late arriving data
schema enforcement
schema evolution
type widening
time travel
```

και μετά physical optimization:

</> Plain text

```
small files
OPTIMIZE
data skipping
statistics
deletion vectors
liquid clustering
predictive optimization
```

## Φάση 3 — Unity Catalog πραγματικά σε βάθος

Αυτό είναι πολύ μεγαλύτερο θέμα από:

</> Plain text

```
catalog.schema.table
```

Θα χτίσουμε από storage προς governance.

</> Plain text

```
Cloud storage
```

```
 ↑
external location
 ↑
storage credential

Unity Catalog
|
├─ metastore
├─ catalog
├─ schema
├─ managed table
├─ external table
├─ volume
├─ view
├─ materialized view
├─ function
└─ model
```

Θα μάθεις πραγματική SQL:

</> SQL

```
CREATE CATALOG
CREATE SCHEMA

CREATE TABLE

CREATE VOLUME

GRANT
REVOKE
SHOW GRANTS
```

π.χ.:

</> SQL

```
GRANT USE CATALOG
ON CATALOG prod
TO `data-engineers`;

GRANT USE SCHEMA
ON SCHEMA prod.silver
TO `data-engineers`;

GRANT SELECT
ON TABLE prod.silver.orders
TO `analysts`;
```

Και θα κάνουμε **permission debugging**:

</> Plain text

```
PERMISSION_DENIED
 ↓
Who am I?
```

```
↓
What principal is the job using?
↓
USE CATALOG?
↓
USE SCHEMA?
↓
SELECT/MODIFY?
↓
ownership?
↓
storage credential?
```

Θα καλύψουμε επίσης:

#### </> Plain text

```
users
groups
service principals

ownership
privilege inheritance
least privilege

row filters
column masks
ABAC

lineage
audit/governance

Delta Sharing
Lakehouse Federation
```

## Φάση 4 — Data ingestion

Εδώ θα κάνουμε πραγματικό ingestion.

Θα ξεκινήσουμε από το απλό:

#### </> Python

```
df = (
 spark.read
 .format("csv")
 .option("header", True)
 .schema(schema)
 .load(path)
)
```

και:

Python

```
spark.read.json(...)
spark.read.parquet(...)
```

με:

Plain text

```
explicit schema
vs
schema inference
```

Μετά:

SQL

```
COPY INTO ...
```

και τέλος **Auto Loader**, πολύ βαθιά.

Python

```
(
 spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("cloudFiles.schemaLocation", schema_path)
 .load(source_path)
)
```

Θα εξηγήσουμε:

Plain text

```
cloudFiles
file discovery
directory listing
file notification
checkpoint
schemaLocation
schema inference
schema hints
schema evolution
rescued data column
exactly-once-ish ingestion semantics
```

και θα σπάμε pipelines επίτηδες για debugging.

## Φάση 5 — Structured Streaming

Εδώ θα επιστρέψουμε στο Spark αλλά μέσα στο Databricks.

Θα μάθεις πραγματική syntax:

</> Python

```
spark.readStream
```

και:

</> Python

```
.writeStream
```

όπως:

</> Python

```
query = (
 events
 .writeStream
 .format("delta")
 .option("checkpointLocation", checkpoint)
 .trigger(processingTime="30 seconds")
 .toTable("bronze.events")
)
```

Θα μάθεις:

</> Plain text

```
microbatch
checkpoint
offset
state
watermark
event time
processing time
late data

append
update
complete output modes

stateless
vs
stateful transformations
```

και πολύ σημαντικά:

</> Plain text

```
why exactly-once is subtle
```

και πού μπορείς παρ' όλα αυτά να δημιουργήσεις duplicates.

## Φάση 6 — Lakeflow Spark Declarative Pipelines

Αυτό θα είναι μεγάλη ενότητα.

Επειδή εδώ η Databricks έχει αλλάξει ορολογία αρκετά:

Plain text

```
Delta Live Tables
 ↓
Lakeflow Declarative Pipelines
 ↓
Spark Declarative Pipelines
```

θα σου μάθω κυρίως τη **σημερινή μορφή**, αλλά θα αναγνωρίζεις και legacy tutorials.

Θα δούμε πρώτα πραγματικό pipeline source code, όχι μόνο diagrams.

SQL-style:

SQL

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS
SELECT *
FROM STREAM read_files(...);
```

και Python declarative syntax όπου εφαρμόζεται.

Θα μάθουμε:

Plain text

```
streaming tables
materialized views
temporary views
dependencies
DAG generation
incremental computation
expectations
data quality

AUTO CDC
SCD Type 1
SCD Type 2

serverless pipeline compute
classic pipeline compute

pipeline parameters
event log
pipeline monitoring
pipeline debugging
```

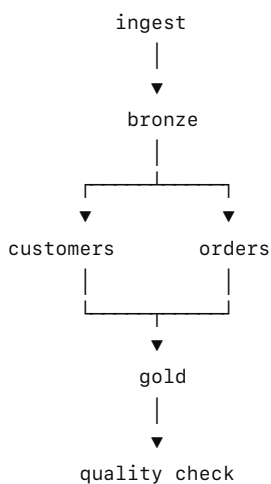


## Φάση 7 — Lakeflow Jobs / orchestration

Εδώ μαθαίνουμε πώς τα προηγούμενα μπαίνουν σε production workflow.

Ένα πραγματικό DAG:

Plain text



Θα μάθεις UI και declarative configuration.

Concepts:

Plain text

```
job
task
run
task dependency
parameters
dynamic value references

retries
timeouts
notifications

if/else
for-each
run-job task

schedules
continuous triggers
file-arrival triggers
table-update triggers
```

και compute:

**</> Plain text**

```
serverless job compute
classic job cluster

new job cluster
shared job cluster
existing all-purpose compute
```

Θα κάνουμε debugging:

**</> Plain text**

```
task failed
 ↓
upstream?
compute?
runtime?
library?
permissions?
parameter?
data?
Spark execution?
```

## Φάση 8 — Databricks SQL

Δεν θα το αφήσουμε ως:

«SQL warehouse για BI».

Θα μάθεις:

**</> Plain text**

```
SQL Editor
queries
query history
query profile

warehouses
serverless / pro / classic
size
min/max clusters
autoscaling
auto stop

AI/BI dashboards
alerts

materialized views
streaming tables

statement execution

JDBC/ODBC
Power BI / Tableau connectivity
```

και performance tuning.

## Φάση 9 — Performance & debugging

Αυτή θα είναι από τις σημαντικότερες ενότητες για συνέντευξη και πραγματική εργασία.

Θα πάρουμε πραγματικά symptom-driven cases.

### Case:

</> Plain text

Job used to take 8 min.  
Now takes 42 min.

Δεν θα πούμε:

μεγαλύτερο cluster.

Θα μάθουμε process:

</> Plain text

Query slow  
↓  
Spark UI  
↓  
Find slow stage  
↓  
Find slow tasks  
↓  
Compare distribution  
↓  
skew?  
shuffle?  
spill?  
I/O?  
join?  
GC?  
small files?

Θα διαβάζουμε:

</> Plain text

Jobs  
Stages  
Tasks  
  
Input size  
Output size  
Shuffle read

```
Shuffle write
Spill memory
Spill disk
GC time
task duration
records
```

και physical plans:

</> Plain text

```
Exchange
BroadcastHashJoin
SortMergeJoin
HashAggregate
Scan
Filter
WholeStageCodegen
Photon operators
```

Θα καλύψουμε:

</> Plain text

```
data skew
salting
broadcasting
AQE
partition tuning
repartition
coalesce

small file problem
file compaction
data skipping
Liquid Clustering

driver OOM
executor OOM

GC pressure
spill

CPU-bound
memory-bound
I/O-bound
```

## Φάση 10 — Production engineering / CLI / CI-CD

Δεν θέλω να καταλήξεις developer που ξέρει μόνο να πατάει:

</> Plain text

```
Run All
```

στο workspace.

Θα μάθεις:

⟨/⟩ Plain text

```
Databricks CLI
Git integration
Git folders
```

```
workspace files
Python modules
wheels
dependencies
```

```
Databricks Connect
IDE workflow
```

και μετά:

### Declarative Automation Bundles

Αυτό είναι το σημερινό όνομα του παλιού **Databricks Asset Bundles**. Η Databricks τα χρησιμοποιεί για να περιγράφεις jobs, pipelines, source files, deployment targets και άλλα Databricks resources ως version-controlled source/configuration, ώστε να μπορείς να κάνεις validation και deployments μέσω CI/CD. [Databricks Docu...](#) +1

Θα γράψουμε πραγματικό:

⟨/⟩ YAML

```
bundle:
 name: orders_project

resources:
 jobs:
 orders_job:
 name: orders-job
 tasks:
 ...
```

και:

⟨/⟩ YAML

```
targets:
 dev:
 mode: development

 prod:
 mode: production
```

Το βασικό bundle file είναι σήμερα το:

⟨/⟩ Plain text

```
databricks.yml
```

και είναι YAML configuration. [Databricks Docu...](#)

Μετά πραγματικά commands:

```
</> Bash
```

```
databricks bundle validate
```

```
databricks bundle deploy -t dev
```

```
databricks bundle run orders_job -t dev
```

Η σημερινή Databricks παρέχει επίσημα workflows για programmatic job και pipeline deployment μέσω Bundles.

[Databricks Docu...](#) +2

## Και θα έχουμε ένα project που μεγαλώνει μαζί μας

Αν κάνουμε ξεχωριστό το example σε κάθε lesson, θα χάσεις τη συνολική εικόνα.

Θέλω επομένως να χτίζουμε παράλληλα ένα **production-style project**.

Π.χ.:

```
</> Plain text
```

```
E-COMMERCE LAKEHOUSE
```

```
sources/
```

```
|
```

```
├─ customers CDC
```

```
├─ orders JSON
```

```
├─ products CSV
```

```
└─ events stream
```

```
↓
```

```
BRONZE
```

```
├─ customers_raw
```

```
├─ orders_raw
```

```
├─ products_raw
```

```
└─ events_raw
```

```
↓
```

```
SILVER
```

```
├─ customers
```

```
├─ orders
```

```
├─ products
```

```
└─ events
```

```
↓
```

Στην πορεία θα προσθέτουμε:

⟨/⟩ Plain text

Auto Loader

CDC

MERGE

Lakeflow pipeline

data quality

Unity Catalog security

Jobs

SQL dashboard

performance tuning

CI/CD

service principal

dev / prod environments

Στο τέλος δεν θα έχεις δει απλώς Databricks concepts.

Θα ξέρεις πώς ενώνονται σε **ένα production system**.

## Πώς θα είναι κάθε lesson

Για να μη γυρίσουμε στο πρόβλημα που είχες όταν διάβαζες slides και έβλεπες scattered πληροφορία, κάθε ενότητα θα ακολουθεί περίπου την ίδια διαδρομή:

⟨/⟩ Plain text

1. Ποιο πρόβλημα έχουμε;
2. Πώς θα το λύναμε χωρίς Databricks;
3. Ποιο Databricks component λύνει αυτό το πρόβλημα;
4. Πώς δουλεύει internally;
5. Πού βρίσκεται στην αρχιτεκτονική;
6. Πώς το χρησιμοποιώ στο UI;
7. Ποια είναι η πραγματική syntax;
8. Ποιο είναι production example;
9. Ποιες είναι οι alternatives;

10. Πότε ΔΕΝ πρέπει να το χρησιμοποιήσω;
11. Best practices
12. Common mistakes
13. Debugging

Και θέλω να επιμείνω ιδιαίτερα στα **6–8**, επειδή σωστά μου επισήμανες προηγουμένως ότι είναι άχρηστο να έχουμε κάνει δύο lessons για ένα εργαλείο και να μην έχεις δει ούτε ένα πραγματικό configuration ή source file.

## Τι θα μάθεις ως syntax

Δεν θα περιοριστούμε σε PySpark.

Θα καλύψουμε τέσσερα interfaces παράλληλα:

### </> Plain text

#### SQL

- |
- ├─ CREATE
- ├─ ALTER
- ├─ MERGE
- ├─ COPY INTO
- ├─ GRANT
- └─ ...

#### PySpark

- |
- ├─ DataFrame API
- ├─ read/write
- ├─ streaming
- └─ Delta APIs

#### Platform / config

- |
- ├─ Jobs
- ├─ Pipelines
- ├─ Bundles YAML
- └─ compute configuration

#### CLI / external development

- |
- ├─ databricks ...

Και όπου υπάρχει UI, θα σου εξηγήω και:

«Πού πατάω στο workspace και σε τι Databricks object αντιστοιχεί αυτό που βλέπω.»

## Τι δεν θα κάνουμε

Δεν θα ακολουθήσουμε blindly τη σειρά των menus:



Plain text

Workspace  
Catalog  
Compute  
Jobs  
SQL  
...

γιατί αυτή είναι καλή σειρά για product tour, όχι για μάθηση.

Θα ακολουθήσουμε **dependency order**:

Plain text

Compute  
↓  
Delta  
↓  
Unity Catalog  
↓  
Ingestion  
↓  
Streaming  
↓  
Declarative Pipelines  
↓  
Jobs  
↓  
SQL analytics  
↓  
Performance/debugging  
↓  
Production CI/CD

Κάθε επόμενο layer χρειάζεται το προηγούμενο.

## Και η εξέταση;

Θα έχουμε δύο διαφορετικά είδη ερωτήσεων.

### Certification-style

π.χ.:

You need to run a scheduled PySpark workload without managing infrastructure. Which compute should you select?

και θα μάθεις:

Plain text

Serverless Jobs

### Engineering-style

Πιο σημαντικό για δουλειά:

A 2 TB ETL job suddenly went from 12 to 45 minutes. One stage has 1,000 tasks; 997 finish in 20 seconds and 3 run for 15 minutes while spilling 80 GB each. What is happening and how do you investigate it?

Εδώ πρέπει να αναγνωρίζεις:

</> Plain text

likely skew

και μετά να ξέρεις **γιατί**, πώς το αποδεικνύεις στο Spark UI και τι fixes υπάρχουν.

Αυτή η δεύτερη κατηγορία είναι που θα σε κάνει πραγματικά καλό.

## Πού ακριβώς συνεχίζουμε τώρα

Δεν θα πάμε ακόμη Unity Catalog.

Το αμέσως επόμενο lesson θα είναι:

### Compute Part 3 — From VM to Spark Task

Θα πάρουμε:

</> Plain text

```
Driver
4 workers
8 cores / worker
32 GB RAM / worker
DBR 18 LTS
Photon enabled
autoscale 2-8
```

και θα απαντήσουμε από first principles:

</> Plain text

```
Τι process τρέχει στον driver;

Τι process τρέχει στον worker;

Τι ακριβώς είναι executor;

Έχω έναν executor ή πολλούς ανά worker;

Τι είναι executor core;

Πώς συνδέεται ένα core με ένα Spark task;

Με 4 × 8 cores πόσα tasks μπορούν
περίπου να τρέξουν ταυτόχρονα;
```

Τι γίνεται αν έχω 1,000 partitions;

Τι γίνεται αν έχω μόνο 4 partitions;

Πού βρίσκεται executor heap;

Τι είναι memory overhead;

Τι γίνεται σε shuffle;

και μετά θα ανοίξουμε νοητικά το **Spark UI Executor/Stage/Task view** πάνω σε πραγματικό example.

Αυτό είναι το επόμενο βήμα που θα ξεκινήσουμε.

[Export the phased plan to a file](#)

Analyzed ▾

</> Python

```
from pathlib import Path

content = """# Databricks Mastery Roadmap

A phased learning plan for mastering Databricks for:

- Technical interviews
- Databricks certifications
- Real-world Data Engineering projects
- Production debugging
- Platform operations
- CI/CD and deployment

Already Covered

Foundations

- Spark execution foundations
- Partitions and shuffle
- Joins and physical plans
- Batch vs streaming vs incremental processing
- CDC
- Schema evolution
- Medallion architecture

Databricks Fundamentals

- Account vs workspace
- Control plane vs compute plane
- Serverless vs classic
- All-purpose vs jobs vs pipelines
- SQL warehouses
- Databricks Runtime (DBR)
- Photon
- Driver and workers
- Pools
- Autoscaling
```

- Compute access

---

## # Phase 1 – Compute Mastery

Goal: understand Databricks compute deeply enough to configure, reason about, troubleshoot, and optimize real Spark workloads

### ## Classic Compute Internals

- Driver
- Executors
- Workers
- CPU cores
- Memory
- Executor memory
- Task slots
- Partitions
- Worker types
- Driver types
- Autoscaling
- Pools
- Spot vs on-demand
- Compute policies
- Access modes
- Databricks Runtime
- Photon
- Termination and auto-termination

### ## Serverless Compute

- What serverless abstracts away
- What remains configurable
- Serverless environments and dependencies
- Limitations
- Scaling
- Serverless notebooks
- Serverless jobs
- Serverless pipelines
- Debugging serverless workloads

### ## Practical Work

Given configurations such as:

```
```text
Driver: 16 cores / 64 GB
Workers: 8 cores / 32 GB
Workers: min 2, max 10
DBR: 18 LTS
Photon: enabled
Access mode: Standard
```
```

We will calculate and reason about:

- Total available cores
- Approximate task parallelism
- Executor memory
- Effect of 2 vs 10 workers

- Shuffle behavior
- Spill
- Driver OOM
- Executor OOM
- Spark UI interpretation

---

## # Phase 2 – Delta Lake in Practice

Goal: move from understanding Delta internals to using Delta correctly in production.

### ## SQL Syntax

- `CREATE TABLE`
- `INSERT`
- `UPDATE`
- `DELETE`
- `MERGE`
- `TRUNCATE`
- `ALTER TABLE`
- `DESCRIBE DETAIL`
- `DESCRIBE HISTORY`
- `RESTORE`
- `VACUUM`
- `OPTIMIZE`

### ## PySpark / Delta APIs

- `spark.read.table(...)`
- `df.write`
- `df.writeTo(...)`
- `saveAsTable(...)`
- `insertInto(...)`
- `DeltaTable.forName(...)`
- `.merge(...)`
- `.update(...)`
- `.delete(...)`

### ## Real Engineering Topics

- Append vs overwrite
- Dynamic overwrite
- Idempotent `MERGE`
- CDC
- Deduplication
- Late-arriving data
- Schema enforcement
- Schema evolution
- Type widening
- Time travel
- Deletion vectors

### ## Physical Optimization

- Small files
- `OPTIMIZE`
- Data skipping
- File statistics
- Liquid Clustering

- Predictive optimization

---

### # Phase 3 – Unity Catalog Mastery

Goal: understand storage, namespace, governance, identities, and permissions as one coherent system.

#### ## Storage and Namespace Model

```
```text
```

Cloud storage

↑

External location

↑

Storage credential

Unity Catalog

- └─ Metastore

- └─ Catalog

- └─ Schema

- └─ Managed table

- └─ External table

- └─ Volume

- └─ View

- └─ Materialized view

- └─ Function

- └─ Model

```
```
```

#### ## SQL Syntax

- `CREATE CATALOG`

- `CREATE SCHEMA`

- `CREATE TABLE`

- `CREATE VOLUME`

- `GRANT`

- `REVOKE`

- `SHOW GRANTS`

Example:

```
```sql
```

GRANT USE CATALOG

ON CATALOG prod

TO `data-engineers`;

GRANT USE SCHEMA

ON SCHEMA prod.silver

TO `data-engineers`;

GRANT SELECT

ON TABLE prod.silver.orders

TO `analysts`;

```
```
```

#### ## Governance Topics

- Users

- Groups

- Service principals
- Ownership
- Privilege inheritance
- Least privilege
- Row filters
- Column masks
- ABAC
- Lineage
- Audit/governance
- Delta Sharing
- Lakehouse Federation

### ## Permission Debugging

```

```text
PERMISSION_DENIED
    ↓
Who am I?
    ↓
What principal is the job using?
    ↓
USE CATALOG?
    ↓
USE SCHEMA?
    ↓
SELECT / MODIFY?
    ↓
Ownership?
    ↓
Storage credential?
```

```

---

### # Phase 4 – Data Ingestion

Goal: ingest files and external data sources correctly, incrementally, and reproducibly.

### ## Core Reads

```

```python
df = (
    spark.read
    .format("csv")
    .option("header", True)
    .schema(schema)
    .load(path)
)
```

```

Also:

- `spark.read.json(...)`
- `spark.read.parquet(...)`

### ## Topics

- Explicit schema vs schema inference
- `COPY INTO`
- Auto Loader

- `cloudFiles`
- File discovery
- Directory listing
- File notification
- Checkpoints
- `schemaLocation`
- Schema inference
- Schema hints
- Schema evolution
- Rescued data column
- Duplicate prevention
- Incremental ingestion semantics

### ## Practical Work

- Build ingestion pipelines
- Intentionally break them
- Diagnose missing files
- Diagnose duplicate files
- Diagnose schema drift
- Diagnose malformed rows

---

### # Phase 5 – Structured Streaming

Goal: understand streaming from execution model to production reliability.

### ## Core Syntax

- `spark.readStream`
- `writeStream`

Example:

```
```python
query = (
    events
    .writeStream
    .format("delta")
    .option("checkpointLocation", checkpoint)
    .trigger(processingTime="30 seconds")
    .toTable("bronze.events")
)
```
```

### ## Topics

- Micro-batches
- Checkpoints
- Offsets
- State
- Watermarks
- Event time
- Processing time
- Late data
- Append mode
- Update mode
- Complete mode
- Stateless transformations



- Stateful transformations
- Exactly-once semantics
- Duplicate scenarios
- Restart behavior
- State-store debugging

---

## # Phase 6 – Lakeflow Spark Declarative Pipelines

Goal: build production declarative pipelines with current Databricks syntax and concepts.

### ## Terminology Evolution

```
```text
Delta Live Tables
      ↓
Lakeflow Declarative Pipelines
      ↓
Spark Declarative Pipelines
```
```

The focus will be on current syntax while recognizing older terminology.

### ## SQL-Style Example

```
```sql
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS
SELECT *
FROM STREAM read_files(...);
```
```

### ## Topics

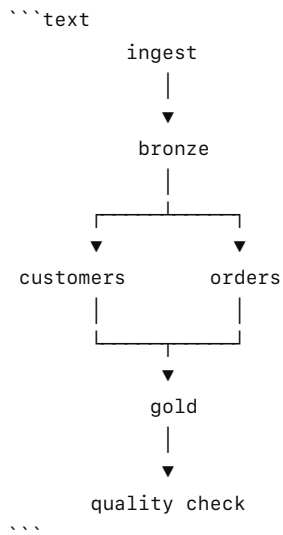
- Streaming tables
- Materialized views
- Temporary views
- Dependency graphs
- DAG generation
- Incremental computation
- Expectations
- Data quality
- `AUTO CDC`
- SCD Type 1
- SCD Type 2
- Serverless pipeline compute
- Classic pipeline compute
- Pipeline parameters
- Event log
- Pipeline monitoring
- Pipeline debugging

---

## # Phase 7 – Lakeflow Jobs and Orchestration

Goal: orchestrate production workflows reliably.

### ## Example DAG



### ## Topics

- Job
- Task
- Run
- Dependencies
- Parameters
- Dynamic value references
- Retries
- Timeouts
- Notifications
- If/else
- For-each
- Run-job task
- Schedules
- Continuous triggers
- File-arrival triggers
- Table-update triggers

### ## Compute Choices

- Serverless job compute
- Classic job compute
- New job cluster
- Shared job cluster
- Existing all-purpose compute

### ## Debugging Model

```

``text
Task failed
 ↓
Upstream?
Compute?
Runtime?
Library?
Permissions?
Parameter?
Data?
Spark execution?
...

```

```

Phase 8 – Databricks SQL

Goal: master SQL analytics and BI compute rather than treating SQL warehouses as a black box.

Topics

- SQL Editor
- Queries
- Query history
- Query profile
- SQL warehouses
- Serverless / Pro / Classic
- Warehouse size
- Min/max clusters
- Autoscaling
- Auto-stop
- AI/BI dashboards
- Alerts
- Materialized views
- Streaming tables
- Statement Execution API
- JDBC / ODBC
- Power BI connectivity
- Tableau connectivity
- SQL performance tuning

Phase 9 – Performance and Debugging

Goal: diagnose real production slowdowns using evidence instead of blindly scaling compute.

Example Scenario

```text
Job used to take 8 min.
Now takes 42 min.
```

Investigation:

```text
Query slow
  ↓
Spark UI
  ↓
Find slow stage
  ↓
Find slow tasks
  ↓
Compare task distribution
  ↓
Skew?
Shuffle?
Spill?
I/O?
Join?
```

```
GC?
Small files?
...

## Spark UI Metrics

- Jobs
- Stages
- Tasks
- Input size
- Output size
- Shuffle read
- Shuffle write
- Memory spill
- Disk spill
- GC time
- Task duration
- Records

## Physical Plan Operators

- `Exchange`
- `BroadcastHashJoin`
- `SortMergeJoin`
- `HashAggregate`
- `Scan`
- `Filter`
- Whole-stage code generation
- Photon operators

## Performance Topics

- Data skew
- Salting
- Broadcasting
- Adaptive Query Execution
- Partition tuning
- `repartition`
- `coalesce`
- Small file problem
- File compaction
- Data skipping
- Liquid Clustering
- Driver OOM
- Executor OOM
- GC pressure
- Spill
- CPU-bound workloads
- Memory-bound workloads
- I/O-bound workloads

---

# Phase 10 – Production Engineering, CLI, and CI/CD

Goal: move beyond notebooks and operate Databricks as a production software platform.

## Developer Workflow

- Databricks CLI
```

- Git integration
- Git folders
- Workspace files
- Python modules
- Python wheels
- Dependency management
- Databricks Connect
- IDE workflows

Declarative Automation Bundles

Formerly called Databricks Asset Bundles.

Example:

```
```yaml
bundle:
 name: orders_project

resources:
 jobs:
 orders_job:
 name: orders-job
 tasks:
 ...
...
```
```

Targets:

```
```yaml
targets:
 dev:
 mode: development

 prod:
 mode: production
...
```
```

Primary file:

```
```text
databricks.yml
...
```
```

Commands:

```
```bash
databricks bundle validate

databricks bundle deploy -t dev

databricks bundle run orders_job -t dev
...
```
```

CI/CD Topics

- Git branching
- Validation
- Testing
- Environment-specific configuration

- Dev/test/prod
- Service principals
- Deployment identities
- Reproducible deployments
- Rollbacks
- Secrets/configuration
- Production release workflow

Ongoing Production-Style Project

We will build one project progressively instead of unrelated toy examples.

E-Commerce Lakehouse

```
```text
sources/
├─ customers CDC
├─ orders JSON
├─ products CSV
└─ events stream
```

↓

### BRONZE

```
├─ customers_raw
├─ orders_raw
├─ products_raw
└─ events_raw
```

↓

### SILVER

```
├─ customers
├─ orders
├─ products
└─ events
```

↓

### GOLD

```
├─ daily_revenue
├─ customer_360
├─ product_performance
└─ conversion_metrics
```
```

Over the course we will add:

- Auto Loader
- CDC
- `MERGE`
- Lakeflow pipelines
- Data-quality checks
- Unity Catalog security
- Jobs
- SQL dashboards
- Performance tuning
- CI/CD

- Service principals
- Dev/prod environments

Standard Structure of Every Lesson

Each lesson should answer:

1. What problem are we solving?
2. How would we solve it without Databricks?
3. Which Databricks component solves it?
4. How does it work internally?
5. Where does it sit in the architecture?
6. How do I use it in the UI?
7. What is the real syntax?
8. What does a production example look like?
9. What alternatives exist?
10. When should I not use it?
11. Best practices
12. Common mistakes
13. Debugging workflow
14. Certification traps
15. Exercises
16. Full solutions
17. Mini-project extension

Interfaces We Will Learn in Parallel

SQL

- `CREATE`
- `ALTER`
- `MERGE`
- `COPY INTO`
- `GRANT`
- `REVOKE`
- Delta DML
- Unity Catalog SQL

PySpark

- DataFrame API
- Read/write APIs
- Streaming APIs
- Delta APIs

Platform Configuration

- Compute
- Jobs
- Pipelines
- Bundles YAML

CLI / External Development

- `databricks ...`
- Git

- Databricks Connect
- Deployment workflows

Learning Order

The curriculum follows dependency order rather than the Databricks menu layout:

```
```text
Compute
 ↓
Delta Lake
 ↓
Unity Catalog
 ↓
Ingestion
 ↓
Structured Streaming
 ↓
Declarative Pipelines
 ↓
Jobs
 ↓
Databricks SQL
 ↓
Performance and Debugging
 ↓
Production CI/CD
```
```

Assessment Strategy

Two kinds of questions will be used throughout.

Certification-Style

Example:

> You need to run a scheduled PySpark workload without managing infrastructure. Which compute should you use?

Expected reasoning:

```
```text
Serverless Jobs
```
```

Engineering-Style

Example:

> A 2 TB ETL job increased from 12 minutes to 45 minutes. A stage has 1,000 tasks; 997 finish in 20 seconds, while 3 take

Expected reasoning:

```
```text
Likely data skew
```
```


Then verify with:

- Spark UI
- Task-duration distribution
- Shuffle read/write
- Spill
- Partition sizes
- Join/grouping keys
- Physical plan

Immediate Next Lesson

Compute Part 3 – From VM to Spark Task

Starting configuration:

```
```text
Driver
4 workers
8 cores / worker
32 GB RAM / worker
DBR 18 LTS
Photon enabled
Autoscale 2-8
```
```

Topics:

- What process runs on the driver?
- What runs on a worker?
- What exactly is an executor?
- How many executors exist per worker?
- What is an executor core?
- How does a core relate to a Spark task?
- With 4×8 cores, how many tasks can run concurrently?
- What happens with 1,000 partitions?
- What happens with only 4 partitions?
- Where is the executor heap?
- What is memory overhead?
- What happens during shuffle?
- What is written to local disk?
- Where does spill occur?
- What changes when autoscaling from 2 to 8 workers?
- What changes with Photon?
- Which of these details disappear from user management in serverless?
- How do these concepts appear in the Spark UI?

```
path = Path("/mnt/data/databricks_mastery_phased_plan.md")
path.write_text(content, encoding="utf-8")
print(path)
```

STDOUT/STDERR

/mnt/data/databricks_mastery_phased_plan.md

Έτοιμο. Το εξήγαγα σε Markdown ώστε να λειτουργεί καλά ως syllabus/roadmap και σε Obsidian ή Git. </>

databricks_mastery_phased_plan.md
Document

So teach me 1st phase

Worked for 16s

Phase 1 — Compute Mastery

Θα το κάνουμε από το πιο χαμηλό επίπεδο προς το πιο αφηρημένο:

</> Plain text

```
VM
↓
Driver / Worker
↓
Executor
↓
Core
↓
Task
↓
Partition
↓
Stage
↓
Job
↓
Autoscaling / node sizing
↓
Access mode
↓
Pools / Spot / termination
↓
Serverless abstraction
↓
Debugging + Spark UI
```

Ο στόχος είναι στο τέλος να μπορείς να κοιτάξεις ένα compute configuration και να λες:

«Ξέρω πόσα machines έχω, τι τρέχει σε καθένα, περίπου πόσα Spark tasks μπορούν να τρέξουν ταυτόχρονα, πού μπορεί να γίνει OOM, πώς επηρεάζεται το shuffle, τι αλλάζει όταν κάνω autoscale και γιατί θα επέλεγα serverless ή classic.»

Η σημερινή Databricks συνιστά serverless ως default για τα περισσότερα νέα workloads και classic όταν χρειάζεσαι capabilities/configuration που το serverless δεν υποστηρίζει. [Databricks Docu...](#) +1

1. Από το μηδέν: τι αγοράζεις όταν επιλέγεις compute;

Έστω ότι δημιουργείς classic compute:

</> Plain text

```
Driver:
8 cores
```

32 GB RAM

Workers:

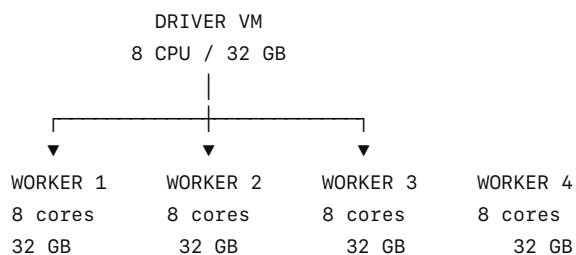
4 ×

8 cores

32 GB RAM

Physically έχεις περίπου:

</> Plain text



Άρα έχεις:

</> Plain text

```
1 driver machine
+
4 worker machines
=
5 cloud VMs
```

Αλλά δεν πρέπει να προσθέσεις τον driver στα distributed worker resources και να πεις:

</> Plain text

```
40 worker cores
```

Οι workers έχουν:

</> Plain text

```
4 × 8 = 32 executor cores
```

και περίπου:

</> Plain text

```
4 × 32 GB = 128 GB worker RAM
```

Ο driver είναι ξεχωριστός coordinator.

2. Τι τρέχει στον Driver;

Ο driver είναι ο εγκέφαλος του Spark application.

Πάνω του βρίσκονται λειτουργίες όπως:

⟨/⟩ Plain text

```
SparkSession
SparkContext
query planning
DAG construction
stage scheduling
task scheduling
tracking executors
collecting task metadata/results
notebook interpreter/session
```

Η Databricks περιγράφει τον driver ως το node που κρατά το state των attached notebooks, διατηρεί το `SparkContext`, ερμηνεύει commands και συντονίζει τους executors. [Databricks Docu...](#) +1

Έστω:

⟨/⟩ Python

```
result = (
    spark.table("sales.orders")
    .groupBy("country")
    .sum("amount")
)

display(result)
```

Ο driver δεν παίρνει εξαρχής όλα τα rows και αρχίζει να κάνει additions.

Πρώτα δημιουργείται κάτι σαν:

⟨/⟩ Plain text

```
Logical Plan

Aggregate
  ↓
Scan sales.orders
```

μετά optimization:

⟨/⟩ Plain text

```
Optimized Plan
```

και τελικά physical plan:

⟨/⟩ Plain text

```
Scan
↓
Partial Aggregate
↓
Exchange
↓
Final Aggregate
```

Ο driver οργανώνει το execution.

Οι workers επεξεργάζονται τα actual partitions.

Driver ≠ worker

Αυτό είναι θεμελιώδες.

⟨/⟩ Plain text

```
DRIVER
=
coordination/control

WORKERS
=
distributed processing
```

Όχι απόλυτα 100% σε κάθε implementation detail, αλλά είναι το σωστό mental model.

Άσκηση 1

Έχεις:

⟨/⟩ Plain text

```
Driver: 4 cores, 16 GB

Workers:
10 × 8 cores, 32 GB
```

Πόσα worker cores υπάρχουν;

Λύση

⟨/⟩ Plain text

```
10 × 8 = 80 worker cores
```

Όχι 84.

Τα 4 του driver δεν είναι worker execution capacity.

3. Τι είναι worker και τι executor;

Σε vanilla Spark μπορεί να έχεις πιο περίπλοκες σχέσεις μεταξύ worker machines και executors.

Στο Databricks classic model όμως υπάρχει μια πολύ βολική απλοποίηση:

Databricks τρέχει έναν Spark executor ανά worker node.

Για αυτό στην Databricks documentation οι όροι `worker` και `executor` χρησιμοποιούνται συχνά σχεδόν εναλλακτικά.

 Databricks Docu... +1

Άρα:

</> Plain text

```
Worker 1
└─ Executor 1

Worker 2
└─ Executor 2

Worker 3
└─ Executor 3
```

Αν έχεις:

</> Plain text

8 workers

έχεις κατά κανόνα:

</> Plain text

8 executors

στο Databricks classic compute.

4. Τι είναι core;

Ένα worker μπορεί π.χ. να έχει:

</> Plain text

8 CPU cores

Spark χρησιμοποιεί τα cores ως basis για task parallelism.

Για απλό Spark reasoning:

περίπου ένα active Spark task ανά available executor core.

Η Databricks αναφέρει 1:1 mapping μεταξύ task slots και available cores. [Databricks Docu...](#)

Άρα worker:

```
</> Plain text
```

```
8 cores
```

μπορεί συνήθως να εκτελεί περίπου:

```
</> Plain text
```

```
8 tasks concurrently
```

5. Τι είναι Task;

Task είναι η μικρότερη βασική execution unit ενός Spark stage.

Έστω DataFrame με:

```
</> Plain text
```

```
100 partitions
```

και κάνεις:

```
</> Python
```

```
df.filter("amount > 10").count()
```

Για ένα stage μπορείς να έχεις περίπου:

```
</> Plain text
```

```
1 partition
```

```
→
```

```
1 task
```

άρα:

```
</> Plain text
```

```
100 partitions
```

```
→
```

```
100 tasks
```

Τώρα έστω έχεις:

</> Plain text

```
4 workers
×
8 cores
=
32 task slots
```

Μπορούν να τρέξουν περίπου:

</> Plain text

```
32 tasks ταυτόχρονα
```

Άρα τα 100 tasks τρέχουν σε «κύματα»:

</> Plain text

```
Wave 1 → 32
Wave 2 → 32
Wave 3 → 32
Wave 4 → 4
```

Conceptually.

6. Το partition καθορίζει πόσα parallel pieces of work υπάρχουν

Αυτό συνδέεται άμεσα με όσα μάθαμε στο Spark.

Έστω:

</> Plain text

```
32 available cores
```

Dataset με 4 partitions

</> Plain text

```
P1 P2 P3 P4
```

Τρέχουν μόνο:

</> Plain text

```
4 tasks
```


Άρα περίπου:

```
</> Plain text
```

```
4 cores busy
28 cores idle
```

Έχεις τεράστιο cluster αλλά δεν του δίνεις αρκετή parallel δουλειά.

Dataset με 32 partitions

```
</> Plain text
```

```
32 partitions
```

μπορείς θεωρητικά να γεμίσεις:

```
</> Plain text
```

```
32 slots
```

πολύ καλά.

Dataset με 1,000 partitions

Δεν είναι πρόβλημα απλώς επειδή είναι περισσότερα από τα cores.

Τρέχουν:

```
</> Plain text
```

```
32 at a time
```

μέχρι να τελειώσουν.

Το potential πρόβλημα είναι αν τα partitions είναι υπερβολικά μικρά:

```
</> Plain text
```

```
1,000,000 tiny tasks
```

τότε scheduling/task startup overhead αρχίζει να γίνεται σημαντικό.

7. Παράδειγμα από την αρχή μέχρι το task

Έχεις:

</> Python

```
df = spark.read.table("orders")

result = (
    df
    .groupBy("country")
    .sum("amount")
)
```

Suppose:

</> Plain text

orders input = 64 partitions

Αρχικά:

</> Plain text

Stage 1

64 input partitions

↓

64 tasks

Κάθε task:

</> Plain text

Task 1 → partition 1

Task 2 → partition 2

...

Task 64 → partition 64

Μετά groupBy προκαλεί shuffle.

Suppose Spark αποφασίζει:

</> Plain text

200 shuffle partitions

Τότε downstream stage:

</> Plain text

Stage 2

200 shuffle partitions

↓

200 tasks

Άρα:

partition count είναι stage-specific.

Δεν έχεις έναν αριθμό partitions «για όλο το job».

8. Με 4 workers × 8 cores τι συμβαίνει;

Έστω:

</> Plain text

```
Workers = 4  
Cores each = 8
```

Total executor cores:

</> Plain text

```
32
```

Stage 1

</> Plain text

```
64 tasks
```

Χοντρικά:

</> Plain text

```
2 waves
```

Stage 2

</> Plain text

```
200 tasks
```

Χοντρικά:

</> Plain text

```
7 waves
```

γιατί:

</> Plain text

```
200 / 32 = 6.25
```

άρα χρειάζεσαι 7 waves.

Αυτό δεν σημαίνει ότι κάθε task παίρνει ακριβώς ίδιο χρόνο· στην πράξη μπορεί να υπάρχει skew.

Άσκηση 2

Compute:

```
</> Plain text
```

```
6 workers
16 cores / worker
```

Stage:

```
</> Plain text
```

```
768 partitions
```

Πόσα task slots περίπου;

Λύση

```
</> Plain text
```

```
6 × 16 = 96
```

Άρα περίπου:

```
</> Plain text
```

```
768 / 96 = 8 waves
```

αν όλα τα tasks έχουν παρόμοια διάρκεια.

9. Τώρα το σημαντικό: ίσα cores ≠ ίση performance

Σκέψου δύο clusters.

Cluster A

```
</> Plain text
```

```
2 workers
16 cores each
128 GB each
```

Totals:

⟨/⟩ Plain text

32 cores
256 GB RAM

Cluster B

⟨/⟩ Plain text

8 workers
4 cores each
32 GB each

Totals:

⟨/⟩ Plain text

32 cores
256 GB RAM

Στα totals είναι παρόμοια.

Η Databricks χρησιμοποιεί ακριβώς αυτό το είδος παραδείγματος στα *sizing best practices*. [Databricks Docu...](#)

Αλλά δεν σημαίνει απαραίτητα ίδια συμπεριφορά.

Γιατί;

10. Scale up vs scale out

Cluster A:

⟨/⟩ Plain text

few large workers

είναι πιο:

⟨/⟩ Plain text

scale up

Cluster B:

⟨/⟩ Plain text

many small workers

είναι πιο:

</> Plain text

```
scale out
```

Αυτό επηρεάζει:

</> Plain text

```
network traffic
shuffle fan-out
executor heap size
per-task memory
scheduling
fault impact
local disk
```

11. Γιατί μεγάλος worker μπορεί να βοηθήσει;

Suppose ένα task χρειάζεται:

</> Plain text

```
10 GB working memory
```

Worker:

</> Plain text

```
4 cores
16 GB total
```

τρέχει 4 concurrent tasks.

Αυτό είναι προβληματικό.

Roughly:

</> Plain text

```
4 × 10 GB
=
40 GB potential working set
```

σε 16 GB worker.

Θα spillάρει ή μπορεί να OOMάρει.

Worker:

Plain text

```
4 cores
64 GB
```

έχει πολύ μεγαλύτερο memory-per-core ratio.

Η Databricks προτείνει memory-optimized workers για heavy shuffle/spill και memory-intensive workloads. [Databricks Docu...](#) +1

12. Memory per core

Αυτό είναι πιο σημαντικό από το να βλέπεις μόνο «πόσα GB».

Compare:

Plain text

```
Worker A:
4 cores
16 GB

= 4 GB/core
```

VS:

Plain text

```
Worker B:
4 cores
32 GB

= 8 GB/core
```

Η Databricks μάλιστα προτείνει, όταν υποψιάζεσαι memory problem, να δοκιμάσεις μεγαλύτερο memory-per-core ratio για να δεις αν αλλάζει η συμπεριφορά. [Databricks Docu...](#)

13. Πού πάει η worker RAM;

Μην φαντάζεσαι:

Plain text

```
32 GB worker
=
32 GB διαθέσιμα εξ ολοκλήρου στα Spark rows
```

Υπάρχουν διαφορετικοί consumers:

Plain text

```
Worker RAM
|
├── JVM heap
│   ├── Spark execution memory
│   ├── cached data
│   ├── JVM objects
│   └── metadata
├── off-heap / native memory
├── Python worker processes
├── Photon/native processing
└── OS / Databricks services
```

Η ακριβής allocation δεν πρέπει να αντιμετωπίζεται σαν απλή:

</> Plain text

RAM / cores

φόρμουλα.

Αλλά για sizing το memory-per-core είναι χρήσιμο heuristic.

14. Execution memory vs storage memory

Spark χρησιμοποιεί memory για δύο βασικές κατηγορίες:

Execution

Operations όπως:

</> Plain text

```
sort
shuffle
hash aggregation
join hash tables
```

Storage

Όπως:

</> Python

```
df.cache()
```

ή persisted blocks.

To Spark έχει unified memory management, οπότε αυτές οι περιοχές μπορούν σε κάποιο βαθμό να μοιράζονται διαθέσιμη managed memory.

Για interview δεν χρειάζεται να θυμάσαι αμέσως κάθε fraction.

Χρειάζεται να καταλάβεις:

`join/sort/groupBy` μπορούν να φάνε execution memory ακόμη κι αν δεν έχεις κάνει ποτέ `cache()`.

15. Τι είναι spill;

Έστω task κάνει:

</> Plain text

Sort 20 GB

αλλά δεν μπορεί να κρατήσει όλα τα intermediate δεδομένα στη RAM.

Τότε:

</> Plain text

```
RAM insufficient
  ↓
write intermediate data
to local worker disk
  ↓
continue processing
```

Αυτό είναι **spill**.

Υπάρχουν metrics:

</> Plain text

Memory Bytes Spilled
Disk Bytes Spilled

στο Spark UI.

16. Spill ≠ failure

Αυτό είναι σημαντικό.

</> Plain text

spill

δεν σημαίνει απαραίτητα:

⟨/⟩ Plain text

```
job broken
```

Spark είναι σχεδιασμένο ώστε operations να μπορούν να spillάρουν.

Αλλά:

⟨/⟩ Plain text

```
huge repeated spill
```

μπορεί να σημαίνει:

⟨/⟩ Plain text

```
insufficient memory
too-large partitions
skew
bad join
too few shuffle partitions
```

και να κάνει το workload πολύ αργό.

Η Databricks αναφέρει memory-intensive shuffle/spill workloads ως use case για memory-optimized workers. [Databricks Docu...](#)

17. Πού γράφεται το shuffle;

Έστω:

⟨/⟩ Python

```
df.groupBy("country").sum("amount")
```

Map-side tasks παράγουν shuffle output.

Conceptually:

⟨/⟩ Plain text

```
Worker 1 local disk
  shuffle files
```

```
Worker 2 local disk
  shuffle files
```

```
Worker 3 local disk
  shuffle files
```

Downstream tasks τα τραβούν μέσω network.

⟨⟩ Plain text

```
Worker A
  ↓ shuffle blocks
network
  ↓
Worker B
```

Άρα heavy shuffle πιέζει:

⟨⟩ Plain text

```
CPU
memory
network
local disk
```

Τα local disks των executors χρησιμοποιούνται μεταξύ άλλων για shuffle spill και caching. [Databricks Docu...](#)

18. Γιατί worker local disk δεν είναι persistent storage;

Αν σταματήσει το cluster:

⟨⟩ Plain text

```
worker VM destroyed
```

τότε local:

⟨⟩ Plain text

```
shuffle files
cached partitions
temp files
spill files
```

μπορούν να χαθούν.

Δεν είναι:

⟨⟩ Plain text

```
Delta table storage
```

Το Delta table είναι σε persistent cloud storage.

Άρα:

⟨⟩ Plain text

```
local worker disk
```

```
=  
temporary compute storage
```

19. Driver OOM vs Executor OOM

Πρέπει να μάθεις να τα ξεχωρίζεις.

Driver OOM

Συνηθισμένα αίτια:

```
</> Python  
  
df.collect()  
df.toPandas()
```

για πολύ μεγάλα data.

Ή:

```
</> Plain text  
  
huge result  
huge query plan  
too many task metadata  
large local Python structures
```

Example:

```
</> Plain text  
  
Workers:  
20 × 64 GB  
  
Driver:  
8 GB
```

και:

```
</> Python  
  
rows = df.collect()
```

100 GB result.

Workers μπορεί να είναι μια χαρά.

O driver καταρρέει.

Executor OOM

Συμβαίνει μέσα σε worker/executor.

Πιθανές αιτίες:

⟨/⟩ Plain text

```
skewed partition
large hash join
huge window partition
too few shuffle partitions
large UDF memory
stateful streaming state
```

Η Databricks αναφέρει skew, μεγάλες broadcasts, UDFs, unpartitioned windows και λίγα shuffle partitions ως συχνές αιτίες memory failures. [Databricks Docu...](#)

20. Debugging memory: μην αυξάνεις αμέσως RAM

Suppose executor dies.

Bad reasoning:

⟨/⟩ Plain text

```
OOM
↓
double cluster
```

Better:

⟨/⟩ Plain text

```
OOM
↓
Which executor/task?
↓
Does one partition dominate?
↓
Skew?
↓
Spill?
↓
Join?
↓
Window?
↓
Too few partitions?
↓
Memory-per-core?
```

Μπορεί το πρόβλημα να είναι:

</> Plain text

```
one 100 GB partition
```

και να έχεις 100 workers.

Δεν βοηθά πολύ αν **ένα task** πρέπει να επεξεργαστεί αυτά τα 100 GB.

21. Autoscaling από first principles

Έστω classic compute:

</> Plain text

Worker:

8 cores

32 GB

Min workers = 2

Max workers = 8

Minimum capacity:

</> Plain text

2×8

=

16 task slots

Maximum:

</> Plain text

8×8

=

64 task slots

Άρα autoscaling μπορεί να αλλάξει περίπου:

</> Plain text

parallel task capacity:

16

↓

24

↓

40

↓

64

ανάλογα με το workload.

Classic compute επιτρέπει explicit minimum και maximum workers. [Databricks Docu...](#)

22. Τι ΔΕΝ αλλάζει όταν κάνει autoscale;

Suppose worker type:

</> Plain text

8 cores
32 GB

Autoscaling:

</> Plain text

2 → 8 workers

To worker size παραμένει:

</> Plain text

8 cores
32 GB each

Δεν γίνεται:

</> Plain text

2 × huge VMs
→
8 × tiny VMs

Αυτό είναι **horizontal autoscaling**.

23. Γιατί autoscaling δεν λύνει κάθε bottleneck;

Suppose:

</> Plain text

Stage has 1 partition.

Έχεις:

</> Plain text

```
2 workers
```

και κάνεις scale:

```
</> Plain text
```

```
→ 100 workers
```

Πόσα tasks έχεις;

```
</> Plain text
```

```
1
```

Άρα περίπου:

```
</> Plain text
```

```
1 core busy  
hundreds idle
```

Autoscaling δεν δημιουργεί magically partitions.

24. Άλλο failure: skew

Stage:

```
</> Plain text
```

```
100 partitions
```

99 partitions:

```
</> Plain text
```

```
100 MB each
```

1 partition:

```
</> Plain text
```

```
200 GB
```

Scale:

```
</> Plain text
```

```
4 workers → 40 workers
```


Τα 99 μικρά τελειώνουν γρήγορα.

Ένα task:

```
</> Plain text
```

```
200 GB
```

μένει.

Άρα job τελειώνει όταν τελειώσει:

```
</> Plain text
```

```
the straggler
```

Το πρόβλημα είναι skew, όχι απλώς insufficient parallel capacity.

Άσκηση 3

Έχεις:

```
</> Plain text
```

```
4 workers  
8 cores each
```

```
Stage:  
30 partitions
```

Αν autoscale σε 8 workers, θα επιταχυνθεί σημαντικά το συγκεκριμένο stage;

Λύση

Αρχικά:

```
</> Plain text
```

```
32 slots
```

για μόνο:

```
</> Plain text
```

```
30 tasks
```

Άρα ήδη όλα μπορούν να τρέξουν μαζί.

Με 8 workers:

```
</> Plain text
```

```
64 slots
```

αλλά έχεις ακόμη μόνο 30 tasks.

Δεν κερδίζεις ουσιαστικά task parallelism.

25. Serverless autoscaling

Τώρα καταλαβαίνουμε τι κρύβει το serverless.

Classic:

```
</> Plain text
```

```
worker type = X
min = 2
max = 8
```

Serverless:

```
</> Plain text
```

```
run workload
```

και Databricks διαχειρίζεται:

```
</> Plain text
```

```
underlying instances
capacity
scaling
runtime
```

Serverless Jobs έχουν automatic scaling και Photon platform-managed. [Databricks Docu...](#) +1

Άρα:

Autoscaling υπάρχει στο serverless, αλλά δεν είναι συνήθως ο manual `min_workers/max_workers` axis που έχεις στο classic.

26. Single-node compute

Classic δεν σημαίνει υποχρεωτικά distributed cluster.

Μπορείς να επιλέξεις:

</> Plain text

Single node

Τότε:

</> Plain text

```
DRIVER
+ Spark local work
```

Δεν υπάρχουν separate worker nodes.

Η Databricks αναφέρει ότι ο driver λειτουργεί και ως worker, και χρησιμοποιείται για μικρά ή μη-distributed workloads.

[Databricks Docu...](#)

Χρήσιμο για:

</> Plain text

```
small development
small datasets
some local ML
non-distributed workloads
```

Όχι για:

</> Plain text

multi-TB Spark ETL

27. Access Mode: Standard

Τώρα μπαίνουμε σε διαφορετικό axis.

Standard access mode:

</> Plain text

```
Classic Compute
|
├─ Nikos
├─ Maria
└─ John
```

πολλοί users μπορούν να χρησιμοποιούν το ίδιο compute με workload isolation.

Η Databricks το προτείνει για τα περισσότερα workloads. [Databricks Docu...](#) +1

Supported:

Plain text

Python
SQL
Scala

R όχι.

28. Dedicated

Dedicated:

Plain text

Classic Compute
↓
specific user
or group

Χρησιμοποιείται όταν χρειάζεσαι capabilities όπως:

Plain text

RDD APIs
R
GPU
ML Runtime
privileged/lower-level machine access

Η σημερινή Databricks υποστηρίζει Dedicated για single user ή, σε supported setups, group assignment. [Databricks Docu...](#) +1

29. Auto access mode

Στο current UI μπορεί να δεις:

Plain text

Access mode: Auto

Αυτό δεν είναι τρίτο semantic access model.

Σημαίνει:

Databricks διάλεξε Standard ή Dedicated ανάλογα με runtime/hardware.

Σήμερα Auto προτιμά Standard, αλλά μεταβαίνει σε Dedicated αν π.χ. διαλέξεις ML Runtime, GPU ή αρκετά παλιό DBR.

[Databricks Docu...](#) +1

30. Πρακτικός κανόνας

Για Data Engineering:

</> Plain text

Standard

by default.

Dedicated μόνο αν πεις:

</> Plain text

Χρειάζομαι συγκεκριμένη capability
που Standard δεν υποστηρίζει.

Όχι επειδή:

dedicated ακούγεται πιο γρήγορο.

Δεν είναι performance tier.

Είναι security/capability mode.

31. Pools — τώρα με πλήρες mental model

Χωρίς pool:

</> Plain text

```
Start cluster
↓
Ask cloud provider for VMs
↓
Wait for VM provisioning
↓
Prepare runtime
↓
Spark starts
```

Pool:

</> Plain text

pre-warmed VM capacity

Conceptually:

</> Plain text

```
POOL
├─ idle VM
├─ idle VM
├─ idle VM
└─ used VM
```

Όταν cluster ζητά node:

</> Plain text

```
cluster
↓
pool
↓
warm instance available
↓
attach quickly
```

Pools έχουν parameters όπως:

</> Plain text

```
minimum idle instances
maximum capacity
idle instance auto termination
```

στο AWS implementation. [Databricks Docu...](#)

32. Pool vs cluster

Αυτό είναι συχνό μπέρδεμα.

Pool:

δεξαμενή infrastructure instances.

Compute:

Spark execution environment.

Ένα pool μπορεί να εξυπηρετεί περισσότερα από ένα computes.

</> Plain text

```
      POOL
      │
      ├───┘
      │   │   │
      ▼   ▼   ▼
Cluster A Cluster B Cluster C
```

33. Pool vs autoscaling

Suppose cluster wants:

</> Plain text

2 → 6 workers

Autoscaler αποφασίζει:

</> Plain text

I need +4

Pool απαντά:

</> Plain text

Here are 4 warm VMs

Άρα:

</> Plain text

Autoscaler:
HOW MANY?

Pool:
WHERE DO THE VMs COME FROM?

34. Serverless vs pools

Serverless κάνει το πρόβλημα των pools ουσιαστικά platform concern.

Γι' αυτό Databricks σήμερα προτείνει serverless αντί pools όπου το workload είναι compatible. [Databricks Docu...](#)

Serverless δεν υποστηρίζει user-configured instance pools. [Databricks Docu...](#)

35. Spot instances

Classic compute μπορεί να χρησιμοποιεί cheaper spot/preemptible-like worker capacity ανά cloud.

Idea:

</> Plain text

On-demand VM
=
provider promises normal availability

```
Spot
=
cheap spare capacity
but can be reclaimed
```

Έστω worker disappears.

Spark μπορεί συχνά να retry tasks:

```
</> Plain text

task lost
↓
schedule elsewhere
```

Άρα spot είναι ελκυστικό για fault-tolerant batch.

Όχι ιδανικό για κάθε workload.

Για παράδειγμα current Databricks real-time streaming mode απαιτεί spot off επειδή interruptions είναι κακές για latency-sensitive persistent processing. [Databricks Docu...](#)

36. Driver σε spot;

Συνήθως θέλεις driver να είναι πιο stable.

Αν χάσεις worker:

```
</> Plain text

some tasks retry
```

Αν χάσεις driver:

```
</> Plain text

Spark application itself
```

έχει πολύ μεγαλύτερο disruption.

Για αυτό architecture συνήθως διαφοροποιεί driver reliability από worker cost optimization.

37. Auto-termination

All-purpose cluster:

```
</> Plain text

running
↓
```



```
no commands
↓
idle
↓
X minutes
↓
terminate
```

Γιατί;

Επειδή interactive clusters κοστίζουν ακόμα και όταν ο developer πήγε για καφέ.

38. Termination δεν σημαίνει Delete

Μετά termination:

```
</> Plain text

VMs gone
```

αλλά configuration παραμένει:

```
</> Plain text

DBR
worker type
driver type
autoscaling settings
name
permissions
```

Μπορείς να πατήσεις:

```
</> Plain text

Start
```

και να δημιουργηθεί fresh compute.

Ephemeral state όμως χάνεται:

```
</> Plain text

cache
shuffle
process memory
local temp files
Python variables
```

39. Jobs compute lifecycle

Jobs compute είναι διαφορετικό:

⌕ Plain text

```
Job starts
↓
compute starts
↓
tasks run
↓
job finishes
↓
compute goes away
```

Δεν είναι κυρίως:

⌕ Plain text

```
idle timeout
```

αλλά:

⌕ Plain text

```
workload lifecycle
```

Η Databricks γενικά αποθαρρύνει all-purpose compute για automated production jobs. [Databricks Docu...](#) +1

40. Compute permissions

Άλλος axis ξανά.

Classic compute μπορεί να έχει permissions όπως:

⌕ Plain text

```
CAN ATTACH TO
CAN RESTART
CAN MANAGE
```

CAN ATTACH TO σημαίνει ότι μπορείς να συνδέσεις notebook και να χρησιμοποιήσεις το compute. CAN RESTART περιλαμβάνει start/restart/terminate, ενώ CAN MANAGE επιτρέπει αλλαγή compute configuration/permissions.

[Databricks Docu...](#)

Αυτό δεν σημαίνει data access.

Μπορεί να έχεις:

</> Plain text

```
CAN ATTACH TO cluster
```

αλλά:

</> Plain text

```
SELECT prod.payroll  
=  
NO
```

και το query να αποτυγχάνει.

Compute ACL ≠ Unity Catalog ACL.

41. Serverless permissions

Serverless δεν σημαίνει:

όλοι έχουν access.

Workspace έχει serverless permission objects:

</> Plain text

```
Default Interactive Compute  
Default Automated Compute
```

με CAN USE / management permissions.

Το πρώτο αφορά notebooks/Databricks Connect και το δεύτερο automated Jobs/Pipelines. [Databricks Docu...](#)

42. Photon μέσα σε Phase 1

Τώρα το βάζουμε σωστά στο execution stack.

</> Plain text

```
Spark DataFrame/SQL  
↓  
Catalyst  
↓  
Physical Plan  
↓  
[ Photon native  
  or Spark JVM ]  
↓  
workers/CPU
```

Photon είναι native vectorized engine.

Serverless και SQL warehouses το έχουν enabled, ενώ στα classic all-purpose/jobs/pipelines είναι configurable και στο current UI είναι default-on. [Databricks Docu...](#)

43. Τι επιταχύνει καλά το Photon;

Typical:

⟨/⟩ Plain text

```
scans
filters
joins
aggregations
Delta/Parquet writes
SQL/DataFrames
```

Δεν είναι:

⟨/⟩ Plain text

```
arbitrary Python accelerator
```

Αν γράψεις:

⟨/⟩ Python

```
for x in range(10_000_000):
    do_python_work(x)
```

Photon δεν μετατρέπει magically αυτόν τον Python loop σε fast native SQL execution.

44. Photon και UDFs

Αν μεγάλο μέρος του query μπορεί να εκφραστεί με built-in:

⟨/⟩ Python

```
F.sum()
F.when()
F.col()
F.regexp_extract()
```

ο optimizer/native engine έχει πολύ μεγαλύτερη ευκαιρία.

Αν τα πάντα είναι opaque Python UDF:

```
</> Python  
  
my_python_udf(...)
```

γίνεσαι λιγότερο optimizer-friendly.

General best practice:

προτίμησε built-in Spark SQL/DataFrame functions όπου γίνεται.

45. Serverless — τι εξαφανίζεται από εσένα;

Classic UI:

```
</> Plain text  
  
DBR  
driver node type  
worker node type  
worker count  
min/max workers  
Photon  
pool  
spot  
access mode  
Spark configs  
init scripts  
...
```

Serverless:

```
</> Plain text  
  
much less infrastructure configuration
```

Databricks δεν επιτρέπει στο serverless user control για πράγματα όπως:

```
</> Plain text  
  
compute policies  
instance pools  
most Spark cluster configurations  
compute-scoped init scripts  
compute-scoped libraries  
``` :chatgpt-content-reference{index="24"}
```

Αυτό δεν είναι limitation μόνο· είναι και το abstraction benefit.

---

# 46. Τι θυσιάζεις με serverless;

Έλεγχο.

Για παράδειγμα current serverless limitations περιλαμβάνουν:

```
```text
no R
no RDD APIs
Spark Connect APIs only
no cluster instance pools
limited Spark configs
``` :chatgpt-content-reference{index="25"}
```

## 47. Compute selection algorithm

Για πρακτική δουλειά:



Αυτό συμφωνεί με current Databricks recommendations. [Databricks Docu...](#) +1

## 48. Πραγματικό sizing case

Έχουμε ETL:

</> Plain text

Input:

2 TB

Operations:

filter

```
join
groupBy

Daily batch
```

Initial classic compute:

```
</> Plain text

Driver:
8 cores
32 GB

Workers:
4 x
8 cores
32 GB

Total worker cores:
32

Total worker memory:
128 GB

Autoscale:
4-12
```

Suppose initial file scan creates:

```
</> Plain text

2,000 partitions
```

At 4 workers:

```
</> Plain text

32 concurrent tasks
```

At 12:

```
</> Plain text

96 concurrent tasks
```

Άρα scan phase μπορεί να αποκτήσει πολύ περισσότερο parallelism.

## 49. Αλλά join stage έχει skew

Suppose post-shuffle:

⟨/⟩ Plain text

```
499 partitions:
2 GB each

1 partition:
300 GB
```

Scaling 4 → 12 workers:

⟨/⟩ Plain text

```
small partitions finish faster
```

αλλά 300 GB task παραμένει bottleneck.

Άρα Spark UI θα δείξει:

⟨/⟩ Plain text

```
many tasks done

one task:
huge duration
huge shuffle read
huge spill
```

Diagnosis:

⟨/⟩ Plain text

```
skew
```

όχι:

⟨/⟩ Plain text

```
need 50 workers
```

---

## 50. Τι θα κοιτούσες στο Spark UI;

Για slow stage:

⟨/⟩ Plain text

```
Stages
↓
open stage
↓
task metrics
```



Κουτάς distributions:

</> Plain text

```
Duration
Input Size
Shuffle Read
Shuffle Write
Records
Memory Spill
Disk Spill
GC Time
```

Av:

</> Plain text

```
median task = 20 sec
max task = 15 min
```

με τεράστιο shuffle read στο max:

</> Plain text

```
strong skew signal
```

## 51. CPU-bound scenario

Spark UI / cluster metrics:

</> Plain text

```
CPU high
little spill
memory okay
tasks balanced
```

Πιθανό:

</> Plain text

```
CPU-bound
```

Solutions μπορεί να περιλαμβάνουν:

</> Plain text

```
more cores
compute-optimized worker
Photon
better algorithm/query
```

όχι απλώς περισσότερη RAM.

## 52. Memory-bound

</> Plain text

```
high spill
executor OOM
GC high
CPU not fully useful
```

Potential fixes:

</> Plain text

```
higher memory/core
more partitions
remove skew
smaller broadcast
different join
memory-optimized worker
```

## 53. I/O-bound

</> Plain text

```
CPU low
tasks spend time reading
```

Potential:

</> Plain text

```
storage/file-layout issue
too many small files
network
poor pruning
reading too much data
```

Scaling CPU μπορεί να μην βοηθήσει.

## 54. Classic config best-practice matrix

Workload symptom	Πρώτη κατεύθυνση σκέψης
CPU consistently saturated	περισσότερα/γρηγορότερα cores, Photon, query optimization
Executor OOM	memory/core, skew, partitions, join/window
Huge disk spill	memory pressure, partition size, skew
Driver OOM	collect/toPandas, driver size, plan/result size
Long cluster startup	serverless ή pool
Idle interactive cost	auto-termination
Variable workload	autoscaling
Need R/RDD/GPU	Dedicated classic
Typical DE workloads	Standard or serverless
Tiny number of partitions	adding workers won't help
Few extreme stragglers	skew investigation

## 55. Practical syntax: inspect runtime and Spark config

Inside notebook:

</> Python

spark.version

για Spark version.

You can inspect config:

</> Python

spark.conf.get("spark.sql.shuffle.partitions")

Databricks documents `spark.conf.get(...)` as the way to inspect Spark configuration. [Databricks Docu...](#)

## 56. See partition count

</> Python

df.rdd.getNumPartitions()

**Important caveat:** αυτό απαιτεί RDD access, άρα είναι κατάλληλο σε compatible classic compute, όχι current serverless Spark Connect environments όπου RDD APIs δεν υποστηρίζονται. [Databricks Docu...](#)

Alternative reasoning in serverless often comes from:

Plain text

```
query plan
Spark UI / query profile
```

rather than low-level RDD access.

## 57. Repartition

Python

```
df2 = df.repartition(200)
```

Meaning conceptually:

Plain text

```
redistribute data
→
200 partitions
```

Usually involves shuffle.

## 58. Coalesce

Python

```
df2 = df.coalesce(20)
```

Typically reduces partitions without full redistribution.

Useful when:

Plain text

```
too many partitions
```

but dangerous if you collapse too aggressively and lose parallelism.

## 59. Explain

```
</> Python
```

```
df.explain("formatted")
```

Look for:

```
</> Plain text
```

```
Exchange
HashAggregate
BroadcastHashJoin
SortMergeJoin
```

This bridges compute sizing with execution.

Example:

```
</> Plain text
```

```
Exchange hashpartitioning(customer_id, 200)
```

tells you shuffle partitioning is happening.

---

## 60. Practical experiment you should run

Create:

```
</> Python
```

```
from pyspark.sql import functions as F

df = spark.range(0, 10_000_000)

df = (
 df
 .withColumn("group_id", F.col("id") % 100)
)

result = (
 df
 .groupBy("group_id")
 .count()
)
```

Then:

```
</> Python
```

```
result.explain("formatted")
```

Look for Exchange .

Run:

</> Python

```
display(result)
```

Then open Spark UI and inspect:

</> Plain text

```
Job
Stage
Tasks
Shuffle
```

This is much more educational than merely reading definitions.

---

## 61. Experiment: create bad parallelism

</> Python

```
tiny_parallelism = df.coalesce(1)

tiny_parallelism.count()
```

Compare with:

</> Python

```
df.count()
```

On sufficiently large data, the single-partition version reduces usable parallelism dramatically.

Ask yourself:

</> Plain text

```
Why are most cores idle?
```

Answer:

</> Plain text

```
only one partition
→ one task
```

## 62. Experiment: force more partitions

</> Python

```
many = df.repartition(500)

many.groupBy("group_id").count().write.mode("overwrite").format("noop").save()
```

Then inspect number of tasks.

The point is not:

500 is good.

The point is to see:

</> Plain text

```
partitioning
→ number of tasks
→ usable core parallelism
```

## 63. Certification trap: executors

Question:

Databricks classic compute has 6 workers. How many Spark executors?

For Databricks architecture:

</> Plain text

6

one executor per worker. [Databricks Docu...](#)

## 64. Certification trap: zero workers

Can multi-node Spark compute with:

</> Plain text

0 workers

run Spark distributed commands?

No.

The Databricks docs explicitly state that without worker nodes non-Spark commands can run on driver, but Spark commands fail. [Databricks Docu...](#)

If you want actual single-node Spark:

```
</> Plain text
```

```
Single Node
```

is a separate mode.

---

## 65. Certification trap: autoscaling

Autoscaling primarily changes:

```
</> Plain text
```

```
number of workers
```

not:

```
</> Plain text
```

```
worker instance size
```

for classic compute.

---

## 66. Certification trap: Standard vs Dedicated

Do not choose Dedicated because:

```
</> Plain text
```

```
production is important
```

Choose it because workload requires capabilities Standard cannot support.

Standard is Databricks' recommended classic access mode for most workloads. [Databricks Docu...](#)

---

## 67. Certification trap: Spark dynamic allocation

You may know vanilla Spark:

```
</> Plain text
```

```
spark.dynamicAllocation.enabled
```

But Databricks explicitly lists those dynamic-allocation properties as unsupported on classic and tells you to use **Databricks autoscaling** instead. [Databricks Docu...](#)



This is an excellent exam/interview distinction.

---

## 68. Important 2026 nuance: UI changed

Current UI no longer has a separate obvious “multi-node” selector.

Compute is multi-node by default, and you choose **Single node** when desired. Driver type moved under Advanced Performance and defaults to Auto. [🌐 Databricks Docu...](#)

So older tutorials/screenshots may look different.

---

## 69. Big Phase-1 test

Try before reading solutions.

### Q1

Compute:

⟨/⟩ Plain text

Driver:

4 cores

Workers:

5 × 8 cores

How many executor cores?

### Q2

A stage has:

⟨/⟩ Plain text

160 partitions

and cluster has:

⟨/⟩ Plain text

40 worker cores

Approximately how many waves under ideal balance?

### Q3

You increase from:

⟨/⟩ Plain text

5 workers → 20 workers

but stage only has 8 partitions.

Will it scale well?

#### Q4

What is the relationship between worker and executor in Databricks classic?

#### Q5

A single task spills 80 GB while all others spill almost nothing. What do you suspect first?

#### Q6

All executors die with OOM during a hash aggregation. What kinds of things investigate?

#### Q7

Driver crashes after:

```
</> Python
df.toPandas()
```

Workers have plenty of RAM. Why?

#### Q8

What is the difference between:

```
</> Plain text
more workers
```

and:

```
</> Plain text
larger worker type
```

?

#### Q9

Classic autoscaling 2–10 workers means what physically?

#### Q10

Does serverless autoscale?

#### Q11

Can you set traditional `min_workers=2,max_workers=10` for ordinary serverless notebook compute?

#### Q12

Why would you choose Dedicated?

**Q13**

Why would you choose Standard?

**Q14**

What problem does pool solve?

**Q15**

Does pool decide how many workers Spark needs?

**Q16**

What disappears when compute terminates?

**Q17**

What remains?

**Q18**

Why is Spot cheaper but riskier?

**Q19**

Photon means GPU?

**Q20**

Which is more likely to help a CPU-bound SQL aggregation: larger driver or Photon?

**Q21**

You have 64 cores but 1 partition. What's the limiting resource?

**Q22**

You have 500 balanced partitions and 8 cores. What's one obvious avenue?

**Q23**

Does more RAM automatically cure skew?

**Q24**

What does `Exchange` usually tell you?

**Q25**

Why can too many tiny partitions also be bad?

---

## Solutions

1

Plain text

$5 \times 8 = 40$

The driver's cores aren't executor cores.

2

</> Plain text

```
160 / 40 = 4 waves
```

assuming balanced task durations.

3

No.

Only 8 tasks exist, so worker scaling mostly leaves cores idle.

4

Databricks classic runs one executor per worker node.

5

Data skew / unusually large partition.

6

Check:

</> Plain text

```
partition sizes
shuffle partition count
skew
hash-table size
join strategy
memory/core
UDFs
windows
```

7

`toPandas()` materializes distributed data on the driver.

8

More workers:

</> Plain text

```
horizontal scale-out
```

Larger workers:

</> Plain text

```
vertical scale-up
```

**9**

Databricks may vary worker count between 2 and 10 while retaining the selected worker type.

**10**

Yes, but infrastructure scaling is platform-managed.

**11**

No, not using the classic configuration model.

**12**

When you need features like R, RDD APIs, GPU/ML Runtime, privileged machine access, or other Standard limitations.

[Databricks Docu...](#)

**13**

General DE/ETL and shared collaborative workloads; it is the recommended classic default. [Databricks Docu...](#)

**14**

Reduces instance provisioning/startup latency by keeping warm instances.

**15**

No.

Autoscaling determines worker count. Pool provides instances.

**16**

Ephemeral:

**</> Plain text**

```
executor memory
cache
shuffle
local temporary files
runtime processes
```

**17**

Persistent:

**</> Plain text**

```
Delta tables
workspace notebooks
Unity Catalog metadata
compute configuration
job definitions
```

**18**

Cloud provider can reclaim Spot capacity.

**19**

No.

Photon is CPU-native/vectorized query execution, not GPU.

20

Photon/query optimization is much more directly relevant.

21

Parallelism: one partition → one task.

22

More cores/workers can help if tasks are sufficiently large and independent.

23

No. Skew is a data-distribution problem. More memory may reduce failure/spill but doesn't inherently redistribute skewed data.

24

Data redistribution/shuffle boundary is likely occurring.

25

Task scheduling/setup overhead, tiny files, excessive metadata and inefficient execution can dominate useful work.

---

## Phase 1: what you should now be able to explain

You should be able to say, without notes:

### </> Plain text

A classic Databricks cluster has one Spark driver and one executor per worker.

Each worker contributes CPU cores, and Spark generally maps one task slot to each available executor core.

Partitions determine how many independent tasks a stage can expose. Therefore adding workers only helps if the stage has sufficient useful parallelism.

Worker memory is used for execution, caching, native/Python processing and platform overhead. When execution data cannot fit, Spark may spill to local worker disk.

Driver memory is separate. Actions such as `collect()` and `toPandas()` can OOM the driver even when executors have a large aggregate RAM capacity.

Classic autoscaling changes worker count between configured limits. Serverless also scales, but Databricks controls the underlying infrastructure.

Standard and Dedicated are access/capability modes, not performance tiers.

That is the conceptual core of **Phase 1**.

The next step within this phase should now become much more practical: **we should open a realistic Create Compute configuration field by field and design three clusters together — an interactive development cluster, a production batch ETL cluster, and a memory-heavy Spark job — then predict their Spark UI behavior and debug intentionally bad configurations.**

Sunday 10:55 AM

 databricks\_mastery\_phased\_plan....  
Document

Follow these plans phases and teach me the phase 2

Προχωράμε ακριβώς σύμφωνα με το roadmap που εξήγαγες: **Phase 2 — Delta Lake in Practice**. Στο αρχείο ο στόχος της φάσης είναι να περάσεις από το «καταλαβαίνω τα internals του Delta» στο «μπορώ να το χρησιμοποιώ σωστά σε production», με SQL syntax, PySpark/Delta APIs, πραγματικά engineering patterns και physical optimization.  databricks\_mastery\_phased\_plan

Θα το κάνουμε σε 6 ενότητες:

</> Plain text

PHASE 2 – DELTA LAKE IN PRACTICE

1. Table creation + writes

2. UPDATE / DELETE / MERGE

3. Write modes + overwrite semantics

4. Schema enforcement / evolution / type widening

5. History / time travel / restore / vacuum

6. Physical optimization

- └ small files
- └ OPTIMIZE
- └ data skipping
- └ file statistics
- └ deletion vectors
- └ liquid clustering
- └ predictive optimization

Και όπως συμφωνήσαμε, σε κάθε κομμάτι θα δούμε:

- τι πρόβλημα λύνει,
- πώς δουλεύει internally,
- πραγματική syntax,
- production use case,
- debugging,
- best practices,
- certification traps,
- ασκήσεις με λύσεις.

# 1. Δημιουργία Delta table

Σήμερα στο Databricks, τα tables είναι Delta by default εκτός αν ζητήσεις διαφορετικό format. Η Databricks μάλιστα προτείνει Unity Catalog managed Delta tables για τα περισσότερα workloads. [Databricks Docu...](#) +1

Έστω το project μας:

</> Plain text

```
catalog: ecommerce
schema: bronze
table: orders_raw
```

Μπορείς να δημιουργήσεις table με explicit schema:

</> SQL

```
CREATE TABLE ecommerce.bronze.orders_raw (
 order_id BIGINT,
 customer_id BIGINT,
 amount DECIMAL(10,2),
 status STRING,
 order_ts TIMESTAMP
);
```

Δεν χρειάζεται πλέον πάντα:

</> SQL

```
USING DELTA
```

γιατί Delta είναι το default table format στο Databricks. [Databricks Docu...](#)

Παρόλα αυτά, το explicit:

</> SQL

```
USING DELTA
```

είναι χρήσιμο εκπαιδευτικά και όταν θέλεις να είσαι απολύτως σαφής:

</> SQL

```
CREATE TABLE ecommerce.bronze.orders_raw (
 order_id BIGINT,
 customer_id BIGINT,
 amount DECIMAL(10,2),
 status STRING,
 order_ts TIMESTAMP
)
USING DELTA;
```



## 2. CREATE TABLE vs CREATE OR REPLACE TABLE

Αυτά δεν είναι το ίδιο.

</> SQL

```
CREATE TABLE ecommerce.bronze.orders_raw (...)
```

σημαίνει:

δημιουργήσέ το, αλλά αν υπάρχει ήδη συνήθως αποτυγχάνει.

Ενώ:

</> SQL

```
CREATE OR REPLACE TABLE ecommerce.bronze.orders_raw (
 ...
);
```

σημαίνει:

αντικατάστησε τον υπάρχοντα table definition/state με αυτόν.

Για development είναι βολικό.

Για production μπορεί να είναι επικίνδυνο αν δεν το χρησιμοποιείς συνειδητά.

## 3. CTAS — Create Table As Select

Πολύ σημαντικό pattern.

</> SQL

```
CREATE OR REPLACE TABLE ecommerce.silver.orders AS
SELECT
 CAST(order_id AS BIGINT) AS order_id,
 customer_id,
 CAST(amount AS DECIMAL(10,2)) AS amount,
 UPPER(status) AS status,
 order_ts
FROM ecommerce.bronze.orders_raw;
```

Αυτό είναι:

### CTAS — CREATE TABLE AS SELECT

Δεν ορίζεις ξεχωριστά schema.

Το schema προκύπτει από το query.

Η Databricks χρησιμοποιεί αυτό το pattern ακόμη και στα επίσημα Delta tutorials. [Databricks Docu...](#)

## 4. CREATE TABLE LIKE

Έχεις production-like schema:

</> Plain text

```
ecommerce.silver.orders
```

και θέλεις empty table με ίδια δομή:

</> SQL

```
CREATE TABLE ecommerce.silver.orders_test
LIKE ecommerce.silver.orders;
```

Αυτό αντιγράφει table definition/metadata, όχι τα δεδομένα. Delta υποστηρίζει CREATE TABLE LIKE σε σύγχρονες runtimes.

[Databricks Docu...](#)

Χρήσιμο για:

</> Plain text

```
dev → test
prod-like staging table
schema cloning
migration testing
```

## 5. Δημιουργία μέσω PySpark

Modern recommended API:

</> Python

```
df.writeTo("ecommerce.bronze.orders_raw").create()
```

ή:

</> Python

```
df.writeTo("ecommerce.bronze.orders_raw").createOrReplace()
```

Η Databricks πλέον προτείνει το DataFrameWriterV2 API ( writeTo ) για πολλά modern table writes επειδή έχει καθαρότερα create/replace semantics και καλύτερο table/property support. [Databricks Docu...](#)

Παλιότερο και ακόμη παντού:

</> Python

```
df.write.saveAsTable("ecommerce.bronze.orders_raw")
```

## 6. writeTo() vs saveAsTable()

### saveAsTable()

Classic API:

</> Python

```
(
 df.write
 .mode("overwrite")
 .saveAsTable("ecommerce.bronze.orders_raw")
)
```

Είναι απλό και πολύ διαδεδομένο.

### writeTo()

Νεότερο V2 API:

</> Python

```
(
 df.writeTo("ecommerce.bronze.orders_raw")
 .using("delta")
 .createOrReplace()
)
```

Πλεονεκτήματα:

</> Plain text

```
clear create/replace semantics
better partitioning support
table properties
clustering support
conditional overwrite features
``` :chatgpt-content-reference{index="6"}
```

Για νέο code, θέλω να ξέρεις και τα δύο.

Άσκηση 1

Θέλεις να δημιουργήσεις empty table με ίδιο schema με:

```
```text
prod.silver.customers
```

χωρίς δεδομένα.

### Λύση

```
</> SQL
```

```
CREATE TABLE dev.silver.customers
LIKE prod.silver.customers;
```

## 7. INSERT

Ας έχουμε:

```
</> Plain text
```

```
ecommerce.silver.orders
```

και θέλουμε νέα rows:

```
</> SQL
```

```
INSERT INTO ecommerce.silver.orders
VALUES
 (1001, 51, 29.90, 'NEW', current_timestamp()),
 (1002, 92, 55.00, 'NEW', current_timestamp());
```

INSERT INTO σημαίνει:

πρόσθεσε rows.

Δεν αντικαθιστά τα υπάρχοντα.

Η Databricks documentation λέει ρητά ότι INTO είναι additive. [🌐 Databricks Docu...](#)

## 8. INSERT INTO ... SELECT

Πιο ρεαλιστικό:

```
</> SQL
```

```
INSERT INTO ecommerce.silver.orders
SELECT
 order_id,
 customer_id,
 amount,
 status,
 order_ts
FROM ecommerce.bronze.orders_raw
WHERE ingestion_date = current_date();
```

Αυτό είναι πολύ συνηθισμένο batch ETL pattern.

## 9. INSERT OVERWRITE

Πρόσεξε.

</> SQL

```
INSERT OVERWRITE TABLE ecommerce.silver.orders
SELECT *
FROM staging.orders;
```

Αυτό δεν κάνει append.

Κάνει overwrite του target scope.

Χωρίς partition restriction μπορεί ουσιαστικά να αντικαταστήσει ολόκληρο table contents. [Databricks Docu...](#)

Άρα:

</> Plain text

```
INSERT INTO
= additive

INSERT OVERWRITE
= replacement
```

## 10. PySpark append

</> Python

```
(
 df.write
 .mode("append")
 .saveAsTable("ecommerce.silver.orders")
)
```

ή:

</> Python

```
df.writeTo("ecommerce.silver.orders").append()
```

To append σημαίνει:

</> Plain text

```
existing rows
+
new rows
```

## 11. PySpark overwrite

</> Python

```
(
 df.write
 .mode("overwrite")
 .saveAsTable("ecommerce.silver.orders")
)
```

Αυτό μπορεί να αντικαταστήσει όλο το table.

Πολύ επικίνδυνο αν το DataFrame περιέχει μόνο subset.

## 12. insertInto() — σημαντική παγίδα

Υπάρχει:

</> Python

```
df.write.insertInto("ecommerce.silver.orders")
```

Αυτό απαιτεί συμβατό schema.

Αλλά το κρίσιμο είναι:

`insertInto()` κάνει column matching **by position**, όχι by name. [🌐 Databricks Docu...](#)

Παράδειγμα:

Target:

</> Plain text

```
id
name
```

DataFrame:

</> Plain text

```
name
id
```

Αν οι types επιτρέπουν coercion, μπορεί να πάρεις λάθος mapping επειδή resolution γίνεται positional.

Για αυτό σε σύγχρονο code θέλω να είσαι προσεκτικός με `insertInto` .

## Άσκηση 2

Target schema:

</> Plain text

```
id BIGINT
name STRING
```

DataFrame columns:

</> Plain text

```
name
id
```

Ποιο API είναι πιο επικίνδυνο;

## Λύση

</> Python

```
insertInto(...)
```

επειδή χρησιμοποιεί position-based resolution. [Databricks Docu...](#)

## 13. UPDATE

Delta υποστηρίζει SQL DML.

</> SQL

```
UPDATE ecommerce.silver.orders
SET status = 'CANCELLED'
WHERE order_id = 1001;
```

Logical meaning:

</> Plain text

```
find matching rows
↓
change current table state
↓
commit new Delta version
```

Δεν σημαίνει απαραίτητα:

</> Plain text

```
open Parquet
edit bytes in place
```

Όπως είδαμε, Delta μπορεί να κάνει rewrite ή deletion-vector based update.

## 14. Conditional update

</> SQL

```
UPDATE ecommerce.silver.orders
SET
 status = 'LATE',
 amount = amount * 0.95
WHERE
 status = 'SHIPPING'
 AND order_ts < current_timestamp() - INTERVAL 7 DAYS;
```

## 15. DELETE

</> SQL

```
DELETE FROM ecommerce.silver.orders
WHERE status = 'CANCELLED';
```

Logical removal.

Physical files μπορεί να παραμείνουν λόγω history/time travel μέχρι `VACUUM` .

## 16. Why UPDATE/DELETE are useful in lakehouse

Σε plain Parquet folders, mutable semantics είναι δύσκολα.

Με Delta:

</> Plain text

```
UPDATE
DELETE
MERGE
```

είναι transactional operations.

Αυτό είναι ένα βασικό reason που λέμε ότι Delta δίνει database-like table semantics πάνω σε object storage.

## 17. TRUNCATE TABLE

Αν θέλεις να αδειάσεις ολόκληρο table:



```
</> SQL
```

```
TRUNCATE TABLE ecommerce.bronze.orders_raw;
```

Conceptually:

```
</> Plain text
```

```
schema remains
table object remains
rows removed
```

Δεν είναι το ίδιο με:

```
</> SQL
```

```
DROP TABLE
```

To `DROP TABLE` αφαιρεί το table object.

## 18. ALTER TABLE

Χρησιμοποιείται για metadata/schema/properties.

Π.χ.:

```
</> SQL
```

```
ALTER TABLE ecommerce.silver.orders
ADD COLUMN source_system STRING;
```

Rename:

```
</> SQL
```

```
ALTER TABLE ecommerce.silver.orders
RENAME COLUMN status TO order_status;
```

Set properties:

```
</> SQL
```

```
ALTER TABLE ecommerce.silver.orders
SET TBLPROPERTIES (
 'delta.enableDeletionVectors' = 'true'
);
```

## 19. MERGE — το σημαντικότερο production DML

Αυτό αξίζει μεγάλη προσοχή.

Target:

</> Plain text

```
customers
```

```
id | name | city
---+-----+-----
1 | Maria | Athens
2 | John | Patras
```

Source updates:

</> Plain text

```
id | name | city
---+-----+-----
2 | John | Volos
3 | Anna | Larisa
```

Θέλουμε:

</> Plain text

```
id=2 → UPDATE
id=3 → INSERT
```

Αυτό είναι **upsert**.

SQL:

</> SQL

```
MERGE INTO ecommerce.silver.customers AS t
USING ecommerce.staging.customer_updates AS s

ON t.id = s.id

WHEN MATCHED THEN
 UPDATE SET
 t.name = s.name,
 t.city = s.city

WHEN NOT MATCHED THEN
 INSERT (
 id,
 name,
 city
)
 VALUES (
 s.id,
 s.name
```

```
 s.name,
 s.city
);
```

MERGE είναι Delta-only DML για insert/update/delete based on matching conditions. [Databricks Docu...](#) +1

## 20. Mental model του MERGE

Για κάθε source row:

</> Plain text

```
source row
 ↓
find target match using ON
 ↓
┌───┴───┐
| MATCHED | NOT MATCHED | target-only |
| | | |
| | | |
| UPDATE | INSERT | optionally |
| DELETE | | UPDATE/DELETE |
```

## 21. WHEN NOT MATCHED BY SOURCE

Έστω target:

</> Plain text

```
1
2
3
```

source authoritative snapshot:

</> Plain text

```
1
2
```

και θέλεις να διαγράψεις το 3:

</> SQL

```
MERGE INTO target t
 USING source s
 ON t.id = s.id

 WHEN MATCHED THEN UPDATE SET *

 WHEN NOT MATCHED THEN INSERT *
```

```
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

Αυτό είναι ισχυρό.

Και επικίνδυνο.

Αν source είναι μόνο:

```
</> Plain text
```

```
today's changed rows
```

και όχι full snapshot, τότε θα διαγράψεις legitimate target rows.

## 22. Duplicate match problem

Source:

```
</> Plain text
```

```
id=7, city=Athens, seq=10
id=7, city=Volos, seq=11
```

Target:

```
</> Plain text
```

```
id=7
```

Ποιο source row πρέπει να κερδίσει;

Δεν ξέρουμε.

Άρα πρώτα κάνουμε dedup.

## 23. Deduplication με Window

```
</> SQL
```

```
WITH latest AS (
 SELECT
 *,
 ROW_NUMBER() OVER (
 PARTITION BY id
 ORDER BY sequence_number DESC
) AS rn
 FROM ecommerce.staging.customer_updates
)
```

```
SELECT *
FROM latest
WHERE rn = 1;
```

Μετά:

```
</> SQL

MERGE ...
USING latest ...
```

Αυτό είναι πραγματικό CDC pattern.

## 24. Idempotent MERGE

Πολύ σημαντικό production concept.

Idempotent σημαίνει:

αν τρέξω την ίδια operation δεύτερη φορά με το ίδιο input, το final state να μην αλλάξει ξανά.

Παράδειγμα:

```
</> Plain text

source:
id=5, status='PAID'
```

To MERGE :

```
</> Plain text

first run:
updates id=5

second run:
updates same row to same final state
```

final table state ίδιο.

Αυτό είναι πολύ χρήσιμο όταν job retry-άρει.

## 25. Non-idempotent write

```
</> SQL

UPDATE orders
SET amount = amount + 10
WHERE id = 5;
```

Αν run γίνει δύο φορές:

</> Plain text

```
+10
+10
```

Άρα δεν είναι idempotent.

## 26. MERGE in Python

</> Python

```
from delta.tables import DeltaTable

target = DeltaTable.forName(
 spark,
 "ecommerce.silver.customers"
)

source = spark.table(
 "ecommerce.staging.customer_updates"
)

(
 target.alias("t")
 .merge(
 source.alias("s"),
 "t.id = s.id"
)
 .whenMatchedUpdate(
 set={
 "name": "s.name",
 "city": "s.city"
 }
)
 .whenNotMatchedInsert(
 values={
 "id": "s.id",
```

To SQL και το Python API περιγράφουν την ίδια merge semantics. [Databricks Docu...](#)

### Άσκηση 3

Source έχει:

</> Plain text

```
id=10 version=1 amount=100
id=10 version=2 amount=120
```

Πριν το MERGE , τι κάνεις;

## Λύση

Deduplicate/rank by business key και ordering field:

```
</> SQL
```

```
ROW_NUMBER() OVER (
 PARTITION BY id
 ORDER BY version DESC
)
```

και κρατάς `rn = 1` .

## 27. Append vs overwrite — production thinking

Έστω daily file:

```
</> Plain text
```

```
2026-09-27 orders
```

### Append

```
</> Python
```

```
df.write.mode("append").saveAsTable(...)
```

καλό όταν:

```
</> Plain text
```

```
data are genuinely new
no overlap
```

### Full overwrite

```
</> Python
```

```
df.write.mode("overwrite").saveAsTable(...)
```

καλό όταν:

```
</> Plain text
```

```
target should exactly equal this new DataFrame
```

## 28. To overwrite trap

Έχεις table με 3 χρόνια data.

DataFrame:

</> Plain text

```
only yesterday
```

και κάνεις:

</> Python

```
df.write.mode("overwrite").saveAsTable(...)
```

Μπορεί να χάσεις τα άλλα 3 χρόνια logical state.

Αυτό είναι από τα πιο κλασικά production mistakes.

---

## 29. Selective overwrite

Αν θέλεις να αντικαταστήσεις μόνο:

</> Plain text

```
date = 2026-09-27
```

σήμερα υπάρχουν καλύτερες επιλογές από `legacy partitionOverwriteMode` .

Η Databricks συνιστά:

</> Plain text

```
REPLACE WHERE
ή
REPLACE USING
```

για νέα workloads. [Databricks Docu...](#) +1

---

## 30. REPLACE WHERE

Παράδειγμα:

</> SQL

```
INSERT INTO ecommerce.silver.orders
REPLACE WHERE order_date = DATE'2026-09-27'
SELECT *
FROM ecommerce.staging.orders_daily;
```



## Semantics:

</> Plain text

```
replace rows matching predicate
leave everything else untouched
```

REPLACE WHERE υποστηρίζεται σε όλα τα compute types. [Databricks Docu...](#)

## 31. Dynamic overwrite με REPLACE USING

Νεότερο pattern:

</> SQL

```
INSERT INTO ecommerce.silver.orders
REPLACE USING (order_date)
SELECT *
FROM ecommerce.staging.orders_daily;
```

Αυτό αντικαθιστά rows που match-άρουν στις specified key columns με βάση το incoming dataset.

Η Databricks σήμερα προτείνει REPLACE USING για dynamic overwrite αντί του legacy partitionOverwriteMode .

[Databricks Docu...](#) +1

## 32. Legacy partitionOverwriteMode

Old style:

</> Python

```
(
 df.write
 .mode("overwrite")
 .option("partitionOverwriteMode", "dynamic")
 .saveAsTable("ecommerce.silver.orders")
)
```

Αυτό παραμένει supported σε συγκεκριμένα contexts, αλλά είναι legacy και δεν είναι η preferred επιλογή για νέα workloads.

Επίσης έχει compute limitations. [Databricks Docu...](#) +1

## 33. Γιατί dynamic partition overwrite είναι επικίνδυνο;

Έστω incoming data κατά λάθος περιέχει μία row:

</> Plain text

```
order_date = 2024-01-01
```

ενώ ήθελες μόνο:

</> Plain text

2026-09-27

Dynamic overwrite μπορεί να αντικαταστήσει όλη την partition του 2024-01-01.

Άρα:

πάντα validate το incoming key/partition domain.

## Άσκηση 4

Θέλεις να αντικαταστήσεις μόνο:

</> Plain text

country='GR'

και να κρατήσεις DE/FR unchanged.

Τι είναι καλύτερο από full overwrite;

## Λύση

Selective overwrite, π.χ.:

</> SQL

```
REPLACE WHERE country = 'GR'
```

## 34. Schema enforcement

Target:

</> Plain text

```
id BIGINT
amount DECIMAL
```

Source:

</> Plain text

```
id BIGINT
amount STRUCT
```

Δεν θέλεις να γίνει silent corruption.

Delta κάνει schema validation on write. [Databricks Docu...](#)

Rules include:

Plain text

```
columns must exist
types must match or be safely castable
```

## 35. Safe casting

Target:

Plain text

```
amount BIGINT
```

Source:

Plain text

```
amount INT
```

To Databricks μπορεί να κάνει safe cast.

Αλλά:

Plain text

```
amount = complex struct
```

δεν μπορεί να γίνει safe cast σε BIGINT.

## 36. Schema evolution

Source σήμερα:

Plain text

```
id
name
```

αύριο:

Plain text

```
id
name
segment
```

Αν `segment` είναι `intentional`, μπορείς να επιτρέψεις `evolution`.

Python:

```
</> Python

(
 df.write
 .option("mergeSchema", "true")
 .mode("append")
 .saveAsTable("ecommerce.silver.customers")
)
```

`mergeSchema` είναι `per-write schema evolution control`. [Databricks Docu...](#) +1

## 37. Γιατί `per-write` > `global autoMerge`

Μπορείς να κάνεις:

```
</> Python

spark.conf.set(
 "spark.databricks.delta.schema.autoMerge.enabled",
 "true"
)
```

αλλά τότε πολλά `subsequent writes` μπορεί να `evolve schema`.

Σε `production` προτιμάμε `explicit/local evolution` όταν γίνεται:

```
</> Python

.option("mergeSchema", "true")
```

ώστε να είναι ξεκάθαρο ποια `write operation` έχει δικαίωμα να αλλάξει το `schema`.

## 38. `overwriteSchema`

Αυτό είναι διαφορετικό.

```
</> Python

(
 df.write
 .mode("overwrite")
 .option("overwriteSchema", "true")
```

```
.saveAsTable(...)
)
```

σημαίνει:

overwrite τα data και αντικατάστησε επίσης schema/partitioning.

Είναι πολύ ισχυρό.

Δεν είναι:

</> Plain text

```
just add a new column
```

Είναι περισσότερο:

</> Plain text

```
replace table schema during overwrite
``` :chatgpt-content-reference{index="19"}
```

39. Type widening

Έστω:

```
```text  
quantity INT
```

και dataset μεγαλώνει τόσο που θέλεις:

</> Plain text

```
quantity BIGINT
```

Αυτό είναι widening.

Η Databricks έχει πλέον type widening GA, και υποστηρίζει conversions όπως:

</> Plain text

```
BYTE → SHORT/INT/BIGINT/DECIMAL/DOUBLE
SHORT → INT/BIGINT/DECIMAL/DOUBLE
INT → BIGINT/DECIMAL/DOUBLE
FLOAT → DOUBLE
```

χωρίς rewrite των underlying data files για supported changes. [Databricks Docu...](#) +1

Enable:

```
</> SQL
```

```
ALTER TABLE ecommerce.silver.orders
SET TBLPROPERTIES (
 'delta.enableTypeWidening' = 'true'
);
```

και:

```
</> SQL
```

```
ALTER TABLE ecommerce.silver.orders
ALTER COLUMN quantity TYPE BIGINT;
```

## 40. Why widening ≠ arbitrary cast

```
</> Plain text
```

```
INT → BIGINT
```

είναι generally widening.

```
</> Plain text
```

```
STRING → STRUCT
```

δεν είναι.

Άρα type evolution δεν σημαίνει:

οποιοσδήποτε type σε οποιονδήποτε άλλο.

## 41. DESCRIBE DETAIL

Θες να μάθεις:

```
</> Plain text
```

```
format
location
size
number of files
partition columns
table properties/features
```

Τρέχεις:

&lt;/&gt; SQL

```
DESCRIBE DETAIL ecommerce.silver.orders;
```

Επιστρέφει μία row με table-level metadata. [Databricks Docu...](#)

Πολύ χρήσιμο debugging command.

## 42. DESCRIBE HISTORY

&lt;/&gt; SQL

```
DESCRIBE HISTORY ecommerce.silver.orders;
```

βλέπεις:

&lt;/&gt; Plain text

```
version
timestamp
user
operation
operationParameters
operationMetrics
...
```

To history retention είναι default 30 days. [Databricks Docu...](#) +1

## 43. Γιατί history είναι τόσο δυνατό;

Production bug:

&lt;/&gt; Plain text

```
09:00 correct
11:00 wrong
```

Κάνεις:

&lt;/&gt; SQL

```
DESCRIBE HISTORY ecommerce.silver.orders;
```

και βρίσκεις:

&lt;/&gt; Plain text

```
v120 WRITE
```

```
v121 MERGE
v122 UPDATE
```

με timestamps.

Έτσι ξέρεις ποια operation να ερευνήσεις.

## 44. Time travel

```
</> SQL
```

```
SELECT *
FROM ecommerce.silver.orders
VERSION AS OF 121;
```

ή:

```
</> SQL
```

```
SELECT *
FROM ecommerce.silver.orders
TIMESTAMP AS OF '2026-09-26 10:00:00';
```

Αυτό διαβάζει old snapshot.

Δεν αλλάζει το current table.

## 45. Python time travel

```
</> Python
```

```
old_df = (
 spark.read
 .option("versionAsOf", 121)
 .table("ecommerce.silver.orders")
)
```

ή path-based equivalents όπου κατάλληλο.

## 46. RESTORE

Αν θέλεις old state να γίνει ξανά current:

```
</> SQL
```

```
RESTORE TABLE ecommerce.silver.orders
TO VERSION AS OF 121;
```



Δεν «γυρνάει πίσω τον χρόνο».

Δημιουργεί νέα commit/version με old logical state. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
v121 good
v122 bad
v123 bad
v124 RESTORE state equivalent to v121
```

## 47. Time travel ≠ backup

Πολύ σημαντικό.

Η Databricks λέει ρητά να μη χρησιμοποιείς Delta history σαν long-term backup mechanism. Default data-file retention για VACUUM είναι 7 days, ενώ log retention είναι 30 days. [Databricks Docu...](#) +1

Άρα:

</> Plain text

```
history exists
```

δεν σημαίνει απαραίτητα:

</> Plain text

```
old files still exist forever
```

## 48. VACUUM

Old files μπορεί να είναι logically removed αλλά physically present.

</> SQL

```
VACUUM ecommerce.silver.orders;
```

καθαρίζει unreferenced files πέρα από retention window.

Default retention είναι 7 days. [Databricks Docu...](#)

## 49. Γιατί όχι VACUUM RETAIN 0 HOURS;

Επειδή μπορεί να καταστρέψεις:

</> Plain text

```
time travel
long-running readers
recovery possibilities
```

Short retention είναι dangerous.

Μην το χρησιμοποιείς σαν:

θέλω τώρα χώρο, άρα μηδέν ώρες.

## 50. OPTIMIZE

Έστω table έχει:

</> Plain text

```
50,000 files × 1 MB
```

Αυτό είναι κακό για:

</> Plain text

```
metadata overhead
file listing
task scheduling
file open cost
```

Τρέχεις:

</> SQL

```
OPTIMIZE ecommerce.silver.orders;
```

Το OPTIMIZE βελτιώνει physical layout. Χωρίς clustering/Z-order semantics, κάνει bin packing μικρών files σε πιο ισορροπημένα file sizes. [Databricks Docu...](#)

## 51. OPTIMIZE ≠ VACUUM

</> Plain text

```
OPTIMIZE
=
rewrite/reorganize active data files
```

```
VACUUM
=
```

```
delete old unreferenced files
```

Ένα πολύ συνηθισμένο λάθος είναι να τα μπερδεύεις.

## 52. Small file problem

Γιατί τα μικρά files είναι κακά;

Έστω:

```
</> Plain text
```

```
10 GB dataset
```

Option A:

```
</> Plain text
```

```
10 files × 1 GB
```

Option B:

```
</> Plain text
```

```
100,000 files × 100 KB
```

Το B δημιουργεί:

```
</> Plain text
```

```
huge metadata/file-open overhead
many tiny Spark tasks
inefficient scheduling
poor scan efficiency
```

Άρα δεν κοιτάμε μόνο:

```
</> Plain text
```

```
total bytes
```

αλλά και:

```
</> Plain text
```

```
number/size distribution of files
```

## 53. Data skipping

Έστω file statistics λένε:

</> Plain text

```
file A:
customer_id min=1 max=1000

file B:
customer_id min=1001 max=2000

file C:
customer_id min=2001 max=3000
```

Query:

</> SQL

```
SELECT *
FROM orders
WHERE customer_id = 2500;
```

Engine μπορεί να πει:

</> Plain text

```
A cannot contain 2500
skip

B cannot contain 2500
skip

C may contain 2500
read
```

Αυτό είναι **data skipping**.

Δεν χρειάζεται να ανοίξει κάθε file.

---

## 54. File statistics

Delta stores useful file-level statistics.

Τυπικά:

</> Plain text

```
min values
max values
null counts
row count
```

και αυτά βοηθούν pruning/data skipping/query planning.

Predictive optimization επίσης συλλέγει statistics σε Unity Catalog managed tables. [Databricks Docu...](#)

## 55. Deletion vectors

Χωρίς DV:

Plain text

```
delete one row
↓
rewrite whole Parquet file
```

Με DV:

Plain text

```
data file remains
+
metadata marks row as deleted
```

Reads εφαρμόζουν το deletion vector.

Αυτό επιταχύνει:

Plain text

```
DELETE
UPDATE
MERGE
``` :chatgpt-content-reference{index="28"}
```

56. Γιατί DV βοηθούν concurrency;

Αν δύο writers αλλάζουν διαφορετικά rows του ίδιου physical file, row-level semantics με DV μπορούν να μειώσουν conflicts

To file δεν χρειάζεται πάντα full rewrite για κάθε row change.

57. Liquid clustering

Αυτό είναι πλέον πολύ σημαντικό.

Παλιότερη λογική:

```
```text
partition table by date
+
maybe ZORDER customer_id
```

## Modern Databricks:

Plain text

Liquid Clustering

Η Databricks περιγράφει το liquid clustering ως layout technique που αντικαθιστά παραδοσιακό partitioning και ZORDER για πολλά use cases. Τα clustering keys μπορούν να αλλάζουν καθώς αλλάζουν τα query patterns. [Databricks Docu...](#)

## 58. Syntax

SQL

```
ALTER TABLE ecommerce.silver.orders
CLUSTER BY (customer_id);
```

Για να εφαρμοστεί clustering σε existing data:

SQL

```
OPTIMIZE ecommerce.silver.orders FULL;
```:chatgpt-content-reference{index="30"}
```

59. Γιατί "liquid";

Επειδή δεν είσαι τόσο «κλειδωμένος» σε static partition layout.

Traditional partition:

```
```text
partitionBy(country)
```

είναι physical table design decision.

Liquid clustering:

Plain text

CLUSTER BY (...)

είναι πιο adaptable.

Μπορείς να αλλάξεις clustering keys χωρίς να απαιτείς immediate rewrite όλου του historical data. [Databricks Docu...](#)

## 60. Πότε liquid clustering έχει νόημα;

Columns που χρησιμοποιούνται συχνά σε:

</> Plain text

```
filters
joins
selective lookups
```

και έχουν αρκετή cardinality ώστε το physical ordering/layout να βοηθήσει.

Παράδειγμα:

</> SQL

```
CLUSTER BY (customer_id)
```

αν έχεις συχνά:

</> SQL

```
WHERE customer_id = ?
```

## 61. Predictive optimization

Αν έχεις Unity Catalog managed tables, η Databricks μπορεί να αναλάβει maintenance αυτόματα.

Predictive optimization μπορεί να τρέχει:

</> Plain text

```
OPTIMIZE
VACUUM
ANALYZE
```

και να αποφασίζει ποια tables ωφελούνται. Η Databricks το συνιστά για Unity Catalog managed tables. [🌐 Databricks Docu...](#)

## 62. Τι σημαίνει πρακτικά;

Χωρίς predictive optimization:

</> Plain text

```
you schedule:
OPTIMIZE every Sunday
VACUUM every Monday
ANALYZE every...
```

Με predictive optimization:

</> Plain text

```
Databricks monitors
↓
table needs maintenance?
↓
runs maintenance when useful
```

Αυτό μειώνει manual maintenance.

## 63. Τι δεν σημαίνει predictive optimization;

Δεν σημαίνει:

δεν χρειάζεται να καταλαβαίνω optimization.

Γιατί σε debugging πρέπει να ξέρεις:

</> Plain text

```
why files are small
why skipping doesn't work
why clustering key is bad
why table scan is huge
```

Automation δεν αντικαθιστά engineering understanding.

## 64. Production project — Phase 2 implementation

Τώρα εφαρμόζουμε τα concepts στο e-commerce project.

### Bronze

</> SQL

```
CREATE TABLE ecommerce.bronze.orders_raw (
 order_id BIGINT,
 customer_id BIGINT,
 amount DECIMAL(10,2),
 status STRING,
 order_ts TIMESTAMP,
 ingestion_ts TIMESTAMP
);
```

### Silver

</> SQL

```
CREATE TABLE ecommerce.silver.orders (

```



```
order_id BIGINT,
customer_id BIGINT,
amount DECIMAL(10,2),
status STRING,
order_ts TIMESTAMP,
updated_at TIMESTAMP
);
```

## Staging updates

</> SQL

```
CREATE OR REPLACE TABLE ecommerce.staging.orders_updates AS

SELECT * FROM VALUES
 (1001L, 51L, 29.90D, 'PAID'),
 (1003L, 92L, 44.50D, 'NEW')
AS t(order_id, customer_id, amount, status);
```

## Idempotent merge

</> SQL

```
MERGE INTO ecommerce.silver.orders AS t

USING ecommerce.staging.orders_updates AS s

ON t.order_id = s.order_id

WHEN MATCHED THEN UPDATE SET
 t.customer_id = s.customer_id,
 t.amount = s.amount,
 t.status = s.status,
 t.updated_at = current_timestamp()

WHEN NOT MATCHED THEN INSERT (
 order_id,
 customer_id,
 amount,
 status,
 updated_at
)
VALUES (
 s.order_id,
 s.customer_id,
 s.amount,
 s.status,
 current_timestamp()
);
```

## 65. Debugging workflow 1 — merge fails

Error:

</> Plain text

```
multiple source rows matched target row
```

Investigate:

</> SQL

```
SELECT
 order_id,
 COUNT(*)
FROM ecommerce.staging.orders_updates
GROUP BY order_id
HAVING COUNT(*) > 1;
```

Then determine ordering field:

</> Plain text

```
event_sequence
updated_at
version
```

Deduplicate before merge.

---

## 66. Debugging workflow 2 — schema mismatch

Write fails.

Run:

</> SQL

```
DESCRIBE TABLE ecommerce.silver.orders;
```

Compare with:

</> Python

```
df.printSchema()
```

Questions:

</> Plain text

```
new column?
different type?
column renamed?
```

```
intentional evolution?
source bug?
```

Do not blindly enable `mergeSchema` .

## 67. Debugging workflow 3 — storage exploded

Run:

```
</> SQL

DESCRIBE DETAIL ecommerce.silver.orders;
```

Look at:

```
</> Plain text

numFiles
sizeInBytes
```

If:

```
</> Plain text

hundreds of thousands of files
tiny average size
```

you have small-file pressure.

## 68. Debugging workflow 4 — query scans too much

Suppose:

```
</> SQL

WHERE customer_id = 1234
```

still scans most data.

Investigate:

```
</> Plain text

table layout
clustering
statistics
filter selectivity
data skipping
```

Maybe liquid clustering on `customer_id` is appropriate.

## 69. Debugging workflow 5 — someone broke data

Run:

</> SQL

```
DESCRIBE HISTORY ecommerce.silver.orders;
```

Then:

</> SQL

```
SELECT *
FROM ecommerce.silver.orders
VERSION AS OF <previous_good_version>;
```

If needed:

</> SQL

```
RESTORE TABLE ecommerce.silver.orders
TO VERSION AS OF <good_version>;
```

## 70. Best practices summary

Για νέα Delta projects:

1. Prefer Unity Catalog managed Delta tables unless you have a reason not to. [Databricks Docu...](#) +1
2. Prefer `writeTo()` / V2 semantics for modern table creation where practical. [Databricks Docu...](#)
3. Use `MERGE` for idempotent upserts.
4. Deduplicate source before `MERGE`.
5. Avoid blind full `overwrite`.
6. Prefer `REPLACE WHERE` / `REPLACE USING` over legacy dynamic partition overwrite for new workloads. [Databricks Docu...](#)
7. Use schema enforcement as protection, not nuisance.
8. Enable schema evolution explicitly and locally when possible.
9. Use `DESCRIBE HISTORY` and time travel for debugging.
0. Do not treat time travel as long-term backup. [Databricks Docu...](#)
1. Understand `OPTIMIZE` vs `VACUUM`.
2. Prefer liquid clustering for many modern table-layout cases rather than designing everything around static partitions. [Databricks Docu...](#)
3. Enable predictive optimization for Unity Catalog managed tables where supported. [Databricks Docu...](#)

## Μεγάλο Phase 2 test

## Q1

Ποια η διαφορά:

```
</> Plain text
```

```
INSERT INTO
```

και:

```
</> Plain text
```

```
INSERT OVERWRITE
```

## Απάντηση

INSERT INTO προσθέτει rows. INSERT OVERWRITE αντικαθιστά target scope.

## Q2

Τι διαφορά έχει:

```
</> Python
```

```
saveAsTable(...)
```

από:

```
</> Python
```

```
writeTo(...)
```

;

## Απάντηση

`saveAsTable()` είναι παλιότερο `DataFrameWriter` API. `writeTo()` είναι `DataFrameWriterV2` και παρέχει πιο explicit create/replace/partitioning/table semantics. [⊕ Databricks Docu...](#)

## Q3

Γιατί το `insertInto()` θέλει προσοχή;

## Απάντηση

Επειδή κάνει position-based column resolution αντί name-based. [⊕ Databricks Docu...](#)

## Q4

Πότε χρησιμοποιείς `append`;

## Απάντηση

Όταν incoming rows είναι genuinely additive και δεν αντικαθιστούν existing state.

---

## Q5

Πότε full overwrite είναι επικίνδυνο;

## Απάντηση

Όταν DataFrame περιέχει μόνο subset του table.

---

## Q6

Ποιο modern pattern προτείνεται για selective overwrite;

## Απάντηση

REPLACE WHERE , REPLACE USING ή REPLACE ON ανά use case. [🌐 Databricks Docu...](#)

---

## Q7

Τι κάνει MERGE ;

## Απάντηση

Ταιριάζει source/target rows με condition και μπορεί transactional να κάνει update, insert και delete.

---

## Q8

Γιατί duplicate source keys πριν από MERGE είναι επικίνδυνα;

## Απάντηση

Επειδή πολλαπλά source rows μπορεί να match-άρουν το ίδιο target row, άρα η intended final value είναι ambiguous.

---

## Q9

Πώς το λύνεις;

## Απάντηση

Deduplicate με business key και deterministic ordering field.

---

## Q10

Τι σημαίνει idempotent MERGE ;

## Απάντηση

Το ίδιο input, αν ξανατρέξει, δεν αλλάζει περαιτέρω το final logical state.

## Q11

Τι είναι schema enforcement;

### Απάντηση

Validation ότι incoming columns/types είναι συμβατά με target schema. [Databricks Docu...](#)

## Q12

Τι είναι schema evolution;

### Απάντηση

Controlled modification του target schema ώστε να αποδεχθεί intentional structural changes.

## Q13

mergeSchema και overwriteSchema είναι το ίδιο;

### Απάντηση

Όχι.

⟨⟩ Plain text

mergeSchema  
→ evolve/add compatible schema

overwriteSchema  
→ replace schema during overwrite

## Q14

Τι είναι type widening;

### Απάντηση

Compatible move σε ευρύτερο type, π.χ.:

⟨⟩ Plain text

INT → BIGINT  
FLOAT → DOUBLE

χωρίς underlying data rewrite για supported cases. [Databricks Docu...](#)

## Q15

Τι σου δείχνει DESCRIBE DETAIL ;

## Απάντηση

Current table-level metadata όπως format, location, file count, size, partitioning/features. [Databricks Docu...](#)

---

## Q16

Τι σου δείχνει `DESCRIBE HISTORY` ;

## Απάντηση

Versioned operation provenance: version, timestamp, user, operation, parameters, metrics κ.λπ. [Databricks Docu...](#) +1

---

## Q17

Time travel vs restore;

## Απάντηση

Time travel διαβάξει old snapshot. Restore δημιουργεί νέα current version με old state.

---

## Q18

Γιατί history δεν είναι backup;

## Απάντηση

Επειδή retention/VACUUM μπορεί να αφαιρέσει old data files και history έχει finite retention. [Databricks Docu...](#)

---

## Q19

`OPTIMIZE` vs `VACUUM` ;

## Απάντηση

⟨/⟩ Plain text

`OPTIMIZE`

→ reorganize active files/layout

`VACUUM`

→ physically delete expired unreferenced files

---

## Q20

Τι είναι small-file problem;

## Απάντηση

Πάρα πολλά μικρά physical files δημιουργούν metadata/file-open/task-scheduling overhead.

---



## Q21

Τι είναι data skipping;

### Απάντηση

Χρήση file statistics ώστε files που δεν μπορούν να περιέχουν matching rows να μη διαβαστούν.

## Q22

Τι είναι deletion vector;

### Απάντηση

Metadata structure που marks specific rows ως logically deleted/modified ώστε να αποφεύγεται immediate full Parquet rewrite.

[Databricks Docu...](#)

## Q23

Γιατί liquid clustering είναι διαφορετικό από static partitioning;

### Απάντηση

Τα clustering keys μπορούν να εξελιχθούν χωρίς να χρειάζεται να redesign/rewrite άμεσα ολόκληρο historical layout, και το clustering εφαρμόζεται incrementally. [Databricks Docu...](#)

## Q24

Τι κάνει predictive optimization;

### Απάντηση

Για Unity Catalog managed tables μπορεί να τρέχει αυτόματα:

`</>` Plain text

```
OPTIMIZE
VACUUM
ANALYZE
```

όταν κρίνει ότι χρειάζεται. [Databricks Docu...](#)

## Q25 — interview

"Why is Delta Lake more than Parquet?"

Καλή απάντηση:

Parquet defines how columnar data is stored in files. Delta adds versioned transactional table semantics over those files, including ACID writes, snapshots, DML such as MERGE/UPDATE/DELETE, schema enforcement and evolution, history, time travel, concurrency control, and physical optimization features.

## Πρακτικό assignment Phase 2

Αν θέλεις να το εξασκήσεις σαν πραγματικό engineer, χτίσε αυτά:

</> Plain text

```
ecommerce.bronze.orders_raw
ecommerce.staging.orders_updates
ecommerce.silver.orders
```

Κάνε διαδοχικά:

</> Plain text

1. CREATE TABLE
2. INSERT initial rows
3. append new rows
4. UPDATE one row
5. DELETE one row
6. MERGE updates + inserts
7. intentionally duplicate source key
8. deduplicate with ROW\_NUMBER
9. run MERGE again to prove idempotency
10. add new column with mergeSchema
11. DESCRIBE DETAIL
12. DESCRIBE HISTORY
13. query VERSION AS OF
14. RESTORE
15. create many small files
16. OPTIMIZE
17. inspect numFiles before/after
18. enable clustering
19. OPTIMIZE FULL
20. inspect behavior/history

Αυτό ακριβώς καλύπτει το roadmap της Phase 2: SQL DML, PySpark/Delta APIs, overwrite semantics, idempotent MERGE/CDC, schema evolution, time travel, deletion vectors και physical optimization. [databricks\\_mastery\\_phased\\_plan](#)

Σύμφωνα με τη σειρά του roadmap, **το επόμενο είναι Phase 3 — Unity Catalog Mastery**: metastore → catalog → schema → managed/external tables → volumes → storage credentials/external locations → grants/ownership → service principals → row filters/masks/ABAC → lineage και systematic PERMISSION\_DENIED debugging.

Monday 12:59 AM

[Continue to next phase](#)

Worked for 18s

## Phase 3 — Unity Catalog Mastery

Προχωράμε ακριβώς στη **Phase 3** του roadmap σου: στόχος είναι να καταλάβεις storage, namespace, governance, identities και permissions σαν **ένα ενιαίο σύστημα**, όχι σαν άσχετα menu items. Το roadmap ορίζει ρητά ότι εδώ πρέπει να καλύψουμε metastore, catalog, schema, managed/external tables, volumes, storage credentials, external locations, privileges, ownership, groups/service principals, ABAC, lineage, Delta Sharing, Lakehouse Federation και permission debugging.